

OtomotUZplus

1.0

Wygenerowano przez Doxygen 1.9.8

1 Indeks przestrzeni nazw	1
1.1 Package List	1
2 Indeks plików	2
2.1 Lista plików	2
3 Dokumentacja przestrzeni nazw	4
3.1 Pakiet AppCompatActivity	4
3.1.1 Dokumentacja funkcji	4
3.2 Pakiet BrandGold	4
3.2.1 Dokumentacja zmiennych	5
3.3 Pakiet CarClusterOverlay	7
3.3.1 Dokumentacja zmiennych	7
3.4 Pakiet ComponentActivity	7
3.4.1 Dokumentacja funkcji	7
3.4.2 Dokumentacja zmiennych	8
3.5 Pakiet context	8
3.5.1 Dokumentacja funkcji	10
3.5.2 Dokumentacja zmiennych	12
3.6 Pakiet DarkColorScheme	19
3.6.1 Dokumentacja zmiennych	19
3.7 Pakiet favoriteItems	20
3.7.1 Dokumentacja zmiennych	20
3.8 Pakiet FirebaseMessagingService	20
3.8.1 Dokumentacja funkcji	21
3.9 Pakiet FirebaseRepository	21
3.9.1 Dokumentacja funkcji	22
3.9.2 Dokumentacja zmiennych	25
3.10 Pakiet ListingCardData	25
3.10.1 Dokumentacja zmiennych	26
3.11 Pakiet modifier	28
3.12 Pakiet name	28
3.13 Pakiet NotificationHelper	28
3.13.1 Dokumentacja funkcji	28
3.14 Pakiet postalCode	29
3.15 Pakiet PreferenceManager	29
3.15.1 Dokumentacja funkcji	29
3.15.2 Dokumentacja zmiennych	31
3.16 Pakiet query	32
3.16.1 Dokumentacja zmiennych	32
3.17 Pakiet SearchFilters	34
3.17.1 Dokumentacja zmiennych	34
3.18 Pakiet selectedTab	36

3.18.1 Dokumentacja funkcji	36
3.18.2 Dokumentacja zmiennych	37
3.19 Pakiet strings	38
3.20 Pakiet Typography	38
4 Dokumentacja plików	38
4.1 Dokumentacja pliku app/src/main/AndroidManifest.xml	38
4.2 AndroidManifest.xml	38
4.3 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/data/FirebaseRepository.kt	39
4.3.1 Opis szczegółowy	40
4.4 FirebaseRepository.kt	40
4.5 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/data/NominatimGeocoder.kt	43
4.5.1 Opis szczegółowy	44
4.5.2 Dokumentacja funkcji	44
4.5.3 Dokumentacja zmiennych	45
4.6 NominatimGeocoder.kt	46
4.7 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/data/PreferenceManager.kt	47
4.7.1 Opis szczegółowy	47
4.8 PreferenceManager.kt	47
4.9 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/fcm/MyFirebaseMessagingService.kt	48
4.9.1 Opis szczegółowy	49
4.10 MyFirebaseMessagingService.kt	49
4.11 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/LoginActivity.kt	50
4.11.1 Opis szczegółowy	50
4.12 LoginActivity.kt	50
4.13 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/MainActivity.kt	51
4.13.1 Opis szczegółowy	52
4.13.2 Dokumentacja funkcji	52
4.13.3 Dokumentacja zmiennych	52
4.14 MainActivity.kt	56
4.15 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/models/CarAd.kt	61
4.15.1 Opis szczegółowy	61
4.16 CarAd.kt	61
4.17 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/RegisterActivity.kt	62
4.17.1 Opis szczegółowy	62
4.18 RegisterActivity.kt	63
4.19 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/ui/components/AdmobBanner.kt	64
4.19.1 Opis szczegółowy	64
4.20 AdmobBanner.kt	64
4.21 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/ui/components/CommonComponents.kt	65
4.21.1 Opis szczegółowy	65
4.22 CommonComponents.kt	65

4.23 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/components/ListingCard.kt</code> . .	66
4.23.1 Opis szczegółowy	67
4.24 <code>ListingCard.kt</code>	67
4.25 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/models/AppStrings.kt</code>	69
4.25.1 Opis szczegółowy	69
4.26 <code>AppStrings.kt</code>	70
4.27 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/models/ListingCatalog.kt</code> . . .	74
4.27.1 Opis szczegółowy	74
4.28 <code>ListingCatalog.kt</code>	75
4.29 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/models/ServerValue↔Translations.kt</code>	76
4.29.1 Opis szczegółowy	77
4.29.2 Dokumentacja funkcji	77
4.30 <code>ServerValueTranslations.kt</code>	78
4.31 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/navigation/NavDestinations.kt</code>	79
4.31.1 Opis szczegółowy	79
4.32 <code>NavDestinations.kt</code>	80
4.33 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/screens/add/AddScreen.kt</code> . .	80
4.33.1 Opis szczegółowy	81
4.34 <code>AddScreen.kt</code>	81
4.35 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/screens/details/AdDetail↔Screen.kt</code>	88
4.35.1 Opis szczegółowy	88
4.36 <code>AdDetailScreen.kt</code>	89
4.37 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/screens/favorites/Favorites↔Screen.kt</code>	93
4.37.1 Opis szczegółowy	93
4.38 <code>FavoritesScreen.kt</code>	93
4.39 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/screens/home/HomeScreen.kt</code>	95
4.39.1 Opis szczegółowy	96
4.40 <code>HomeScreen.kt</code>	96
4.41 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/screens/profile/ProfileScreen.kt</code>	101
4.41.1 Opis szczegółowy	102
4.42 <code>ProfileScreen.kt</code>	102
4.43 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/screens/search/CarCluster↔Overlay.kt</code>	106
4.43.1 Opis szczegółowy	106
4.43.2 Dokumentacja funkcji	106
4.43.3 Dokumentacja zmiennych	107
4.44 <code>CarClusterOverlay.kt</code>	107
4.45 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/screens/search/CarMapView.kt</code>	109
4.45.1 Opis szczegółowy	110
4.46 <code>CarMapView.kt</code>	110

4.47 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/screens/search/SearchScreen.kt</code>	112
4.47.1 Opis szczegółowy	113
4.47.2 Dokumentacja funkcji	113
4.47.3 Dokumentacja zmiennych	114
4.48 <code>SearchScreen.kt</code>	116
4.49 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/screens/settings/SettingsScreen.kt</code>	124
4.49.1 Opis szczegółowy	125
4.50 <code>SettingsScreen.kt</code>	125
4.51 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/theme/Color.kt</code>	127
4.51.1 Opis szczegółowy	128
4.52 <code>Color.kt</code>	128
4.53 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/theme/Theme.kt</code>	129
4.53.1 Opis szczegółowy	129
4.54 <code>Theme.kt</code>	129
4.55 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/ui/theme/Type.kt</code>	130
4.55.1 Opis szczegółowy	130
4.56 <code>Type.kt</code>	130
4.57 Dokumentacja pliku <code>app/src/main/java/com/example/otomotuzplus/utis/NotificationHelper.kt</code>	131
4.57.1 Opis szczegółowy	131
4.58 <code>NotificationHelper.kt</code>	131
Indeks	133

1 Indeks przestrzeni nazw

1.1 Package List

Oto lista pakietów wraz z krótkim opisem (o ile jest dostępny):

<code>AppCompatActivity</code>	4
<code>BrandGold</code>	4
<code>CarClusterOverlay</code>	7
<code>ComponentActivity</code>	7
<code>context</code>	8
<code>DarkColorScheme</code>	19
<code>favoriteItems</code>	20
<code>FirebaseMessagingService</code>	20
<code>FirebaseRepository</code>	21
<code>ListingCardData</code>	25

modifier	28
name	28
NotificationHelper	28
postalCode	29
PreferenceManager	29
query	32
SearchFilters	34
selectedTab	36
strings	38
Typography	38

2 Indeks plików

2.1 Lista plików

Tutaj znajduje się lista wszystkich plików z ich krótkimi opisami:

app/src/main/AndroidManifest.xml	38
app/src/main/java/com/example/otomotuzplus/LoginActivity.kt Ekran logowania – punkt wejściowy aplikacji, uwierzytelnianie przez Firebase Auth	50
app/src/main/java/com/example/otomotuzplus/MainActivity.kt Główna Activity po zalogowaniu – korzeń drzewa Compose i zarządca nawigacji stanowej	51
app/src/main/java/com/example/otomotuzplus/RegisterActivity.kt Ekran rejestracji nowego konta użytkownika przez Firebase Auth	62
app/src/main/java/com/example/otomotuzplus/data/FirebaseRepository.kt Warstwa danych – dostęp do Firestore, Firebase Storage i FCM	39
app/src/main/java/com/example/otomotuzplus/data/NominatimGeocoder.kt Funkcje zawieszające do geokodowania polskich lokalizacji przez REST API OpenStreetMap Nominatim	43
app/src/main/java/com/example/otomotuzplus/data/PreferenceManager.kt Wrapper SharedPreferences przechowujący motyw i język aplikacji	47
app/src/main/java/com/example/otomotuzplus/fcm/MyFirebaseMessagingService.kt Serwis FCM odbierający przychodzące wiadomości push	48
app/src/main/java/com/example/otomotuzplus/models/CarAd.kt Model dokumentu Firestore reprezentujący ogłoszenie pojazdu	61
app/src/main/java/com/example/otomotuzplus/ui/components/AdmobBanner.kt Komponent baneru reklamowego AdMob	64
app/src/main/java/com/example/otomotuzplus/ui/components/CommonComponents.kt Wspólne komponenty UI: nagłówek ekranu, placeholder, element ustawień	65

<code>app/src/main/java/com/example/otomotuzplus/ui/components/ListingCard.kt</code> Karta ogłoszenia pojazdu używana na ekranach Home, Search i Favorites	66
<code>app/src/main/java/com/example/otomotuzplus/ui/models/AppStrings.kt</code> Interfejs i implementacje i18n – polskie i angielskie ciągi UI	69
<code>app/src/main/java/com/example/otomotuzplus/ui/models/ListingCatalog.kt</code> Katalogi statyczne: marki, modele, rodzaje paliwa; przykładowe dane	74
<code>app/src/main/java/com/example/otomotuzplus/ui/models/ServerValueTranslations.kt</code> Funkcje pomocnicze mapujące surowe wartości pól Firestore na zlokalizowane ciągi wyświetlania dla rodzaju paliwa i skrzyni biegów	76
<code>app/src/main/java/com/example/otomotuzplus/ui/navigation/NavDestinations.kt</code> Enum nawigacyjny AppDestinations i model elementu paska nawigacji	79
<code>app/src/main/java/com/example/otomotuzplus/ui/screens/add/AddScreen.kt</code> Ekran tworzenia nowego ogłoszenia pojazdu z geokodowaniem i przesyłaniem zdjęć	80
<code>app/src/main/java/com/example/otomotuzplus/ui/screens/details/AdDetailScreen.kt</code> Pełnoekranowy widok szczegółów ogłoszenia z gestowym ujawnianiem kontaktu	88
<code>app/src/main/java/com/example/otomotuzplus/ui/screens/favorites/FavoritesScreen.kt</code> Ekran ulubionych ogłoszeń filtrowanych z bazy Firestore	93
<code>app/src/main/java/com/example/otomotuzplus/ui/screens/home/HomeScreen.kt</code> Ekran główny – karuzela marek i lista najnowszych ogłoszeń	95
<code>app/src/main/java/com/example/otomotuzplus/ui/screens/profile/ProfileScreen.kt</code> Ekran profilu użytkownika z zakładkami Moje ogłoszenia i Historia	101
<code>app/src/main/java/com/example/otomotuzplus/ui/screens/search/CarClusterOverlay.kt</code> Nakładka OSMDroid renderująca znaczniki ogłoszeń z grupowaniem siatkowym	106
<code>app/src/main/java/com/example/otomotuzplus/ui/screens/search/CarMapView.kt</code> Komponent Compose opakowujący mapę OSMDroid ze zgrupowanymi znacznikami	109
<code>app/src/main/java/com/example/otomotuzplus/ui/screens/search/SearchScreen.kt</code> Ekran wyszukiwania i przeglądania ogłoszeń z filtrami i widokiem mapy	112
<code>app/src/main/java/com/example/otomotuzplus/ui/screens/settings/SettingsScreen.kt</code> Ekran ustawień: motyw, język, powiadomienia i wylogowanie	124
<code>app/src/main/java/com/example/otomotuzplus/ui/theme/Color.kt</code> Paleta kolorów Material3 aplikacji OtomotUZplus	127
<code>app/src/main/java/com/example/otomotuzplus/ui/theme/Theme.kt</code> Konfiguracja motywu Material3 z obsługą jasnego, ciemnego i systemowego trybu	129
<code>app/src/main/java/com/example/otomotuzplus/ui/theme/Type.kt</code> Typografia Material3 aplikacji OtomotUZplus	130
<code>app/src/main/java/com/example/otomotuzplus/utils/NotificationHelper.kt</code> Singleton pomocniczy do tworzenia kanału i wysyłania lokalnych powiadomień	131

3 Dokumentacja przestrzeni nazw

3.1 Pakiet AppCompatActivity

Funkcje

- Object [onCreate](#) (Object savedInstanceState)
- Object [onStart](#) ()
- Object [otworzGlowneWnetrzeAppki](#) ()

3.1.1 Dokumentacja funkcji

onCreate()

```
Object AppCompatActivity.onCreate (
    Object savedInstanceState )
```

Definicja w linii 31 pliku [LoginActivity.kt](#).

onStart()

```
Object AppCompatActivity.onStart ( )
```

Definicja w linii 34 pliku [LoginActivity.kt](#).

otworzGlowneWnetrzeAppki()

```
Object AppCompatActivity.otworzGlowneWnetrzeAppki ( )
```

Definicja w linii 37 pliku [LoginActivity.kt](#).

3.2 Pakiet BrandGold

Zmienne

- Object [ActionBlue](#)
- Object [DarkText](#)
- Object [DarkSlate950](#)
- Object [DarkSlate900](#)
- Object [DarkSlate800](#)
- Object [DarkSlate700](#)
- Object [Slate400](#)
- Object [Slate300](#)
- Object [LightBackground](#)
- Object [Slate100](#)
- Object [Slate200](#)
- Object [SuccessGreen](#)
- Object [ErrorRed](#)
- Object [WarningOrange](#)

3.2.1 Dokumentacja zmiennych

ActionBlue

`Object BrandGold.ActionBlue`

Definicja w linii 14 pliku [Color.kt](#).

DarkSlate700

`Object BrandGold.DarkSlate700`

Definicja w linii 20 pliku [Color.kt](#).

DarkSlate800

`Object BrandGold.DarkSlate800`

Definicja w linii 19 pliku [Color.kt](#).

DarkSlate900

`Object BrandGold.DarkSlate900`

Definicja w linii 18 pliku [Color.kt](#).

DarkSlate950

`Object BrandGold.DarkSlate950`

Definicja w linii 17 pliku [Color.kt](#).

DarkText

`Object BrandGold.DarkText`

Definicja w linii 15 pliku [Color.kt](#).

ErrorRed

`Object BrandGold.ErrorRed`

Definicja w linii 29 pliku [Color.kt](#).

LightBackground

`Object BrandGold.LightBackground`

Definicja w linii 24 pliku [Color.kt](#).

Slate100

`Object BrandGold.Slate100`

Definicja w linii 25 pliku [Color.kt](#).

Slate200

`Object BrandGold.Slate200`

Definicja w linii 26 pliku [Color.kt](#).

Slate300

`Object BrandGold.Slate300`

Definicja w linii 22 pliku [Color.kt](#).

Slate400

`Object BrandGold.Slate400`

Definicja w linii 21 pliku [Color.kt](#).

SuccessGreen

`Object BrandGold.SuccessGreen`

Definicja w linii 28 pliku [Color.kt](#).

WarningOrange

`Object BrandGold.WarningOrange`

Definicja w linii 30 pliku [Color.kt](#).

3.3 Pakiet CarClusterOverlay

Zmienne

- Object [cars](#)
- Object [onSingleCarTap](#)
- Object [onClusterTap](#)

3.3.1 Dokumentacja zmiennych

`cars`

Object `CarClusterOverlay.cars`

Definicja w linii 42 pliku [CarClusterOverlay.kt](#).

`onClusterTap`

Object `CarClusterOverlay.onClusterTap`

Definicja w linii 44 pliku [CarClusterOverlay.kt](#).

`onSingleCarTap`

Object `CarClusterOverlay.onSingleCarTap`

Definicja w linii 43 pliku [CarClusterOverlay.kt](#).

3.4 Pakiet ComponentActivity

Funkcje

- Object [onCreate](#) (Object `savedInstanceState`)

Zmienne

- Object [requestPermissionLauncher](#)
- Object [repository](#)
- Object [db](#)
- Object [hasNotificationPermission](#)

3.4.1 Dokumentacja funkcji

`onCreate()`

```
Object ComponentActivity.onCreate (
    Object savedInstanceState )
```

Definicja w linii 73 pliku [MainActivity.kt](#).

3.4.2 Dokumentacja zmiennych

db

`Object ComponentActivity.db`

Definicja w linii 70 pliku [MainActivity.kt](#).

hasNotificationPermission

`Object ComponentActivity.hasNotificationPermission`

Definicja w linii 71 pliku [MainActivity.kt](#).

repository

`Object ComponentActivity.repository`

Definicja w linii 69 pliku [MainActivity.kt](#).

requestPermissionLauncher

`Object ComponentActivity.requestPermissionLauncher`

Definicja w linii 67 pliku [MainActivity.kt](#).

3.5 Pakiet context

Funkcje

- Object [originalToTransformed](#) (Object offset)
- Object [transformedToOriginal](#) (Object offset)
- Object [customTextFieldColors](#) ()
- Object [createTemplImageUri](#) (Object context)
 - Tworzy plik tymczasowy i zwraca URI [FileProvider] odpowiedni do przekazania do `ActivityResultContracts.TakePicture`.*
- Object [onSensorChanged](#) (Object event)
- Object [onAccuracyChanged](#) (Object sensor, Object accuracy)
- Object [SpecItem](#) (Object icon, Object [title](#), Object subtitle, Object modifier)
 - Pojedynczy wiersz w siatce specyfikacji pojazdu.*
- Object [createUserLocationBitmap](#) (Object context)

Zmienne

- Object [repository](#)
- Object [coroutineScope](#)
- Object [title](#)
- Object [priceText](#)
- Object [locationText](#)
- Object [postalCode](#)
- Object [year](#)
- Object [mileageText](#)
- Object [fuelText](#)
- Object [gearboxText](#)
- Object [engineCapacity](#)
- Object [powerText](#)
- Object [phoneNumber](#)
- Object [isUploading](#)
- Object [selectedImageUri](#)
- Object [showPhotoSourceDialog](#)
- Object [templImageUri](#)
- Object [cameraLauncher](#)
- Object [multiplePhotoPickerLauncher](#)
- Object [s](#)
- Object [formatted](#)
- Object [isPetrol](#)
- Object [isDiesel](#)
- Object [isManual](#)
- Object [isAuto](#)
- Object [uri](#)
- Object [validPostal](#)
- Object [coords](#)
- Object [currentUser](#)
- Object [sellerId](#)
- Object [sellerEmail](#)
- Object [newCar](#)
- Object [msg](#)
- Object [location](#)
- Object [price](#)
- Object [seller](#)
- Object [phone](#)
- Object [isContactVisible](#)
- Object [sensorManager](#)
- Object [accelerometer](#)
- Object [listener](#)
- Object [specs](#)
- Object [pagerState](#)
- Object [hasLocationPermission](#)
- Object [permissionLauncher](#)
- Object [mapView](#)
- Object [myLocationOverlayHolder](#)
- Object [lm](#)
- Object [loc](#)
- Object [locationBitmap](#)
- Object [overlay](#)
- Object [showLanguageDialog](#)
- Object [showThemeDialog](#)
- Object [openNotificationSettings](#)
- Object [intent](#)
- Object [label](#)
- Object [themeLabel](#)

3.5.1 Dokumentacja funkcji

createTempImageUri()

```
Object context.createTempImageUri (
    Object context )
```

Tworzy plik tymczasowy i zwraca URI [FileProvider] odpowiedni do przekazania do `ActivityResultContracts.TakePicture`.

Plik jest tworzony w [Context.externalCacheDir] i oznaczony do usunięcia przy zamknięciu JVM. Po zapisaniu przez aplikację aparatu URI jest dodawane do listy `selectedImageUris` wewnątrz [AddScreen].

Parametry

<code>context</code>	Kontekst Activity lub aplikacji.
----------------------	----------------------------------

Zwraca

URI `content://` obsługiwane przez FileProvider `com.example.otomotuzplus.provider`.

Definicja w linii 173 pliku [AddScreen.kt](#).

Odwołuje się do [createTempImageUri\(\)](#).

Odwołania w [createTempImageUri\(\)](#).

createUserLocationBitmap()

```
Object context.createUserLocationBitmap (
    Object context )
```

Definicja w linii 115 pliku [CarMapView.kt](#).

Odwołuje się do [createUserLocationBitmap\(\)](#).

Odwołania w [createUserLocationBitmap\(\)](#).

customTextFieldColors()

```
Object context.customTextFieldColors ( )
```

Definicja w linii 160 pliku [AddScreen.kt](#).

Odwołuje się do [customTextFieldColors\(\)](#).

Odwołania w [customTextFieldColors\(\)](#).

onAccuracyChanged()

```
Object context.onAccuracyChanged (
    Object sensor,
    Object accuracy )
```

Definicja w linii 83 pliku [AdDetailScreen.kt](#).

onSensorChanged()

```
Object context.onSensorChanged (
    Object event )
```

Definicja w linii 81 pliku [AdDetailScreen.kt](#).

originalToTransformed()

```
Object context.originalToTransformed (
    Object offset )
```

Definicja w linii 102 pliku [AddScreen.kt](#).

Odwołuje się do [originalToTransformed\(\)](#).

Odwołania w [originalToTransformed\(\)](#).

SpecItem()

```
Object context.SpecItem (
    Object icon,
    Object title,
    Object subtitle,
    Object modifier )
```

Pojedynczy wiersz w siatce specyfikacji pojazdu.

Wyświetla wiodącą ikonę, pogrubioną wartość główną ([title]) oraz etykietę pomocniczą ([subtitle]) poniżej.

Parametry

<i>icon</i>	Ikona reprezentująca kategorię specyfikacji.
<i>title</i>	Wartość główna (np. "15 000 km").
<i>subtitle</i>	Etykieta kategorii (np. "Przebieg").
<i>modifier</i>	Opcjonalny [Modifier] stosowany do kontenera wiersza.

Definicja w linii 139 pliku [AdDetailScreen.kt](#).

Odwołuje się do [SpecItem\(\)](#).

Odwołania w [SpecItem\(\)](#).

transformedToOriginal()

```
Object context.transformedToOriginal (
    Object offset )
```

Definicja w linii 104 pliku [AddScreen.kt](#).

Odwołuje się do [transformedToOriginal\(\)](#).

Odwołania w [transformedToOriginal\(\)](#).

3.5.2 Dokumentacja zmiennych**accelerometer**

```
Object context.accelerometer
```

Definicja w linii 78 pliku [AdDetailScreen.kt](#).

cameraLauncher

```
Object context.cameraLauncher
```

Definicja w linii 87 pliku [AddScreen.kt](#).

coords

```
Object context.coords
```

Definicja w linii 139 pliku [AddScreen.kt](#).

coroutineScope

```
Object context.coroutineScope
```

Definicja w linii 71 pliku [AddScreen.kt](#).

currentUser

```
Object context.currentUser
```

Definicja w linii 140 pliku [AddScreen.kt](#).

engineCapacity

```
Object context.engineCapacity
```

Definicja w linii 80 pliku [AddScreen.kt](#).

formatted

`Object context.formatted`

Definicja w linii 101 pliku [AddScreen.kt](#).

fuelText

`Object context.fuelText`

Definicja w linii 78 pliku [AddScreen.kt](#).

gearboxText

`Object context.gearboxText`

Definicja w linii 79 pliku [AddScreen.kt](#).

hasLocationPermission

`Object context.hasLocationPermission`

Definicja w linii 78 pliku [CarMapView.kt](#).

intent

`Object context.intent`

Definicja w linii 53 pliku [SettingsScreen.kt](#).

isAuto

`Object context.isAuto`

Definicja w linii 120 pliku [AddScreen.kt](#).

isContactVisible

`Object context.isContactVisible`

Definicja w linii 75 pliku [AdDetailScreen.kt](#).

isDiesel

`Object context.isDiesel`

Definicja w linii 112 pliku [AddScreen.kt](#).

isManual

`Object context.isManual`

Definicja w linii 117 pliku [AddScreen.kt](#).

isPetrol

`Object context.isPetrol`

Definicja w linii 109 pliku [AddScreen.kt](#).

isUploading

`Object context.isUploading`

Definicja w linii 83 pliku [AddScreen.kt](#).

label

`Object context.label`

Definicja w linii 62 pliku [SettingsScreen.kt](#).

listener

`Object context.listener`

Definicja w linii 80 pliku [AdDetailScreen.kt](#).

lm

`Object context.lm`

Definicja w linii 93 pliku [CarMapView.kt](#).

loc

`Object context.loc`

Definicja w linii 94 pliku [CarMapView.kt](#).

location

`Object context.location`

Definicja w linii 70 pliku [AdDetailScreen.kt](#).

locationBitmap

```
Object context.locationBitmap
```

Definicja w linii 101 pliku [CarMapView.kt](#).

locationText

```
Object context.locationText
```

Definicja w linii 74 pliku [AddScreen.kt](#).

mapView

```
Object context.mapView
```

Definicja w linii 86 pliku [CarMapView.kt](#).

mileageText

```
Object context.mileageText
```

Definicja w linii 77 pliku [AddScreen.kt](#).

msg

```
Object context.msg
```

Definicja w linii 144 pliku [AddScreen.kt](#).

multiplePhotoPickerLauncher

```
Object context.multiplePhotoPickerLauncher
```

Definicja w linii 90 pliku [AddScreen.kt](#).

myLocationOverlayHolder

```
Object context.myLocationOverlayHolder
```

Definicja w linii 91 pliku [CarMapView.kt](#).

newCar

```
Object context.newCar
```

Definicja w linii 143 pliku [AddScreen.kt](#).

openNotificationSettings

```
Object context.openNotificationSettings
```

Definicja w linii 52 pliku [SettingsScreen.kt](#).

overlay

```
Object context.overlay
```

Definicja w linii 102 pliku [CarMapView.kt](#).

pagerState

```
Object context.pagerState
```

Definicja w linii 96 pliku [AdDetailScreen.kt](#).

permissionLauncher

```
Object context.permissionLauncher
```

Definicja w linii 81 pliku [CarMapView.kt](#).

phone

```
Object context.phone
```

Definicja w linii 73 pliku [AdDetailScreen.kt](#).

phoneNumber

```
Object context.phoneNumber
```

Definicja w linii 82 pliku [AddScreen.kt](#).

postalCode

```
Object context.postalCode
```

Definicja w linii 75 pliku [AddScreen.kt](#).

powerText

```
Object context.powerText
```

Definicja w linii 81 pliku [AddScreen.kt](#).

price

```
Object context.price
```

Definicja w linii 71 pliku [AdDetailScreen.kt](#).

priceText

```
Object context.priceText
```

Definicja w linii 73 pliku [AddScreen.kt](#).

repository

```
Object context.repository
```

Definicja w linii 70 pliku [AddScreen.kt](#).

s

```
Object context.s
```

Definicja w linii 100 pliku [AddScreen.kt](#).

selectedImageUri

```
Object context.selectedImageUri
```

Definicja w linii 84 pliku [AddScreen.kt](#).

seller

```
Object context.seller
```

Definicja w linii 72 pliku [AdDetailScreen.kt](#).

sellerEmail

```
Object context.sellerEmail
```

Definicja w linii 142 pliku [AddScreen.kt](#).

sellerId

```
Object context.sellerId
```

Definicja w linii 141 pliku [AddScreen.kt](#).

sensorManager

```
Object context.sensorManager
```

Definicja w linii 77 pliku [AddDetailScreen.kt](#).

showLanguageDialog

```
Object context.showLanguageDialog
```

Definicja w linii 50 pliku [SettingsScreen.kt](#).

showPhotoSourceDialog

```
Object context.showPhotoSourceDialog
```

Definicja w linii 85 pliku [AddScreen.kt](#).

showThemeDialog

```
Object context.showThemeDialog
```

Definicja w linii 51 pliku [SettingsScreen.kt](#).

specs

```
Object context.specs
```

Definicja w linii 91 pliku [AddDetailScreen.kt](#).

tempImageUri

```
Object context.tempImageUri
```

Definicja w linii 86 pliku [AddScreen.kt](#).

themeLabel

```
Object context.themeLabel
```

Definicja w linii 72 pliku [SettingsScreen.kt](#).

title

```
Object context.title
```

Definicja w linii 72 pliku [AddScreen.kt](#).

uri

`Object context.uri`

Definicja w linii 131 pliku [AddScreen.kt](#).

validPostal

`Object context.validPostal`

Definicja w linii 138 pliku [AddScreen.kt](#).

year

`Object context.year`

Definicja w linii 76 pliku [AddScreen.kt](#).

3.6 Pakiet DarkColorScheme

Zmienne

- Object [LightColorScheme](#)
- Object [colorScheme](#)
- Object [view](#)
- Object [window](#)

3.6.1 Dokumentacja zmiennych

colorScheme

`Object DarkColorScheme.colorScheme`

Definicja w linii 21 pliku [Theme.kt](#).

LightColorScheme

`Object DarkColorScheme.LightColorScheme`

Definicja w linii 19 pliku [Theme.kt](#).

view

`Object DarkColorScheme.view`

Definicja w linii 22 pliku [Theme.kt](#).

window

Object `DarkColorScheme.window`

Definicja w linii [23](#) pliku [Theme.kt](#).

3.7 Pakiet favoriteItems

Zmienne

- Object [key](#)
- Object [carKey](#)
- Object [value](#)

3.7.1 Dokumentacja zmiennych

carKey

Object `favoriteItems.carKey`

Definicja w linii [60](#) pliku [FavoritesScreen.kt](#).

key

Object `favoriteItems.key`

Definicja w linii [52](#) pliku [FavoritesScreen.kt](#).

value

Object `favoriteItems.value`

Definicja w linii [69](#) pliku [FavoritesScreen.kt](#).

3.8 Pakiet FirebaseMessagingService

Funkcje

- Object [onMessageReceived](#) (Object `remoteMessage`)
- Object [sendNotification](#) (Object `title`, Object `messageBody`)
- Object [onNewToken](#) (Object `token`)

3.8.1 Dokumentacja funkcji

onMessageReceived()

```
Object FirebaseMessagingService.onMessageReceived (
    Object remoteMessage )
```

Definicja w linii 32 pliku [MyFirebaseMessagingService.kt](#).

onNewToken()

```
Object FirebaseMessagingService.onNewToken (
    Object token )
```

Definicja w linii 38 pliku [MyFirebaseMessagingService.kt](#).

sendNotification()

```
Object FirebaseMessagingService.sendNotification (
    Object title,
    Object messageBody )
```

Definicja w linii 35 pliku [MyFirebaseMessagingService.kt](#).

3.9 Pakiet FirebaseRepository

Funkcje

- Object [uploadTestCar](#) (Object onSuccess)
Zapisuje zakodowane na stałe testowe ogłoszenie do Firestore.
- Object [getAllCars](#) (Object onSuccess)
Pobiera jednorazowo wszystkie dokumenty z kolekcji `listings`.
- Object [addCar](#) (Object car, Object onSuccess)
Dodaje nowy dokument ogłoszenia do kolekcji `listings`.
- Object [observeCars](#) (Object onCarsChanged)
Dodaje nasłuchiwanie migawek Firestore w czasie rzeczywistym do kolekcji `listings`.
- Object [uploadImages](#) (Object uris, Object onComplete)
Przesyła partię lokalnych URI obrazów do Firebase Storage w katalogu `car_images/`.
- Object [updateFcmToken](#) (Object userId, Object token)
Scala bieżący token rejestracji FCM z dokumentem użytkownika w Firestore.
- Object [sendLikeNotification](#) (Object sellerId, Object carTitle)
Zapisuje dokument powiadomienia o polubieniu do kolekcji `notifications`.
- Object [proceedWithSending](#) (Object token, Object carTitle)
Wysyła powiadomienie push FCM bezpośrednio przez API HTTP v1 w wątku w tle.

Zmienne

- Object [db](#)
- Object [storage](#)

3.9.1 Dokumentacja funkcji

addCar()

```
Object FirebaseRepository.addCar (
    Object car,
    Object onSuccess )
```

Dodaje nowy dokument ogłoszenia do kolekcji `listings`.

Parametry

<i>car</i>	Obiekt [CarAd] do zapisania. Pole [CarAd.id] jest ignorowane; Firestore generuje identyfikator dokumentu automatycznie.
<i>onSuccess</i>	Wywoływane po pomyślnym utworzeniu dokumentu.
<i>onFailure</i>	Wywoływane z [Exception] w razie błędu.

Definicja w linii 60 pliku [FirebaseRepository.kt](#).

Odwołuje się do [addCar\(\)](#).

Odwołania w [addCar\(\)](#).

getAllCars()

```
Object FirebaseRepository.getAllCars (
    Object onSuccess )
```

Pobiera jednorazowo wszystkie dokumenty z kolekcji `listings`.

Preferuj `[observeCars]` dla ekranów UI wymagających aktualizacji na żywo.

Parametry

<i>onSuccess</i>	Wywoływane z pełną listą obiektów [CarAd] po sukcesie.
<i>onFailure</i>	Wywoływane z [Exception] w razie błędu.

Definicja w linii 49 pliku [FirebaseRepository.kt](#).

Odwołuje się do [getAllCars\(\)](#).

Odwołania w [getAllCars\(\)](#).

observeCars()

```
Object FirebaseRepository.observeCars (
    Object onCarsChanged )
```

Dołącza nasłuchiwanca migawek Firestore w czasie rzeczywistym do kolekcji `listings`.

Callback wywoływany natychmiast z aktualnym stanem, a następnie przy każdej kolejnej zmianie. Nasłuchiwaniec żyje przez cały czas istnienia komponentu; wywołujący odpowiada za jego usunięcie (implementacja nie udostępnia uchwytu — nasłuchiwaniec powiązany jest z cyklem życia Activity przez `LaunchedEffect`).

Parametry

<code>onCarsChanged</code>	Wywoływane na wątku głównym ze zaktualizowaną listą przy każdej zmianie kolekcji. Wadliwe dokumenty są pomijane.
----------------------------	--

Definicja w linii 74 pliku `FirestoreRepository.kt`.

Odwołuje się do `observeCars()`.

Odwołania w `observeCars()`.

`proceedWithSending()`

```
Object FirestoreRepository.proceedWithSending (
    Object token,
    Object carTitle )
```

Wysyła powiadomienie push FCM bezpośrednio przez API HTTP v1 w wątku w tle.

Pobiera krótkotrwały token dostępu OAuth2 z Firebase Auth, następnie wykonuje żądanie HTTP POST do `fcm.↔googleapis.com`. Operacja typu fire-and-forget; wynik tylko logowany, nigdy nie przekazywany do UI.

Parametry

<code>token</code>	Token rejestracji FCM urządzenia odbiorcy. Jeśli null lub pusty, wywołanie jest ignorowane.
<code>carTitle</code>	Tytuł polubionego ogłoszenia; osadzony w treści wiadomości.

Definicja w linii 124 pliku `FirestoreRepository.kt`.

Odwołuje się do `proceedWithSending()`.

Odwołania w `proceedWithSending()`.

`sendLikeNotification()`

```
Object FirestoreRepository.sendLikeNotification (
    Object sellerId,
    Object carTitle )
```

Zapisuje dokument powiadomienia o polubieniu do kolekcji `notifications`.

`MainActivity` nasłuchuje tej kolekcji przez listener migawek i wyświetla lokalne powiadomienie gdy nowy dokument zostanie dodany dla bieżącego użytkownika.

Parametry

<code>seller↔Id</code>	UID sprzedającego posiadającego polubione ogłoszenie w Firebase Auth.
<code>carTitle</code>	Tytuł polubionego ogłoszenia; umieszczony w treści powiadomienia.

Definicja w linii 110 pliku [FirebaseRepository.kt](#).

Odwołuje się do [sendLikeNotification\(\)](#).

Odwołania w [sendLikeNotification\(\)](#).

updateFcmToken()

```
Object FirebaseRepository.updateFcmToken (
    Object userId,
    Object token )
```

Scala bieżący token rejestracji FCM z dokumentem użytkownika w Firestore.

Używa `SetOptions.merge()` aby zachować pozostałe pola dokumentu `users`. Wywoływane przez `[MyFirebaseMessagingService.onNewToken]` przy każdym odświeżeniu tokenu.

Parametry

<code>userId</code>	UID zalogowanego użytkownika w Firebase Auth.
<code>token</code>	Nowy token rejestracji FCM.

Definicja w linii 99 pliku [FirebaseRepository.kt](#).

Odwołuje się do [updateFcmToken\(\)](#).

Odwołania w [updateFcmToken\(\)](#).

uploadImages()

```
Object FirebaseRepository.uploadImages (
    Object uris,
    Object onComplete )
```

Przesyła partię lokalnych URI obrazów do Firebase Storage w katalogu `car_images/`.

Każdy plik przechowywany pod nazwą UUID. Wszystkie przesyłania działają równolegle, a `[onComplete]` wywołane jest gdy wszystkie zakończą działanie (sukces lub błąd). Nieudane przesyłania poszczególnych plików są pomijane na liście wynikowej.

Parametry

<code>uris</code>	Lokalne URI treści (zdjęcia z galerii lub zrobione aparatem przez <code>FileProvider</code>).
<code>onComplete</code>	Wywoływane z listą adresów URL pobierania po zakończeniu wszystkich przesyłań. Zwraca pustą listę gdy <code>[uris]</code> jest puste.

Definicja w linii 88 pliku [FirebaseRepository.kt](#).

Odwołuje się do [uploadImages\(\)](#).

Odwołania w [uploadImages\(\)](#).

uploadTestCar()

```
Object FirebaseRepository.uploadTestCar (
    Object onSuccess )
```

Zapisuje zakodowane na stałe testowe ogłoszenie do Firestore.

Przeznaczone wyłącznie do celów deweloperskich i debugowania.

Parametry

<code>onSuccess</code>	Wywoływane po pomyślnym zapisaniu dokumentu.
<code>onFailure</code>	Wywoływane z [Exception] gdy zapis się nie powiedzie.

Definicja w linii 38 pliku [FirebaseRepository.kt](#).

Odwołuje się do [uploadTestCar\(\)](#).

Odwołania w [uploadTestCar\(\)](#).

3.9.2 Dokumentacja zmiennych

db

```
Object FirebaseRepository.db
```

Definicja w linii 27 pliku [FirebaseRepository.kt](#).

storage

```
Object FirebaseRepository.storage
```

Definicja w linii 28 pliku [FirebaseRepository.kt](#).

3.10 Pakiet ListingCardData

Zmienne

- Object [title](#)
Karta Material3 wyświetlająca skrócone podsumowanie ogłoszenia pojazdu.
- Object [year](#)
- Object [mileageText](#)
- Object [fuelText](#)
- Object [bodyTypeText](#)
- Object [driveTypeText](#)
- Object [locationText](#)
- Object [priceText](#)
- Object [isFavorite](#)
- Object [onFavoriteClick](#)
- Object [coverImageUrl](#)
- Object [metadataLine1](#)
- Object [metadataLine2](#)
- Object [price](#)

3.10.1 Dokumentacja zmiennych

bodyTypeText

`Object ListingCardData.bodyTypeText`

Definicja w linii 63 pliku [ListingCard.kt](#).

coverImageUrl

`Object ListingCardData.coverImageUrl`

Definicja w linii 69 pliku [ListingCard.kt](#).

driveTypeText

`Object ListingCardData.driveTypeText`

Definicja w linii 64 pliku [ListingCard.kt](#).

fuelText

`Object ListingCardData.fuelText`

Definicja w linii 62 pliku [ListingCard.kt](#).

isFavorite

`Object ListingCardData.isFavorite`

Definicja w linii 67 pliku [ListingCard.kt](#).

locationText

`Object ListingCardData.locationText`

Definicja w linii 65 pliku [ListingCard.kt](#).

metadataLine1

`Object ListingCardData.metadataLine1`

Definicja w linii 81 pliku [ListingCard.kt](#).

metadataLine2

Object ListingCardData.metadataLine2

Definicja w linii 82 pliku [ListingCard.kt](#).

mileageText

Object ListingCardData.mileageText

Definicja w linii 61 pliku [ListingCard.kt](#).

onFavoriteClick

Object ListingCardData.onFavoriteClick

Definicja w linii 68 pliku [ListingCard.kt](#).

price

Object ListingCardData.price

Definicja w linii 83 pliku [ListingCard.kt](#).

priceText

Object ListingCardData.priceText

Definicja w linii 66 pliku [ListingCard.kt](#).

title

Object ListingCardData.title

Karta Material3 wyświetlająca skrócone podsumowanie ogłoszenia pojazdu.

Karta pokazuje zdjęcie okładki (lub tło zastępcze), przycisk przełącznika ulubionych w prawym górnym rogu i dwa wiersze metadanych pod tytułem.

Parametry

<i>item</i>	Dane ogłoszenia do wyświetlenia.
<i>modifier</i>	Opcjonalny [Modifier] stosowany do zewnętrznej [Card].

Definicja w linii 59 pliku [ListingCard.kt](#).

year

`Object ListingCardData.year`

Definicja w linii 60 pliku [ListingCard.kt](#).

3.11 Pakiet modifier

3.12 Pakiet name

3.13 Pakiet NotificationHelper

Funkcje

- Object [createNotificationChannel](#) (Object context, Object strings)
Identyfikator kanału powiadomień współdzielony z [MyFirebaseMessagingService].
- Object [sendNotification](#) (Object context, Object title, Object message)
Wysyła lokalne powiadomienie z podanym tytułem i treścią.

3.13.1 Dokumentacja funkcji

createNotificationChannel()

```
Object NotificationHelper.createNotificationChannel (  
    Object context,  
    Object strings )
```

Identyfikator kanału powiadomień współdzielony z [MyFirebaseMessagingService].

Tworzy kanał powiadomień `offers_channel` (tylko Android 8+).

Bezpieczne do wielokrotnego wywoływania — Android deduplikuje tworzenie kanałów po ID.

Parametry

<i>context</i>	Kontekst aplikacji lub Activity.
<i>strings</i>	Aktywne [AppStrings] dostarczające zlokalizowaną nazwę kanału.

Definicja w linii 36 pliku [NotificationHelper.kt](#).

sendNotification()

```
Object NotificationHelper.sendNotification (  
    Object context,  
    Object title,  
    Object message )
```


Wysyła lokalne powiadomienie z podanym tytułem i treścią.

Używa identyfikatora powiadomienia opartego na znaczniku czasu, aby kolejne wywołania nie nadpisywały się nawzajem. Dotknięcie powiadomienia uruchamia [MainActivity] i czyści stos back.

Parametry

<i>context</i>	Kontekst aplikacji lub Activity.
<i>title</i>	Ciąg tytułu powiadomienia.
<i>message</i>	Tekst treści powiadomienia.

Definicja w linii 50 pliku [NotificationHelper.kt](#).

3.14 Pakiet postalCode

3.15 Pakiet PreferenceManager

Funkcje

- Object [setNotificationsRefused](#) (Object refused)
Zapisuje informację czy użytkownik jawnie odrzucił prośbę o uprawnienie do powiadomień.
- Object [wasNotificationsRefused](#) ()
- Object [setThemeMode](#) (Object mode)
Zapisuje wybrany [ThemeMode].
- Object [getThemeMode](#) ()
- Object [setLanguage](#) (Object lang)
Zapisuje wybrany język interfejsu.
- Object [getLanguage](#) ()

Zmienne

- Object [prefs](#)
- Object [KEY_NOTIFICATIONS_REFUSED](#)

3.15.1 Dokumentacja funkcji

[getLanguage\(\)](#)

```
Object PreferenceManager.getLanguage ( )
```

Zwraca

Zapisany identyfikator języka; domyślnie "Polski" jeśli nie ustawiono.

Definicja w linii 71 pliku [PreferenceManager.kt](#).

Odwołuje się do [getLanguage\(\)](#).

Odwołania w [getLanguage\(\)](#).

getThemeMode()

```
Object PreferenceManager.getThemeMode ( )
```

Zwraca

Zapisany [ThemeMode]; domyślnie [ThemeMode.SYSTEM] jeśli nie ustawiono.

Definicja w linii 58 pliku [PreferenceManager.kt](#).

Odwołuje się do [getThemeMode\(\)](#).

Odwołania w [getThemeMode\(\)](#).

setLanguage()

```
Object PreferenceManager.setLanguage (
    Object lang )
```

Zapisuje wybrany język interfejsu.

Parametry

<i>lang</i>	Identyfikator języka — "Polski" lub "English".
-------------	--

Definicja w linii 65 pliku [PreferenceManager.kt](#).

Odwołuje się do [setLanguage\(\)](#).

Odwołania w [setLanguage\(\)](#).

setNotificationsRefused()

```
Object PreferenceManager.setNotificationsRefused (
    Object refused )
```

Zapisuje informację czy użytkownik jawnie odrzucił prośbę o uprawnienie do powiadomień.

Gdy `true`, okno dialogowe uzasadnienia w [OtomotUZplusApp] jest pomijane przez cały pozostały czas życia aplikacji na tym urządzeniu.

Parametry

<i>refused</i>	<code>true</code> jeśli użytkownik kliknął "Może później" w oknie uprawnień.
----------------	--

Definicja w linii 40 pliku [PreferenceManager.kt](#).

Odwołuje się do [setNotificationsRefused\(\)](#).

Odwołania w [setNotificationsRefused\(\)](#).

setThemeMode()

```
Object PreferenceManager.setThemeMode (
    Object mode )
```

Zapisuje wybrany [ThemeMode].

Parametry

<i>mode</i>	Motyw do zastosowania przy następnej rekompozycji.
-------------	--

Definicja w linii 52 pliku [PreferenceManager.kt](#).

Odwołuje się do [setThemeMode\(\)](#).

Odwołania w [setThemeMode\(\)](#).

wasNotificationsRefused()

```
Object PreferenceManager.wasNotificationsRefused ( )
```

Zwraca

`true` jeśli użytkownik wcześniej odrzucił prośbę o uprawnienie do powiadomień.

Definicja w linii 46 pliku [PreferenceManager.kt](#).

Odwołuje się do [wasNotificationsRefused\(\)](#).

Odwołania w [wasNotificationsRefused\(\)](#).

3.15.2 Dokumentacja zmiennych**KEY_NOTIFICATIONS_REFUSED**

```
Object PreferenceManager.KEY_NOTIFICATIONS_REFUSED
```

Definicja w linii 30 pliku [PreferenceManager.kt](#).

prefs

```
Object PreferenceManager.prefs
```

Definicja w linii 29 pliku [PreferenceManager.kt](#).

3.16 Pakiet query

Zmienne

- Object [context](#)
- Object [brands](#)
- Object [arrivals](#)
- Object [originalCarFromDb](#)
- Object [carKey](#)
- Object [config](#)
- Object [screenWidth](#)
- Object [placeholderSize](#)
- Object [textSize](#)
- Object [tileHeight](#)
- Object [padding](#)
- Object [titleSize](#)
- Object [subtitleSize](#)
- Object [spacingIconToTitle](#)
- Object [spacingTitleToSubtitle](#)
- Object [value](#)

3.16.1 Dokumentacja zmiennych

arrivals

Object `query.arrivals`

Definicja w linii [107](#) pliku [HomeScreen.kt](#).

brands

Object `query.brands`

Definicja w linii [104](#) pliku [HomeScreen.kt](#).

carKey

Object `query.carKey`

Definicja w linii [130](#) pliku [HomeScreen.kt](#).

config

Object `query.config`

Definicja w linii [145](#) pliku [HomeScreen.kt](#).

context

```
Object query.context
```

Definicja w linii 101 pliku [HomeScreen.kt](#).

originalCarFromDb

```
Object query.originalCarFromDb
```

Definicja w linii 128 pliku [HomeScreen.kt](#).

padding

```
Object query.padding
```

Definicja w linii 156 pliku [HomeScreen.kt](#).

placeholderSize

```
Object query.placeholderSize
```

Definicja w linii 147 pliku [HomeScreen.kt](#).

screenWidth

```
Object query.screenWidth
```

Definicja w linii 146 pliku [HomeScreen.kt](#).

spacingIconToTitle

```
Object query.spacingIconToTitle
```

Definicja w linii 159 pliku [HomeScreen.kt](#).

spacingTitleToSubtitle

```
Object query.spacingTitleToSubtitle
```

Definicja w linii 160 pliku [HomeScreen.kt](#).

subtitleSize

```
Object query.subtitleSize
```

Definicja w linii 158 pliku [HomeScreen.kt](#).

textSize

Object query.textSize

Definicja w linii 148 pliku [HomeScreen.kt](#).

tileHeight

Object query.tileHeight

Definicja w linii 155 pliku [HomeScreen.kt](#).

titleSize

Object query.titleSize

Definicja w linii 157 pliku [HomeScreen.kt](#).

value

Object query.value

Definicja w linii 180 pliku [HomeScreen.kt](#).

3.17 Pakiet SearchFilters

Zmienne

- Object [brand](#)
- Object [modelQuery](#)
- Object [minPrice](#)
- Object [maxPrice](#)
- Object [minYear](#)
- Object [maxYear](#)
- Object [maxMileage](#)
- Object [fuelType](#)
- Object [transmission](#)
- Object [location](#)
- Object [radiusKm](#)

3.17.1 Dokumentacja zmiennych

brand

Object SearchFilters.brand

Definicja w linii 109 pliku [SearchScreen.kt](#).

fuelType

`Object SearchFilters.fuelType`

Definicja w linii 116 pliku [SearchScreen.kt](#).

location

`Object SearchFilters.location`

Definicja w linii 118 pliku [SearchScreen.kt](#).

maxMileage

`Object SearchFilters.maxMileage`

Definicja w linii 115 pliku [SearchScreen.kt](#).

maxPrice

`Object SearchFilters.maxPrice`

Definicja w linii 112 pliku [SearchScreen.kt](#).

maxYear

`Object SearchFilters.maxYear`

Definicja w linii 114 pliku [SearchScreen.kt](#).

minPrice

`Object SearchFilters.minPrice`

Definicja w linii 111 pliku [SearchScreen.kt](#).

minYear

`Object SearchFilters.minYear`

Definicja w linii 113 pliku [SearchScreen.kt](#).

modelQuery

`Object SearchFilters.modelQuery`

Definicja w linii 110 pliku [SearchScreen.kt](#).

radiusKm

```
Object SearchFilters.radiusKm
```

Definicja w linii 119 pliku [SearchScreen.kt](#).

transmission

```
Object SearchFilters.transmission
```

Definicja w linii 117 pliku [SearchScreen.kt](#).

3.18 Pakiet selectedTab**Funkcje**

- Object [MyListingsSection](#) (Object strings, Object [myCars](#), Object onClick)
Treść zakładki "Moje ogłoszenia" wewnątrz [ProfileScreen].
- Object [CarListingCard](#) (Object car, Object strings, Object onClick)
Zwarta karta używana w zakładce "Moje ogłoszenia".

Zmienne

- Object [tabs](#)
- Object [user](#)
- Object [userEmail](#)
- Object [creationTimestamp](#)
- Object [memberYear](#)
- Object [myCars](#)
- Object [isSelected](#)
- Object [value](#)

3.18.1 Dokumentacja funkcji**CarListingCard()**

```
Object selectedTab.CarListingCard (
    Object car,
    Object strings,
    Object onClick )
```

Zwarta karta używana w zakładce "Moje ogłoszenia".

Wyświetla miniaturę zdjęcia (lub ikonę zastępczą), cenę, tytuł, linię specyfikacji rok + przebieg oraz odznakę statusu "Aktywne".

Parametry

<i>car</i>	Obiekt [CarAd] do reprezentowania.
<i>strings</i>	Aktywne [AppStrings] dla zlokalizowanych przyrostków jednostek i etykiet.
<i>onClick</i>	Callback wywoływany po dotknięciu karty.

Definicja w linii 105 pliku [ProfileScreen.kt](#).

Odwołuje się do [CarListingCard\(\)](#).

Odwołania w [CarListingCard\(\)](#).

MyListingsSection()

```
Object selectedTab.MyListingsSection (
    Object strings,
    Object myCars,
    Object onCarClick )
```

Treść zakładki "Moje ogłoszenia" wewnątrz [ProfileScreen].

Renderuje [LazyColumn] kart [CarListingCard], lub komunikat pustego stanu gdy użytkownik nie ma żadnych ogłoszeń.

Parametry

<i>strings</i>	Aktywne [AppStrings] dla zlokalizowanych etykiet.
<i>myCars</i>	Ogłoszenia należące do bieżącego użytkownika.
<i>onCarClick</i>	Callback z dotkniętym [CarAd] otwierający ekran szczegółów.

Definicja w linii 92 pliku [ProfileScreen.kt](#).

Odwołuje się do [MyListingsSection\(\)](#).

Odwołania w [MyListingsSection\(\)](#).

3.18.2 Dokumentacja zmiennych

creationTimestamp

```
Object selectedTab.creationTimestamp
```

Definicja w linii 58 pliku [ProfileScreen.kt](#).

isSelected

```
Object selectedTab.isSelected
```

Definicja w linii 71 pliku [ProfileScreen.kt](#).

memberYear

```
Object selectedTab.memberYear
```

Definicja w linii 59 pliku [ProfileScreen.kt](#).

myCars

Object selectedTab.myCars

Definicja w linii 62 pliku [ProfileScreen.kt](#).

tabs

Object selectedTab.tabs

Definicja w linii 53 pliku [ProfileScreen.kt](#).

user

Object selectedTab.user

Definicja w linii 55 pliku [ProfileScreen.kt](#).

userEmail

Object selectedTab.userEmail

Definicja w linii 56 pliku [ProfileScreen.kt](#).

value

Object selectedTab.value

Definicja w linii 108 pliku [ProfileScreen.kt](#).

3.19 Pakiet strings

3.20 Pakiet Typography

4 Dokumentacja plików

4.1 Dokumentacja pliku app/src/main/AndroidManifest.xml

4.2 AndroidManifest.xml

[Idź do dokumentacji tego pliku.](#)

```
00001 <?xml version="1.0" encoding="utf-8"?>
00002 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
00003     xmlns:tools="http://schemas.android.com/tools">
00004
00005     <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
00006     <uses-permission android:name="android.permission.INTERNET" />
00007     <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

```

00008     <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
00009
00010     <uses-feature android:name="android.hardware.camera" android:required="false" />
00011
00012     <application
00013         android:allowBackup="true"
00014         android:dataExtractionRules="@xml/data_extraction_rules"
00015         android:fullBackupContent="@xml/backup_rules"
00016         android:icon="@mipmap/ic_launcher"
00017         android:label="@string/app_name"
00018         android:roundIcon="@mipmap/ic_launcher_round"
00019         android:supportRtl="true"
00020         android:theme="@style/Theme.OtomotUZplus">
00021
00022         <activity
00023             android:name=".RegisterActivity"
00024             android:exported="false"
00025             android:screenOrientation="portrait"
00026             android:theme="@style/Theme.MaterialComponents.Light.NoActionBar"/>
00027
00028         <activity
00029             android:name=".LoginActivity"
00030             android:exported="true"
00031             android:screenOrientation="portrait"
00032             android:theme="@style/Theme.MaterialComponents.Light.NoActionBar">
00033             <intent-filter>
00034                 <action android:name="android.intent.action.MAIN" />
00035                 <category android:name="android.intent.category.LAUNCHER" />
00036             </intent-filter>
00037         </activity>
00038
00039         <activity
00040             android:name=".MainActivity"
00041             android:exported="false"
00042             android:label="OtomotUZplus"
00043             android:screenOrientation="portrait"
00044             android:theme="@style/Theme.OtomotUZplus" />
00045
00046         <service
00047             android:name=".fcm.MyFirebaseMessagingService"
00048             android:exported="false">
00049             <intent-filter>
00050                 <action android:name="com.google.firebase.MESSAGING_EVENT" />
00051             </intent-filter>
00052         </service>
00053
00054         <provider
00055             android:name="androidx.core.content.FileProvider"
00056             android:authorities="com.example.otomotuzplus.provider"
00057             android:exported="false"
00058             android:grantUriPermissions="true">
00059             <meta-data
00060                 android:name="android.support.FILE_PROVIDER_PATHS"
00061                 android:resource="@xml/file_paths" />
00062         </provider>
00063
00064         <meta-data
00065             android:name="com.google.android.gms.ads.APPLICATION_ID"
00066             android:value="ca-app-pub-3940256099942544~3347511713"/>
00067     </application>
00068
00069 </manifest>

```

4.3 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/data/FirebaseRepository.kt

Warstwa danych – dostęp do Firestore, Firebase Storage i FCM.

Pakiety

- package [FirebaseRepository](#)

Funkcje

- Object [FirebaseRepository.uploadTestCar](#) (Object onSuccess)
Zapisuje zakodowane na stałe testowe ogłoszenie do Firestore.
- Object [FirebaseRepository.getAllCars](#) (Object onSuccess)
Pobiera jednorazowo wszystkie dokumenty z kolekcji `listings`.
- Object [FirebaseRepository.addCar](#) (Object car, Object onSuccess)
Dodaje nowy dokument ogłoszenia do kolekcji `listings`.
- Object [FirebaseRepository.observeCars](#) (Object onCarsChanged)
Dodaje nasłuchiwanie zmian w Firestore w czasie rzeczywistym do kolekcji `listings`.
- Object [FirebaseRepository.uploadImages](#) (Object uris, Object onComplete)
Przesyła partię lokalnych URI obrazów do Firebase Storage w katalogu `car_images/`.
- Object [FirebaseRepository.updateFcmToken](#) (Object userId, Object token)
Scala bieżący token rejestracji FCM z dokumentem użytkownika w Firestore.
- Object [FirebaseRepository.sendLikeNotification](#) (Object sellerId, Object carTitle)
Zapisuje dokument powiadomienia o polubieniu do kolekcji `notifications`.
- Object [FirebaseRepository.proceedWithSending](#) (Object token, Object carTitle)
Wysyła powiadomienie push FCM bezpośrednio przez API HTTP v1 w wątku w tle.

Zmienne

- Object [FirebaseRepository.db](#)
- Object [FirebaseRepository.storage](#)

4.3.1 Opis szczegółowy

Warstwa danych – dostęp do Firestore, Firebase Storage i FCM.

Definicja w pliku [FirebaseRepository.kt](#).

4.4 FirebaseRepository.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file FirebaseRepository.kt
00003  * @brief Warstwa danych - dostęp do Firestore, Firebase Storage i FCM.
00004  */
00005 package com.example.otomotuzplus.data
00006
00007 /**
00008  * @file FirebaseRepository.kt
00009  * @brief Warstwa dostępu do danych dla Firestore i Firebase Storage.
00010  */
00011
00012 import com.example.otomotuzplus.models.CarAd
00013 import com.google.firebase.firestore.FirebaseFirestore
00014 import android.net.Uri
00015 import com.google.firebase.storage.FirebaseStorage
00016 import java.util.UUID
00017
00018 /**
00019  * Centralny obiekt dostępu do danych opakowujący kolekcje Firestore
00020  * ('listings' / 'users' / 'notifications') oraz Firebase Storage ('car_images/').
00021  *
00022  * Wszystkie operacje Firestore są asynchroniczne i przekazują wynik
00023  * przez lambdy zwrotne zamiast koroutyn, więc wywołujący na wątku głównym
00024  * nie musi uruchamiać zakresu koroutyn.
00025  */
00026 class FirebaseRepository {
00027     private val db = FirebaseFirestore.getInstance()
```

```

00028     private val storage = FirebaseStorage.getInstance()
00029
00030     /**
00031      * Zapisuje zakodowane na stałe testowe ogłoszenie do Firestore.
00032      *
00033      * Przeznaczone wyłącznie do celów deweloperskich i debugowania.
00034      *
00035      * @param onSuccess Wywoływane po pomyślnym zapisaniu dokumentu.
00036      * @param onFailure Wywoływane z [Exception] gdy zapis się nie powiedzie.
00037      */
00038     fun uploadTestCar(onSuccess: () -> Unit, onFailure: (Exception) -> Unit) {
00039         val testCar = CarAd(
00040             title = "Opel Corsa",
00041             priceText = "19 000 zł",
00042             locationText = "Kraków",
00043             year = "2019",
00044             mileageText = "98 400 km",
00045             fuelText = "Benzyna",
00046             gearboxText = "Manualna",
00047             engineCapacity = "1200 cm3",
00048             powerText = "100 KM",
00049             imageUrls = emptyList()
00050         )
00051
00052         db.collection("listings")
00053             .add(testCar)
00054             .addOnSuccessListener { onSuccess() }
00055             .addOnFailureListener { e: Exception -> onFailure(e) }
00056     }
00057
00058     /**
00059      * Pobiera jednorazowo wszystkie dokumenty z kolekcji `listings`.
00060      *
00061      * Preferuj [observeCars] dla ekranów UI wymagających aktualizacji na żywo.
00062      *
00063      * @param onSuccess Wywoływane z pełną listą obiektów [CarAd] po sukcesie.
00064      * @param onFailure Wywoływane z [Exception] w razie błędu.
00065      */
00066     fun getAllCars(onSuccess: (List<CarAd>) -> Unit, onFailure: (Exception) -> Unit) {
00067         db.collection("listings")
00068             .get()
00069             .addOnSuccessListener { result ->
00070                 val carsList = mutableListOf<CarAd>()
00071                 for (document in result) {
00072                     val car = document.toObject(CarAd::class.java).copy(id = document.id)
00073                     carsList.add(car)
00074                 }
00075                 onSuccess(carsList)
00076             }
00077             .addOnFailureListener { e: Exception ->
00078                 onFailure(e)
00079             }
00080     }
00081
00082     /**
00083      * Dodaje nowy dokument ogłoszenia do kolekcji `listings`.
00084      *
00085      * @param car Obiekt [CarAd] do zapisania. Pole [CarAd.id] jest ignorowane;
00086      * Firestore generuje identyfikator dokumentu automatycznie.
00087      * @param onSuccess Wywoływane po pomyślnym utworzeniu dokumentu.
00088      * @param onFailure Wywoływane z [Exception] w razie błędu.
00089      */
00090     fun addCar(car: CarAd, onSuccess: () -> Unit, onFailure: (Exception) -> Unit) {
00091         db.collection("listings")
00092             .add(car)
00093             .addOnSuccessListener { onSuccess() }
00094             .addOnFailureListener { e: Exception -> onFailure(e) }
00095     }
00096
00097     /**
00098      * Dołącza nasłuchiwanca migawek Firestore w czasie rzeczywistym do kolekcji `listings`.
00099      *
00100      * Callback wywoływany natychmiast z aktualnym stanem, a następnie przy każdej
00101      * kolejnej zmianie. Nasłuchiwanca żyje przez cały czas istnienia komponentu;
00102      * wywołujący odpowiada za jego usunięcie (implementacja nie udostępnia uchwytu --
00103      * nasłuchiwanca powiązany jest z cyklem życia Activity przez `LaunchedEffect`).
00104      *
00105      * @param onCarsChanged Wywoływane na wątku głównym ze zaktualizowaną listą przy
00106      * każdej zmianie kolekcji. Wadliwe dokumenty są pomijane.
00107      */
00108     fun observeCars(onCarsChanged: (List<CarAd>) -> Unit) {
00109         db.collection("listings")
00110             .addSnapshotListener { snapshot, e ->
00111                 if (e != null || snapshot == null) {
00112                     println("BŁĄD NASŁUCHU FIREBASE: ${e?.message}")
00113                     return@addSnapshotListener
00114                 }
00115             }
00116     }

```

```

00115         val cars = snapshot.documents.mapNotNull { doc ->
00116             try {
00117                 val car = doc.toObject(CarAd::class.java)
00118                 car?.copy(id = doc.id)
00119             } catch (ex: Exception) {
00120                 println("BŁĄD PARSOWANIA AUTA: ${ex.message}")
00121                 null
00122             }
00123         }
00124         onCarsChanged(cars)
00125     }
00126 }
00127
00128 /**
00129  * Przesyła partię lokalnych URI obrazów do Firebase Storage w katalogu `car_images/`.
00130  *
00131  * Każdy plik przechowywany pod nazwą UUID. Wszystkie przesyłania działają równolegle,
00132  * a [onComplete] wywoływane jest gdy wszystkie zakończą działanie (sukces lub błąd).
00133  * Nieudane przesyłania poszczególnych plików są pomijane na liście wynikowej.
00134  *
00135  * @param uris Lokalne URI treści (zdjęcia z galerii lub zrobione aparatem przez `FileProvider`).
00136  * @param onComplete Wywoływane z listą adresów URL pobierania po zakończeniu wszystkich
00137  *   przesyłań. Zwraca pustą listę gdy [uris] jest puste.
00138  */
00139 fun uploadImages(uris: List<Uri>, onComplete: (List<String>) -> Unit) {
00140     if (uris.isEmpty()) {
00141         onComplete(emptyList())
00142         return
00143     }
00144
00145     val uploadedUrls = mutableListOf<String>()
00146     var uploadCount = 0
00147
00148     for (uri in uris) {
00149         val fileName = UUID.randomUUID().toString()
00150         val storageRef = storage.reference.child("car_images/$fileName")
00151
00152         storageRef.putFile(uri).continueWithTask { task ->
00153             if (!task.isSuccessful) {
00154                 task.exception?.let { throw it }
00155             }
00156             storageRef.downloadUrl
00157         }.addOnCompleteListener { task ->
00158             if (task.isSuccessful) {
00159                 val downloadUri = task.result
00160                 uploadedUrls.add(downloadUri.toString())
00161             }
00162
00163             uploadCount++
00164             if (uploadCount == uris.size) {
00165                 onComplete(uploadedUrls)
00166             }
00167         }
00168     }
00169 }
00170
00171 /**
00172  * Scala bieżący token rejestracji FCM z dokumentem użytkownika w Firestore.
00173  *
00174  * Używa `SetOptions.merge()` aby zachować pozostałe pola dokumentu `users`.
00175  * Wywoływane przez [MyFirebaseMessagingService.onNewToken] przy każdym odświeżeniu tokenu.
00176  *
00177  * @param userId UID zalogowanego użytkownika w Firebase Auth.
00178  * @param token Nowy token rejestracji FCM.
00179  */
00179 fun updateFcmToken(userId: String, token: String) {
00180     val tokenData = mapOf("fcmToken" to token)
00181     db.collection("users")
00182         .document(userId)
00183         .set(
00184             tokenData,
00185             com.google.firebase.firestore.SetOptions.merge()
00186         )
00187         .addOnSuccessListener {
00188             android.util.Log.d(
00189                 "FirebaseRepository",
00190                 "Token FCM zaktualizowany pomyślnie w Firestore."
00191             )
00192         }
00193 }
00194
00195 /**
00196  * Zapisuje dokument powiadomienia o polubieniu do kolekcji `notifications`.
00197  *
00198  * `MainActivity` nasłuchuje tej kolekcji przez listener migawek i wyświetla
00199  * lokalne powiadomienie gdy nowy dokument zostanie dodany dla bieżącego użytkownika.
00200  *
00201  * @param sellerId UID sprzedającego posiadającego polubione ogłoszenie w Firebase Auth.
00202  * @param carTitle Tytuł polubionego ogłoszenia; umieszczony w treści powiadomienia.

```

```

00202      */
00203      fun sendLikeNotification(sellerId: String, carTitle: String) {
00204          val notification = hashMapOf(
00205              "toUser" to sellerId,
00206              "carTitle" to carTitle,
00207              "timestamp" to com.google.firebase.firestore.FieldValue.serverTimestamp()
00208          )
00209          db.collection("notifications").add(notification)
00210      }
00211
00212      /**
00213       * Wysyła powiadomienie push FCM bezpośrednio przez API HTTP v1 w wątku w tle.
00214       *
00215       * Pobiera krótkotrwały token dostępu OAuth2 z Firebase Auth, następnie wykonuje
00216       * żądanie HTTP POST do `fcm.googleapis.com`. Operacja typu fire-and-forget;
00217       * wynik tylko logowany, nigdy nie przekazywany do UI.
00218       *
00219       * @param token Token rejestracji FCM urządzenia odbiorcy. Jeśli null lub pusty,
00220       *   wywołanie jest ignorowane.
00221       * @param carTitle Tytuł polubionego ogłoszenia; osadzony w treści wiadomości.
00222       */
00223      fun proceedWithSending(token: String?, carTitle: String) {
00224          if (!token.isNullOrEmpty()) {
00225              Thread {
00226                  try {
00227                      android.util.Log.d("FCM_HTTP_V1", "Uruchamiam wątek wysyłania requestu HTTP...")
00228                      val internalTokenTask = com.google.firebase.auth.FirebaseAuth.getInstance()
00229                          .getAccessToken(false)
00230                      val accessToken =
00231                          com.google.android.gms.tasks.Tasks.await(internalTokenTask).token
00232
00233                      if (accessToken.isNullOrEmpty()) {
00234                          android.util.Log.e("FCM_HTTP_V1", "BŁĄD: Access Token z Firebase Auth jest
00235                          pusty!")
00236                          return@Thread
00237                      }
00238                      val url =
00239                          java.net.URL("https://fcm.googleapis.com/v1/projects/otomotuzplus/messages:send")
00240                      val conn = url.openConnection() as java.net.HttpURLConnection
00241                      conn.requestMethod = "POST"
00242                      conn.setRequestProperty("Content-Type", "application/json")
00243                      conn.setRequestProperty("Authorization", "Bearer $accessToken")
00244                      conn.doOutput = true
00245
00246                      val jsonRequest = """
00247                      {
00248                          "message": {
00249                              "token": "$token",
00250                              "data": {
00251                                  "title": "Ktoś polubił Twoje ogłoszenie!",
00252                                  "body": "Twoje auto $carTitle spodobało się nowemu użytkownikowi."
00253                              }
00254                          }
00255                      }
00256                      """.trimIndent()
00257                      conn.outputStream.use { os ->
00258                          os.write(jsonRequest.toByteArray(Charsets.UTF_8))
00259                      }
00260
00261                      val responseCode = conn.responseCode
00262                      android.util.Log.d("FCM_HTTP_V1", "KOD ODPOWIEDZI HTTP SERWERA GOOGLE:
00263                      $responseCode")
00264                      } catch (e: Exception) {
00265                          android.util.Log.e("FCM_HTTP_V1", "Błąd wysyłania przez HTTP v1", e)
00266                      }
00267                  }.start()
00268              } else {
00269                  android.util.Log.w("FCM_HTTP_V1", "PRZERWANO: Nie udało się przypisać żadnego tokenu FCM
00270                  dla tego sprzedawcy.")
00271              }
00272          }
00273      }

```

4.5 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/data/NominatimGeocoder.kt

Funkcje zawieszające do geokodowania polskich lokalizacji przez REST API OpenStreetMap Nominatim.

Pakiety

- package [postalCode](#)

Funkcje

- Object [geocodeCity](#) (Object city)
Zamienia polską nazwę miasta na parę współrzędnych WGS-84.

Zmienne

- Object [clean](#)
- Object [conn](#)
- Object [json](#)
- Object [obj](#)
- Object [encoded](#)

4.5.1 Opis szczegółowy

Funkcje zawieszające do geokodowania polskich lokalizacji przez REST API OpenStreetMap Nominatim.

Obie funkcje działają na [kotlinx.coroutines.Dispatchers.IO] i zwracają `null` przy błędzie sieci lub pustym wyniku. Nagłówek User-Agent ustawiony na `OtomotUZplus/1.0` zgodnie z polityką użytkowania Nominatim.

Definicja w pliku [NominatimGeocoder.kt](#).

4.5.2 Dokumentacja funkcji

geocodeCity()

```
Object geocodeCity (  
    Object city )
```

Zamienia polską nazwę miasta na parę współrzędnych WGS-84.

Wyszukiwanie ograniczone do Polski (`countrycodes=pl`). Nazwa miasta kodowana jako URL przed wysłaniem zapytania.

Parametry

<code>city</code>	Czytelna nazwa miasta (np. "Warszawa", "Kraków").
-------------------	---

Zwraca

[Pair] (szerokość, długość) lub `null` jeśli miasto nie zostało znalezione lub wystąpił błąd sieci.

Definicja w linii 47 pliku [NominatimGeocoder.kt](#).

4.5.3 Dokumentacja zmiennych

clean

`Object clean`

Definicja w linii 31 pliku [NominatimGeocoder.kt](#).

conn

`Object conn`

Definicja w linii 32 pliku [NominatimGeocoder.kt](#).

encoded

`Object encoded`

Definicja w linii 49 pliku [NominatimGeocoder.kt](#).

json

`Object json`

Definicja w linii 33 pliku [NominatimGeocoder.kt](#).

obj

`Object obj`

Definicja w linii 34 pliku [NominatimGeocoder.kt](#).

4.6 NominatimGeocoder.kt

[Idź do dokumentacji tego pliku.](#)

```

00001 /**
00002  * @file NominatimGeocoder.kt
00003  * @brief Funkcje zawieszające do geokodowania polskich lokalizacji
00004  * przez REST API OpenStreetMap Nominatim.
00005  *
00006  * Obie funkcje działają na [kotlinx.coroutines.Dispatchers.IO] i zwracają
00007  * `null` przy błędzie sieci lub pustym wyniku. Nagłówek User-Agent ustawiony
00008  * na `OtomotUZplus/1.0` zgodnie z polityką użytkowania Nominatim.
00009  */
00010 package com.example.otomotuzplus.data
00011
00012 import kotlinx.coroutines.Dispatchers
00013 import kotlinx.coroutines.withContext
00014 import org.json.JSONArray
00015 import java.net.HttpURLConnection
00016 import java.net.URL
00017 import java.net.URLEncoder
00018
00019 /**
00020  * Zamienia polski kod pocztowy na parę współrzędnych WGS-84.
00021  *
00022  * Separator myślnika usuwany przed zapytaniem, więc akceptowane są formaty
00023  * "00000" i "00-000".
00024  *
00025  * @param postalCode Polski kod pocztowy z separatorem lub bez.
00026  * @return [Pair] (szerokość, długość) lub `null` jeśli kod nie może być
00027  * rozpoznany lub wystąpił błąd sieci.
00028  */
00029 suspend fun geocodePostalCode(postalCode: String): Pair<Double, Double>? =
00030     withContext(Dispatchers.IO) {
00031         val clean = postalCode.replace("-", "")
00032         val conn =
00033             URL("https://nominatim.openstreetmap.org/search?postalcode=$clean&country=pl&format=json&limit=1")
00034                 .openConnection() as HttpURLConnection
00035         try {
00036             conn.setRequestProperty("User-Agent", "OtomotUZplus/1.0")
00037             conn.connectTimeout = 5000
00038             conn.readTimeout = 5000
00039             val json = JSONArray(conn.inputStream.bufferedReader().use { it.readText() })
00040             if (json.length() > 0) {
00041                 val obj = json.getJSONObject(0)
00042                 Pair(obj.getString("lat").toDouble(), obj.getString("lon").toDouble())
00043             } else null
00044         } catch (_: Exception) { null }
00045         finally { conn.disconnect() }
00046     }
00047 /**
00048  * Zamienia polską nazwę miasta na parę współrzędnych WGS-84.
00049  *
00050  * Wyszukiwanie ograniczone do Polski (`countrycodes=pl`). Nazwa miasta
00051  * kodowana jako URL przed wysłaniem zapytania.
00052  *
00053  * @param city Czytelna nazwa miasta (np. "Warszawa", "Kraków").
00054  * @return [Pair] (szerokość, długość) lub `null` jeśli miasto nie zostało
00055  * znalezione lub wystąpił błąd sieci.
00056  */
00057 suspend fun geocodeCity(city: String): Pair<Double, Double>? =
00058     withContext(Dispatchers.IO) {
00059         val encoded = URLEncoder.encode(city, "UTF-8")
00060         val conn =
00061             URL("https://nominatim.openstreetmap.org/search?q=$encoded&countrycodes=pl&format=json&limit=1")
00062                 .openConnection() as HttpURLConnection
00063         try {
00064             conn.setRequestProperty("User-Agent", "OtomotUZplus/1.0")
00065             conn.connectTimeout = 5000
00066             conn.readTimeout = 5000
00067             val json = JSONArray(conn.inputStream.bufferedReader().use { it.readText() })
00068             if (json.length() > 0) {
00069                 val obj = json.getJSONObject(0)
00070                 Pair(obj.getString("lat").toDouble(), obj.getString("lon").toDouble())
00071             } else null
00072         } catch (_: Exception) { null }
00073         finally { conn.disconnect() }
00074     }

```

4.7 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/data/PreferenceManager.kt

Wrapper SharedPreferences przechowujący motyw i język aplikacji.

Pakiety

- package [PreferenceManager](#)

Funkcje

- Object [PreferenceManager.setNotificationsRefused](#) (Object refused)
Zapisuje informację czy użytkownik jawnie odrzucił prośbę o uprawnienie do powiadomień.
- Object [PreferenceManager.wasNotificationsRefused](#) ()
- Object [PreferenceManager.setThemeMode](#) (Object mode)
Zapisuje wybrany [ThemeMode].
- Object [PreferenceManager.getThemeMode](#) ()
- Object [PreferenceManager.setLanguage](#) (Object lang)
Zapisuje wybrany język interfejsu.
- Object [PreferenceManager.getLanguage](#) ()

Zmienne

- Object [PreferenceManager.prefs](#)
- Object [PreferenceManager.KEY_NOTIFICATIONS_REFUSED](#)

4.7.1 Opis szczegółowy

Wrapper SharedPreferences przechowujący motyw i język aplikacji.

Definicja w pliku [PreferenceManager.kt](#).

4.8 PreferenceManager.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file PreferenceManager.kt
00003  * @brief Wrapper SharedPreferences przechowujący motyw i język aplikacji.
00004  */
00005 package com.example.otomotuzplus.data
00006
00007 import android.content.Context
00008 import android.content.SharedPreferences
00009
00010 /**
00011  * Wybór motywu aplikacji.
00012  *
00013  * - [LIGHT] zawsze używa jasnego schematu kolorów Material3.
00014  * - [DARK] zawsze używa ciemnego schematu kolorów Material3.
00015  * - [SYSTEM] podąża za ustawieniem trybu ciemnego systemu operacyjnego
00016  * przez [androidx.compose.foundation.isSystemInDarkTheme].
00017  */
00018 enum class ThemeMode { LIGHT, DARK, SYSTEM }
00019
00020 /**
00021  * Cienka nakładka na [SharedPreferences] przechowująca preferencje użytkownika
00022  * między restartami aplikacji.
```

```

00023 *
00024 * Wszystkie zapisy używają `apply()` (asynchronicznie), więc nie blokują wątku
00025 * głównego. Plik preferencji nosi nazwę `app_prefs`.
00026 *
00027 * @param context Kontekst aplikacji lub Activity używany do otwarcia pliku preferencji.
00028 */
00029 class PreferenceManager(context: Context) {
00030     private val prefs: SharedPreferences = context.getSharedPreferences("app_prefs",
Context.MODE_PRIVATE)
00031     private val KEY_NOTIFICATIONS_REFUSED = "notifications_refused"
00032
00033     /**
00034      * Zapisuje informację czy użytkownik jawnie odrzucił prośbę o uprawnienie do powiadomień.
00035      *
00036      * Gdy `true`, okno dialogowe uzasadnienia w [OtomotUZplusApp] jest pomijane przez
00037      * cały pozostały czas życia aplikacji na tym urządzeniu.
00038      *
00039      * @param refused `true` jeśli użytkownik kliknął "Może później" w oknie uprawnień.
00040      */
00041     fun setNotificationsRefused(refused: Boolean) {
00042         prefs.edit().putBoolean(KEY_NOTIFICATIONS_REFUSED, refused).apply()
00043     }
00044
00045     /**
00046      * @return `true` jeśli użytkownik wcześniej odrzucił prośbę o uprawnienie do powiadomień.
00047      */
00048     fun wasNotificationsRefused(): Boolean {
00049         return prefs.getBoolean(KEY_NOTIFICATIONS_REFUSED, false)
00050     }
00051     /**
00052      * Zapisuje wybrany [ThemeMode].
00053      * @param mode Motyw do zastosowania przy następnej rekompozycji.
00054      */
00055     fun setThemeMode(mode: ThemeMode) {
00056         prefs.edit().putString("theme_mode", mode.name).apply()
00057     }
00058
00059     /**
00060      * @return Zapisany [ThemeMode]; domyślnie [ThemeMode.SYSTEM] jeśli nie ustawiono.
00061      */
00062     fun getThemeMode(): ThemeMode {
00063         val name = prefs.getString("theme_mode", ThemeMode.SYSTEM.name)
00064         return ThemeMode.valueOf(name ?: ThemeMode.SYSTEM.name)
00065     }
00066
00067     /**
00068      * Zapisuje wybrany język interfejsu.
00069      * @param lang Identyfikator języka -- `Polski` lub `English`.
00070      */
00071     fun setLanguage(lang: String) {
00072         prefs.edit().putString("language", lang).apply()
00073     }
00074
00075     /**
00076      * @return Zapisany identyfikator języka; domyślnie `Polski` jeśli nie ustawiono.
00077      */
00078     fun getLanguage(): String {
00079         return prefs.getString("language", "Polski") ?: "Polski"
00080     }
00081 }

```

4.9 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/fcm/MyFirebaseMessagingService.kt

Serwis FCM odbierający przychodzące wiadomości push.

Pakiety

- package [FirebaseMessagingService](#)

Funkcje

- Object [FirebaseMessagingService.onMessageReceived](#) (Object remoteMessage)
- Object [FirebaseMessagingService.sendNotification](#) (Object title, Object messageBody)
- Object [FirebaseMessagingService.onNewToken](#) (Object token)

4.9.1 Opis szczegółowy

Serwis FCM odbierający przychodzące wiadomości push.

Definicja w pliku [MyFirebaseMessagingService.kt](#).

4.10 MyFirebaseMessagingService.kt

[Idź do dokumentacji tego pliku.](#)

```

00001 /**
00002  * @file MyFirebaseMessagingService.kt
00003  * @brief Serwis FCM odbierający przychodzące wiadomości push.
00004  */
00005 package com.example.otomotuzplus.fcm
00006
00007 import android.app.NotificationManager
00008 import android.app.PendingIntent
00009 import android.content.Context
00010 import android.content.Intent
00011 import androidx.core.app.NotificationCompat
00012 import com.example.otomotuzplus.MainActivity
00013 import com.example.otomotuzplus.R
00014 import com.example.otomotuzplus.utils.NotificationHelper
00015 import com.google.firebase.messaging.FirebaseMessagingService
00016 import com.google.firebase.messaging.RemoteMessage
00017
00018 /**
00019  * Serwis Firebase Cloud Messaging obsługujący przychodzące wiadomości push.
00020  *
00021  * Obsługiwane są dwa formaty ładunku:
00022  * - **Wiadomość danych** ('remoteMessage.data' niepusta): klucze "title" i "body"
00023  *   są wyodrębniane i przekazywane do [sendNotification].
00024  * - **Wiadomość powiadomienia**: używane są standardowe pola powiadomień z Firebase.
00025  *
00026  * [onNewToken] jest wywoływany przez SDK FCM gdy token rejestracji ulegnie zmianie.
00027  * Jeśli użytkownik jest wtedy zalogowany, nowy token powinien być zapisany przez
00028  * [FirebaseRepository.updateFcmToken] (jeszcze nie zaimplementowane tutaj).
00029  */
00030 class MyFirebaseMessagingService : FirebaseMessagingService() {
00031
00032     override fun onMessageReceived(remoteMessage: RemoteMessage) {
00033         if (remoteMessage.data.isNotEmpty()) {
00034             val title = remoteMessage.data["title"]?: "OtomotUZplus"
00035             val body = remoteMessage.data["body"]?: "Sprawdź nowe zmiany!"
00036             sendNotification(title, body)
00037         }
00038         else {
00039             remoteMessage.notification?.let {
00040                 sendNotification(it.title?: "OtomotUZplus", it.body?: "")
00041             }
00042         }
00043     }
00044
00045     private fun sendNotification(title: String, messageBody: String) {
00046         val intent = Intent(this, MainActivity::class.java).apply {
00047             addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP)
00048         }
00049
00050         val pendingIntent = PendingIntent.getActivity(
00051             this,
00052             System.currentTimeMillis().toInt(),
00053             intent,
00054             PendingIntent.FLAG_IMMUTABLE
00055         )
00056
00057         val notificationBuilder = NotificationCompat.Builder(this, NotificationHelper.CHANNEL_ID)
00058             .setSmallIcon(R.drawable.ic_launcher_foreground)
00059             .setContentTitle(title)
00060             .setContentText(messageBody)
00061             .setAutoCancel(true)
00062             .setContentIntent(pendingIntent)
00063             .setPriority(NotificationCompat.PRIORITY_DEFAULT)
00064
00065         val notificationManager = getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager
00066
00067         val uniqueNotificationId = System.currentTimeMillis().toInt()
00068         notificationManager.notify(uniqueNotificationId, notificationBuilder.build())
00069     }

```

```

00070
00071     override fun onNewToken(token: String) {
00072         super.onNewToken(token)
00073         // Jeśli użytkownik jest zalogowany, wyślij nowy token do bazy danych
00074     }
00075 }

```

4.11 Dokumentacja pliku `app/src/main/java/com/example/otomotuzplus/LoginActivity.kt`

Ekran logowania – punkt wejściowy aplikacji, uwierzytelnianie przez Firebase Auth.

Pakiety

- package [AppCompatActivity](#)

Funkcje

- Object [AppCompatActivity.onCreate](#) (Object savedInstanceState)
- Object [AppCompatActivity.onStart](#) ()
- Object [AppCompatActivity.otworzGlowneWnetrzeAppki](#) ()

4.11.1 Opis szczegółowy

Ekran logowania – punkt wejściowy aplikacji, uwierzytelnianie przez Firebase Auth.

Definicja w pliku [LoginActivity.kt](#).

4.12 LoginActivity.kt

[Idź do dokumentacji tego pliku.](#)

```

00001 /**
00002  * @file LoginActivity.kt
00003  * @brief Ekran logowania - punkt wejściowy aplikacji, uwierzytelnianie przez Firebase Auth.
00004  */
00005 package com.example.otomotuzplus
00006
00007 import android.content.Intent
00008 import android.os.Bundle
00009 import android.widget.Button
00010 import android.widget.TextView
00011 import android.widget.Toast
00012 import androidx.appcompat.app.AppCompatActivity
00013 import com.google.android.material.textfield.TextInputEditText
00014 import com.google.firebase.auth.FirebaseAuth
00015 import kotlin.jvm.java
00016
00017 /**
00018  * Activity wejściowa zadeklarowana jako launcher w `AndroidManifest.xml`.
00019  *
00020  * Wyświetla pola e-mail i hasło oraz deleguje uwierzytelnianie do
00021  * Firebase Auth ('signInWithEmailAndPassword'). Po pomyślnym zalogowaniu
00022  * (lub gdy użytkownik jest już zalogowany przy starcie) uruchamia
00023  * [MainActivity] z flagami `FLAG_ACTIVITY_NEW_TASK | FLAG_ACTIVITY_CLEAR_TASK`,
00024  * aby wyczyścić stos back i uniemożliwić powrót do ekranu logowania.
00025  *
00026  * Nawigacja do [RegisterActivity] dostępna przez link tekstowy.
00027  */
00028 class LoginActivity : AppCompatActivity() {
00029
00030     private lateinit var auth: FirebaseAuth
00031
00032     override fun onCreate(savedInstanceState: Bundle?) {
00033         super.onCreate(savedInstanceState)

```

```

00034         window.statusBarColor = android.graphics.Color.parseColor("#FBD70E")
00035         androidx.core.view.WindowCompat.getInsetsController(window,
window.decorView).isAppearanceLightStatusBars = true
00036         setContentView(R.layout.activity_login)
00037
00038         auth = FirebaseAuth.getInstance()
00039
00040         val emailET = findViewById<TextInputEditText>(R.id.emailEditText)
00041         val passwordET = findViewById<TextInputEditText>(R.id.passwordEditText)
00042         val loginBtn = findViewById<Button>(R.id.loginButton)
00043         val registerTV = findViewById<TextView>(R.id.goToRegisterText)
00044
00045         loginBtn.setOnClickListener {
00046             val email = emailET.text.toString().trim()
00047             val pass = passwordET.text.toString().trim()
00048
00049             if (email.isEmpty() || pass.isEmpty()) {
00050                 Toast.makeText(this, "Wypełnij wszystkie pola", Toast.LENGTH_SHORT).show()
00051                 return@setOnClickListener
00052             }
00053
00054             auth.signInWithEmailAndPassword(email, pass)
00055                 .addOnCompleteListener(this) { task ->
00056                     if (task.isSuccessful) {
00057                         Toast.makeText(this, "Zalogowano pomyślnie!", Toast.LENGTH_SHORT).show()
00058                         otworzGlowneWnetrzeAppki()
00059                     } else {
00060                         Toast.makeText(this, "Błąd logowania: ${task.exception?.message}",
Toast.LENGTH_LONG).show()
00061                     }
00062                 }
00063             }
00064
00065             registerTV.setOnClickListener {
00066                 val intent = Intent(this, RegisterActivity::class.java)
00067                 startActivity(intent)
00068             }
00069         }
00070
00071         override fun onStart() {
00072             super.onStart()
00073             val currentUser = auth.currentUser
00074             if (currentUser != null) {
00075                 otworzGlowneWnetrzeAppki()
00076             }
00077         }
00078
00079         private fun otworzGlowneWnetrzeAppki() {
00080             val intent = Intent(this, MainActivity::class.java)
00081             intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
00082             startActivity(intent)
00083             finish()
00084         }
00085     }

```

4.13 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/MainActivity.kt

Główna Activity po zalogowaniu – korzeń drzewa Compose i zarządca nawigacji stanowej.

Pakiety

- package [ComponentActivity](#)

Funkcje

- Object [ComponentActivity.onCreate](#) (Object savedInstanceState)
- Object [openSearch](#) (Object query, Object brand, Object showFilters)

Zmienne

- Object [ComponentActivity.requestPermissionLauncher](#)
- Object [ComponentActivity.repository](#)
- Object [ComponentActivity.db](#)
- Object [ComponentActivity.hasNotificationPermission](#)
- Object [strings](#)

Główna kompozycja posiadająca cały współdzielony stan nawigacji i UI aplikacji.

- Object [context](#)
- Object [shouldShowDialog](#)
- Object [currentDestination](#)
- Object [showSettings](#)
- Object [pendingSearchQuery](#)
- Object [pendingSearchBrand](#)
- Object [pendingSearchShowFilters](#)
- Object [favoriteCars](#)
- Object [selectedCar](#)
- Object [allCarsFromDb](#)
- Object [toggleFavorite](#)
- Object [realId](#)
- Object [likedCar](#)
- Object [showRationaleDialog](#)
- Object [permission](#)
- Object [isGranted](#)
- Object [navItems](#)
- Object [navItemColors](#)
- Object [navSuiteColors](#)

4.13.1 Opis szczegółowy

Główna Activity po zalogowaniu – korzeń drzewa Compose i zarządca nawigacji stanowej.

Definicja w pliku [MainActivity.kt](#).

4.13.2 Dokumentacja funkcji

openSearch()

```
Object openSearch (
    Object query,
    Object brand,
    Object showFilters )
```

Definicja w linii 136 pliku [MainActivity.kt](#).

4.13.3 Dokumentacja zmiennych

allCarsFromDb

```
Object allCarsFromDb
```

Definicja w linii 118 pliku [MainActivity.kt](#).

context

Object context

Definicja w linii 108 pliku [MainActivity.kt](#).

currentDestination

Object currentDestination

Definicja w linii 111 pliku [MainActivity.kt](#).

favoriteCars

Object favoriteCars

Definicja w linii 116 pliku [MainActivity.kt](#).

isGranted

Object isGranted

Definicja w linii 143 pliku [MainActivity.kt](#).

likedCar

Object likedCar

Definicja w linii 128 pliku [MainActivity.kt](#).

navItemColors

Object navItemColors

Definicja w linii 152 pliku [MainActivity.kt](#).

navItems

Object navItems

Definicja w linii 150 pliku [MainActivity.kt](#).

navSuiteColors

Object navSuiteColors

Definicja w linii 154 pliku [MainActivity.kt](#).

pendingSearchBrand

Object pendingSearchBrand

Definicja w linii 114 pliku [MainActivity.kt](#).

pendingSearchQuery

Object pendingSearchQuery

Definicja w linii 113 pliku [MainActivity.kt](#).

pendingSearchShowFilters

Object pendingSearchShowFilters

Definicja w linii 115 pliku [MainActivity.kt](#).

permission

Object permission

Definicja w linii 142 pliku [MainActivity.kt](#).

realId

Object realId

Definicja w linii 126 pliku [MainActivity.kt](#).

selectedCar

Object selectedCar

Definicja w linii 117 pliku [MainActivity.kt](#).

shouldShowDialog

Object shouldShowDialog

Definicja w linii 110 pliku [MainActivity.kt](#).

showRationaleDialog

Object showRationaleDialog

Definicja w linii 134 pliku [MainActivity.kt](#).

showSettings

Object `showSettings`

Definicja w linii 112 pliku `MainActivity.kt`.

strings

Object `strings`

Główna kompozycja posiadająca cały współdzielony stan nawigacji i UI aplikacji.

4.13.3.1 Model nawigacji

Nawigacja sterowana stanem zamiast trasami:

- `currentDestination` wybiera który ekran zakładki jest pokazany wewnątrz `[NavigationSuiteScaffold]`.
- `selectedCar != null` nakłada `[AdDetailScreen]` na cały scaffold.
- `showSettings == true` nakłada `[SettingsScreen]` na cały scaffold.
- `[BackHandler]` zdejmuje warstwy: szczegóły → ustawienia → HOME.

4.13.3.2 Głębokie linki między ekranami

Zmienne `pendingSearch*` przenoszą stan początkowych filtrów z ekranu Home do `[SearchScreen]`. `[SearchScreen]` konsumuje je raz przez `onInitialFiltersConsumed`.

4.13.3.3 Przepływ danych

`allCarsFromDb` wypełniane raz przez `[LaunchedEffect]` subskrybujący `[FirebaseRepository.observeCars]` i przekazywane jawnie do każdego ekranu który go potrzebuje — brak warstwy `ViewModel` ani `DI`.

Parametry

<code>repository</code>	Singleton dostępu do danych.
<code>themeMode</code>	Aktywny motyw.
<code>onThemeChange</code>	Callback do zapisania nowego motywu.
<code>currentLanguage</code>	Aktywny identyfikator języka.
<code>onLanguageChange</code>	Callback do zapisania nowego języka.
<code>notificationsRefused</code>	Czy użytkownik odrzucił okno dialogowe uprawnień.
<code>onSetNotificationsRefused</code>	Callback do zapisania flagi odrzucenia.
<code>onRequestNotificationPermission</code>	Callback uruchamiający systemowe okno uprawnień.
<code>notificationsPermissionGranted</code>	Aktualny stan przyznania uprawnień.

Definicja w linii 107 pliku `MainActivity.kt`.

toggleFavorite

Object toggleFavorite

Definicja w linii 123 pliku [MainActivity.kt](#).

4.14 MainActivity.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file MainActivity.kt
00003  * @brief Główna Activity po zalogowaniu - korzeń drzewa Compose i zarządca nawigacji stanowej.
00004  */
00005 @file:OptIn(ExperimentalMaterial3Api::class)
00006
00007 package com.example.otomotuzplus
00008
00009 import android.Manifest
00010 import android.content.pm.PackageManager
00011 import android.os.Build
00012 import android.os.Bundle
00013 import android.util.Log
00014 import androidx.activity.ComponentActivity
00015 import androidx.activity.compose.BackHandler
00016 import androidx.activity.compose.setContent
00017 import androidx.activity.enableEdgeToEdge
00018 import androidx.activity.result.contract.ActivityResultContracts
00019 import androidx.compose.foundation.isSystemInDarkTheme
00020 import androidx.compose.foundation.layout.*
00021 import androidx.compose.material.icons.Icons
00022 import androidx.compose.material.icons.filled.*
00023 import androidx.compose.material.icons.outlined.*
00024 import androidx.compose.material3.*
00025 import androidx.compose.material3.adaptive.navigationsuite.NavigationSuiteDefaults
00026 import androidx.compose.material3.adaptive.navigationsuite.NavigationSuiteScaffold
00027 import androidx.compose.runtime.*
00028 import androidx.compose.runtime.saveable.rememberSaveable
00029 import androidx.compose.ui.Modifier
00030 import androidx.compose.ui.graphics.Color
00031 import androidx.compose.ui.platform.LocalContext
00032 import androidx.core.content.ContextCompat
00033 import com.example.otomotuzplus.data.FirebaseRepository
00034 import com.example.otomotuzplus.data.PreferenceManager
00035 import com.example.otomotuzplus.data.ThemeMode
00036 import com.example.otomotuzplus.ui.models.EnglishStrings
00037 import com.example.otomotuzplus.ui.models.PolishStrings
00038 import com.example.otomotuzplus.ui.navigation.AppDestinations
00039 import com.example.otomotuzplus.ui.navigation.NavigationItem
00040 import com.example.otomotuzplus.ui.screens.favorites.FavoritesScreen
00041 import com.example.otomotuzplus.ui.screens.home.HomeScreen
00042 import com.example.otomotuzplus.ui.screens.profile.ProfileScreen
00043 import com.example.otomotuzplus.ui.screens.search.SearchScreen
00044 import com.example.otomotuzplus.ui.screens.settings.SettingsScreen
00045 import com.example.otomotuzplus.ui.theme.OtomotUZplusTheme
00046 import com.example.otomotuzplus.utils.NotificationHelper
00047 import com.google.firebase.auth.FirebaseAuth
00048 import com.google.firebase.firestore.DocumentChange
00049 import com.google.firebase.messaging.FirebaseMessaging
00050 import com.google.android.gms.ads.MobileAds
00051
00052 /**
00053  * Jedyna Activity po zalogowaniu, która hostuje całe drzewo UI Compose.
00054  *
00055  * Zadania obsługiwane na poziomie Activity:
00056  * - Inicjalizacja AdMob ('MobileAds.initialize').
00057  * - Żądanie uprawnienia 'POST_NOTIFICATIONS' na Android 13+.
00058  * - Odczyt zapisanego motywu i języka z [PreferenceManager] i
00059  *   przekazanie ich do [OtomotUZplusApp].
00060  * - Pobieranie tokenu rejestracji FCM i dołączanie nasłuchiacza migawek Firestore
00061  *   na kolekcji 'notifications', aby przychodzące zdarzenia "polubień"
00062  *   wyzwały lokalne powiadomienia push przez [NotificationHelper].
00063  *
00064  * Cały stan nawigacji w aplikacji żyje w [OtomotUZplusApp] (jako [Composable]),
00065  * nie w tej Activity.
00066  */
00067 class MainActivity : ComponentActivity() {
00068     private val requestPermissionLauncher = registerForActivityResult(
00069         ActivityResultContracts.RequestPermission()
00070     ) { isGranted: Boolean -> hasNotificationPermission = isGranted }
```

```

00071
00072     private val repository = FirebaseRepository()
00073     val db = com.google.firebase.firestore.FirebaseFirestore.getInstance()
00074     private var hasNotificationPermission by mutableStateOf(false)
00075
00076     override fun onCreate(savedInstanceState: Bundle?) {
00077         super.onCreate(savedInstanceState)
00078         val prefManager = PreferenceManager(this)
00079         val lang = prefManager.getLanguage()
00080         val strings = if (lang == "Polski") PolishStrings else EnglishStrings
00081
00082         hasNotificationPermission = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
00083             ContextCompat.checkSelfPermission(this, Manifest.permission.POST_NOTIFICATIONS) ==
PackageManager.PERMISSION_GRANTED
00084         } else {
00085             true
00086         }
00087
00088         NotificationHelper.createNotificationChannel(this, strings)
00089         enableEdgeToEdge()
00090
00091         MobileAds.initialize(this) {}
00092
00093         setContent {
00094             var themeMode by remember { mutableStateOf(prefManager.getThemeMode()) }
00095             var currentLanguage by remember { mutableStateOf(prefManager.getLanguage()) }
00096             var notificationsRefused by remember {
mutableStateOf(prefManager.wasNotificationsRefused()) }
00097
00098             val darkTheme = when (themeMode) {
00099                 ThemeMode.LIGHT -> false
00100                 ThemeMode.DARK -> true
00101                 ThemeMode.SYSTEM -> isSystemInDarkTheme()
00102             }
00103
00104             OtomotUZplusTheme(darkTheme = darkTheme) {
00105                 OtomotUZplusApp(
00106                     repository = repository,
00107                     themeMode = themeMode,
00108                     onThemeChange = {
00109                         themeMode = it
00110                         prefManager.setThemeMode(it)
00111                     },
00112                     currentLanguage = currentLanguage,
00113                     onLanguageChange = {
00114                         currentLanguage = it
00115                         prefManager.setLanguage(it)
00116                     },
00117                     notificationsRefused = notificationsRefused,
00118                     onSetNotificationsRefused = { refused ->
00119                         notificationsRefused = refused
00120                         prefManager.setNotificationsRefused(refused)
00121                     },
00122                     onRequestNotificationPermission = {
00123                         requestPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS)
00124                     },
00125                     notificationsPermissionGranted = hasNotificationPermission
00126                 )
00127             }
00128         }
00129
00130         FirebaseMessaging.getInstance().token.addOnCompleteListener { task ->
00131             if (!task.isSuccessful) {
00132                 Log.w("FCM_DIAG", "Pobieranie tokenu nie powiodło się", task.exception)
00133                 return@addOnCompleteListener
00134             }
00135             val token = task.result
00136             Log.d("FCM_DIAG", "SUKCES! Twój token FCM to: $token")
00137
00138             try {
00139                 val currentUserId = FirebaseAuth.getInstance().currentUser?.uid
00140                 if (currentUserId != null) {
00141
00142                     var isInitialSnapshot = true
00143
00144                     db.collection("notifications")
00145                         .whereEqualTo("toUser", currentUserId)
00146                         .addSnapshotListener { snapshots, e ->
00147                             if (e != null) {
00148                                 Log.w("FCM_DIAG", "Błąd nasłuchiwanie powiadomień", e)
00149                                 return@addSnapshotListener
00150                             }
00151
00152                             if (isInitialSnapshot) {
00153                                 isInitialSnapshot = false
00154                                 return@addSnapshotListener
00155                             }

```

```

00156
00157         for (dc in snapshots!!.documentChanges) {
00158             if (dc.type == DocumentChange.Type.ADDED) {
00159                 val carTitle = dc.document.getString("carTitle") ?: "Auto"
00160                 val message = strings.likeNotificationMessage.format(carTitle)
00161
00162                 NotificationHelper.sendNotification(
00163                     this@MainActivity,
00164                     strings.likeNotificationTitle,
00165                     message
00166                 )
00167             }
00168         }
00169     }
00170 }
00171 } catch (e: Exception) {
00172     Log.e("FCM_DIAG", "Błąd podczas zapisu tokenu do repozytorium: ${e.message}", e)
00173 }
00174 }
00175 }
00176 }
00177
00178 /**
00179  * Główna kompozycja posiadająca cały współdzielony stan nawigacji i UI aplikacji.
00180  *
00181  * ## Model nawigacji
00182  * Nawigacja sterowana stanem zamiast trasami:
00183  * - `currentDestination` wybiera który ekran zakładki jest pokazany wewnątrz
00184  *   [NavigationSuiteScaffold].
00185  * - `selectedCar != null` nakłada [AdDetailScreen] na cały scaffold.
00186  * - `showSettings == true` nakłada [SettingsScreen] na cały scaffold.
00187  * - [BackHandler] zdejmuję warstwy: szczegóły → ustawienia → HOME.
00188  *
00189  * ## Głębokie linki między ekranami
00190  * Zmienne `pendingSearch*` przenoszą stan początkowych filtrów z ekranu Home
00191  * do [SearchScreen]. [SearchScreen] konsumuje je raz przez `onInitialFiltersConsumed`.
00192  *
00193  * ## Przepływ danych
00194  * `allCarsFromDb` wypełniane raz przez [LaunchedEffect] subskrybując
00195  * [FirebaseRepository.observeCars] i przekazywane jawnie do każdego ekranu
00196  * który go potrzebuje -- brak warstwy ViewModel ani DI.
00197  *
00198  * @param repository Singleton dostępu do danych.
00199  * @param themeMode Aktywny motyw.
00200  * @param onThemeChange Callback do zapisania nowego motywu.
00201  * @param currentLanguage Aktywny identyfikator języka.
00202  * @param onLanguageChange Callback do zapisania nowego języka.
00203  * @param notificationsRefused Czy użytkownik odrzucił okno dialogowe uprawnień.
00204  * @param onSetNotificationsRefused Callback do zapisania flagi odrzucenia.
00205  * @param onRequestNotificationPermission Callback uruchamiający systemowe okno uprawnień.
00206  * @param notificationsPermissionGranted Aktualny stan przyznania uprawnień.
00207  */
00208 @Composable
00209 fun OtomotUZplusApp(
00210     repository: FirebaseRepository,
00211     themeMode: ThemeMode,
00212     onThemeChange: (ThemeMode) -> Unit,
00213     currentLanguage: String,
00214     onLanguageChange: (String) -> Unit,
00215     notificationsRefused: Boolean,
00216     onSetNotificationsRefused: (Boolean) -> Unit,
00217     onRequestNotificationPermission: () -> Unit,
00218     notificationsPermissionGranted: Boolean
00219 ) {
00220     val strings = if (currentLanguage == "Polski") PolishStrings else EnglishStrings
00221     val context = LocalContext.current
00222
00223     val shouldShowDialog = !notificationsPermissionGranted && !notificationsRefused &&
00224         Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU
00225     var currentDestination by rememberSaveable { mutableStateOf(AppDestinations.HOME) }
00226     var showSettings by rememberSaveable { mutableStateOf(false) }
00227     var pendingSearchQuery by rememberSaveable { mutableStateOf<String?>(null) }
00228     var pendingSearchBrand by rememberSaveable { mutableStateOf<String?>(null) }
00229     var pendingSearchShowFilters by rememberSaveable { mutableStateOf<Boolean?>(null) }
00230     var favoriteCars by rememberSaveable { mutableStateOf(emptyList<String>()) }
00231     var selectedCar by remember { mutableStateOf<com.example.otomotuzplus.models.CarAd?>(null) }
00232     var allCarsFromDb by remember {
00233         mutableStateOf<List<com.example.otomotuzplus.models.CarAd>>(emptyList()) }
00234
00235     LaunchedEffect(Unit) {
00236         repository.observeCars { updatedCars ->
00237             allCarsFromDb = updatedCars
00238         }
00239     }
00240
00241     val toggleFavorite: (String) -> Unit = { key ->
00242         Log.d("FCM_DIAG", "=== KLIKNIEŃTO SERDUSZKO ===")

```

```

00242         Log.d("FCM_DIAG", "Otrzymany klucz (key): $key")
00243
00244         if (favoriteCars.contains(key)) {
00245             Log.d("FCM_DIAG", "Auto było już w ulubionych -> USUWANIE z ulubionych (brak
powiadomienia).")
00246             favoriteCars = favoriteCars - key
00247         } else {
00248             Log.d("FCM_DIAG", "Auto nie ma w ulubionych -> DODAWANIE do ulubionych.")
00249             favoriteCars = favoriteCars + key
00250
00251             val realId = key.substringBefore("|")
00252             Log.d("FCM_DIAG", "Wycięte czyste ID do bazy danych: $realId")
00253             Log.d("FCM_DIAG", "Liczba aut w lokalnej pamięci podręcznej (allCarsFromDb):
${allCarsFromDb.size}")
00254
00255             val likedCar = allCarsFromDb.find { it.id == realId }
00256             if (likedCar != null) {
00257                 Log.d("FCM_DIAG", "Sukces! Znaleziono auto w bazie: ${likedCar.title}")
00258                 Log.d("FCM_DIAG", "Wartość sellerId dla tego auta to: '${likedCar.sellerId}'")
00259
00260                 if (likedCar.sellerId.isNotEmpty()) {
00261                     Log.d("FCM_DIAG", "Próba uruchomienia repository.sendLikeNotification...")
00262                     repository.sendLikeNotification(likedCar.sellerId, likedCar.title)
00263                 } else {
00264                     Log.w("FCM_DIAG", "OSTRZEŻENIE: sellerId jest PUSTE. Nie ma dokąd wysłać
powiadomienia.")
00265                 }
00266             } else {
00267                 Log.w("FCM_DIAG", "BŁĄD: Mimo wycięcia ID ($realId), nadal nie ma takiego auta w
allCarsFromDb.")
00268             }
00269         }
00270     }
00271     var showRationaleDialog by rememberSaveable { mutableStateOf(false) }
00272
00273     fun openSearch(query: String? = null, brand: String? = null, showFilters: Boolean? = false) {
00274         pendingSearchQuery = query
00275         pendingSearchBrand = brand
00276         pendingSearchShowFilters = showFilters
00277         if (currentDestination != AppDestinations.SEARCH) {
00278             currentDestination = AppDestinations.SEARCH
00279         }
00280     }
00281
00282     BackHandler(enabled = selectedCar != null || showSettings || currentDestination !=
AppDestinations.HOME) {
00283         if (selectedCar != null) {
00284             selectedCar = null
00285         } else if (showSettings) {
00286             showSettings = false
00287         } else {
00288             currentDestination = AppDestinations.HOME
00289         }
00290     }
00291
00292     if (shouldShowDialog) {
00293         LaunchedEffect(Unit) {
00294             val permission = Manifest.permission.POST_NOTIFICATIONS
00295             val isGranted = ContextCompat.checkSelfPermission(context, permission) ==
PackageManager.PERMISSION_GRANTED
00296             if (!isGranted) {
00297                 showRationaleDialog = true
00298             }
00299         }
00300
00301         if (showRationaleDialog) {
00302             AlertDialog(
00303                 onDismissRequest = { },
00304                 icon = { Icon(Icons.Filled.Notifications, contentDescription = null, tint =
MaterialTheme.colorScheme.primary) },
00305                 title = { Text(text = strings.dealNotification, style =
MaterialTheme.typography.headlineSmall) },
00306                 text = { Text(text = strings.dealNotificationDescription, style =
MaterialTheme.typography.bodyMedium) },
00307                 confirmButton = { Button(onClick = { onRequestNotificationPermission() }) {
Text(strings.enable) } },
00308                 dismissButton = { TextButton(onClick = { onSetNotificationsRefused(true) }) {
Text(strings.maybeLater) } }
00309             )
00310         }
00311     }
00312
00313     val navItems = listOf(
00314         NavigationItem(AppDestinations.HOME, strings.home, Icons.Filled.Home, Icons.Outlined.Home),
00315         NavigationItem(AppDestinations.SEARCH, strings.search, Icons.Filled.Search,
Icons.Outlined.Search),
00316         NavigationItem(AppDestinations.ADD, strings.add, Icons.Filled.AddCircle,

```

```

Icons.Outlined.AddCircle),
00317     NavigationItem(AppDestinations.FAVORITES, strings.favorites, Icons.Filled.Favorite,
Icons.Outlined.FavoriteBorder),
00318     NavigationItem(AppDestinations.PROFILE, strings.profile, Icons.Filled.Person,
Icons.Outlined.Person)
00319 )
00320
00321 val navItemColors = NavigationSuiteDefaults.itemColors(
00322     navigationBarItemColors = NavigationBarItemDefaults.colors(
00323         selectedIconColor = MaterialTheme.colorScheme.primary,
00324         selectedTextColor = MaterialTheme.colorScheme.primary,
00325         unselectedIconColor = Color.Gray,
00326         unselectedTextColor = Color.Gray,
00327         indicatorColor = Color.Transparent
00328     )
00329 )
00330
00331 val navSuiteColors = NavigationSuiteDefaults.colors(
00332     navigationBarContainerColor = MaterialTheme.colorScheme.background,
00333     navigationBarContentColor = Color.Gray
00334 )
00335
00336 if (selectedCar != null) {
00337     com.example.otomotuzplus.ui.screens.details.AdDetailScreen(
00338         car = selectedCar!!,
00339         strings = strings,
00340         onBackClick = { selectedCar = null }
00341     )
00342 } else if (showSettings) {
00343     SettingsScreen(
00344         onBack = { showSettings = false },
00345         themeMode = themeMode,
00346         onThemeChange = onThemeChange,
00347         currentLanguage = currentLanguage,
00348         onLanguageChange = onLanguageChange,
00349         strings = strings
00350     )
00351 } else {
00352     NavigationSuiteScaffold(
00353         navigationSuiteItems = {
00354             navItems.forEach { item ->
00355                 item(
00356                     icon = {
00357                         Icon(
00358                             imageVector = if (item.destination == currentDestination)
item.selectedIcon else item.unselectedIcon,
00359                             contentDescription = item.label
00360                         )
00361                     },
00362                     label = { Text(item.label) },
00363                     selected = item.destination == currentDestination,
00364                     onClick = { currentDestination = item.destination },
00365                     colors = navItemColors
00366                 )
00367             }
00368         },
00369         containerColor = MaterialTheme.colorScheme.background,
00370         navigationSuiteColors = navSuiteColors
00371     ) {
00372         Scaffold(
00373             modifier = Modifier.fillMaxSize(),
00374             containerColor = MaterialTheme.colorScheme.background
00375         ) { innerPadding ->
00376             Box(modifier = Modifier.padding(innerPadding).fillMaxSize()) {
00377                 when (currentDestination) {
00378                     AppDestinations.HOME -> HomeScreen(
00379                         strings = strings,
00380                         carsFromDb = allCarsFromDb,
00381                         onNavigateToSearch = { openSearch() },
00382                         onNavigateToAdd = { currentDestination = AppDestinations.ADD },
00383                         onSearchSubmit = { submittedQuery ->
00384                             if (submittedQuery.isNotBlank()) {
00385                                 openSearch(query = submittedQuery)
00386                             }
00387                         },
00388                         onBrandSelect = { selectedBrand -> openSearch(brand = selectedBrand) },
00389                         onSeeAllClick = { openSearch() },
00390                         onNotificationsClick = { },
00391                         favoriteCars = favoriteCars,
00392                         onFavoriteToggle = toggleFavorite,
00393                         onCarClick = { clickedCar -> selectedCar = clickedCar }
00394                     )
00395                     AppDestinations.SEARCH -> SearchScreen(
00396                         strings = strings,
00397                         initialQuery = pendingSearchQuery,
00398                         initialBrand = pendingSearchBrand,

```



```

00400             initialShowFilters = pendingSearchShowFilters,
00401             favoriteCars = favoriteCars,
00402             onFavoriteToggle = toggleFavorite,
00403             onInitialFiltersConsumed = {
00404                 pendingSearchQuery = null
00405                 pendingSearchBrand = null
00406                 pendingSearchShowFilters = null
00407             },
00408             allCarsFromDb = allCarsFromDb,
00409             onCarClick = { clickedCar -> selectedCar = clickedCar }
00410         )
00411
00412         AppDestinations.ADD -> com.example.otomotuzplus.ui.screens.add.AddScreen(
00413             strings = strings,
00414             onNavigateBack = { currentDestination = AppDestinations.HOME }
00415         )
00416
00417         AppDestinations.FAVORITES -> FavoritesScreen(
00418             strings = strings,
00419             favoriteCars = favoriteCars,
00420             onFavoriteToggle = toggleFavorite,
00421             allCarsFromDb = allCarsFromDb,
00422             onCarClick = { clickedCar -> selectedCar = clickedCar }
00423         )
00424
00425         AppDestinations.PROFILE -> ProfileScreen(
00426             onSettingsClick = { showSettings = true },
00427             strings = strings,
00428             allCarsFromDb = allCarsFromDb,
00429             onCarClick = { clickedCar -> selectedCar = clickedCar }
00430         )
00431     }
00432 }
00433
00434 }
00435 }
00436 }

```

4.15 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/models/CarAd.kt

Model dokumentu Firestore reprezentujący ogłoszenie pojazdu.

4.15.1 Opis szczegółowy

Model dokumentu Firestore reprezentujący ogłoszenie pojazdu.

Definicja w pliku [CarAd.kt](#).

4.16 CarAd.kt

[Idź do dokumentacji tego pliku.](#)

```

00001 /**
00002  * @file CarAd.kt
00003  * @brief Model dokumentu Firestore reprezentujący ogłoszenie pojazdu.
00004  */
00005 package com.example.otomotuzplus.models
00006
00007 /**
00008  * Reprezentuje pojedyncze ogłoszenie pojazdu przechowywane w Firestore.
00009  *
00010  * Instancje tworzone przez deserializację dokumentów Firestore za pomocą
00011  * `document.toObject(CarAd::class.java).copy(id = document.id)`.
00012  * Wszystkie pola tekstowe odzwierciedlają surowe dane użytkownika; wartości
00013  * numeryczne (cena, rok, przebieg) przechowywane jako ciągi znaków, aby
00014  * zachować częściowe lub dowolne dane wejściowe.
00015  *
00016  * @property id Identyfikator dokumentu Firestore; pusty ciąg do momentu odczytu.
00017  * @property title Tytuł ogłoszenia wpisany przez sprzedającego (np. "BMW M4 2022").
00018  * @property priceText Surowy ciąg ceny wpisany przez sprzedającego (np. "429 000").
00019  * @property locationText Czytelna nazwa miasta lub obszaru (np. "Kraków").

```

```

00020 * @property postalCode Polski kod pocztowy w formacie "XX-XXX"; używany do geokodowania.
00021 * @property latitude Szerokość geograficzna WGS-84 wyznaczona z [postalCode] przez Nominatim; 0.0
    jeśli niedostępna.
00022 * @property longitude Długość geograficzna WGS-84 wyznaczona z [postalCode] przez Nominatim; 0.0
    jeśli niedostępna.
00023 * @property year Rok produkcji jako czterocyfrowy ciąg (np. "2022").
00024 * @property mileageText Przebieg wpisany przez sprzedającego (np. "15000"); jednostka dodawana przez
    UI.
00025 * @property fuelText Surowa wartość rodzaju paliwa z Firestore (może być po polsku lub angielsku).
00026 * @property gearboxText Surowa wartość skrzyni biegów z Firestore (może być po polsku lub angielsku).
00027 * @property engineCapacity Pojemność silnika (np. "1998"); jednostka dodawana przez UI.
00028 * @property powerText Moc silnika (np. "306"); jednostka dodawana przez UI.
00029 * @property imageUrls Uporządkowana lista adresów URL pobierania zdjęć z Firebase Storage.
00030 * @property sellerId UID konta sprzedającego w Firebase Auth.
00031 * @property sellerEmail Adres e-mail konta sprzedającego.
00032 * @property phoneNumber Numer telefonu kontaktowego sprzedającego; ukryty za gestem potrząśnięcia.
00033 */
00034 data class CarAd(
00035     val id: String = "",
00036     val title: String = "",
00037     val priceText: String = "",
00038     val locationText: String = "",
00039     val postalCode: String = "",
00040     val latitude: Double = 0.0,
00041     val longitude: Double = 0.0,
00042     val year: String = "",
00043     val mileageText: String = "",
00044     val fuelText: String = "",
00045     val gearboxText: String = "",
00046     val engineCapacity: String = "",
00047     val powerText: String = "",
00048     val imageUrls: List<String> = emptyList(),
00049     val sellerId: String = "",
00050     val sellerEmail: String = "",
00051     val phoneNumber: String = ""
00052 )

```

4.17 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/RegisterActivity.kt

Ekran rejestracji nowego konta użytkownika przez Firebase Auth.

Pakiety

- package [AppCompatActivity](#)

Funkcje

- Object [AppCompatActivity.onCreate](#) (Object savedInstanceState)

4.17.1 Opis szczegółowy

Ekran rejestracji nowego konta użytkownika przez Firebase Auth.

Definicja w pliku [RegisterActivity.kt](#).

4.18 RegisterActivity.kt

[Idź do dokumentacji tego pliku.](#)

```

00001 /**
00002  * @file RegisterActivity.kt
00003  * @brief Ekran rejestracji nowego konta użytkownika przez Firebase Auth.
00004  */
00005 package com.example.otomotuzplus
00006
00007 import android.content.Intent
00008 import android.os.Bundle
00009 import android.widget.Button
00010 import android.widget.TextView
00011 import android.widget.Toast
00012 import androidx.appcompat.app.AppCompatActivity
00013 import com.google.android.material.textfield.TextInputEditText
00014 import com.google.firebase.auth.FirebaseAuth
00015
00016 /**
00017  * Activity obsługująca tworzenie nowych kont przez Firebase Auth.
00018  *
00019  * Sprawdza czy:
00020  * - Wszystkie trzy pola (e-mail, hasło, powtórz hasło) są niepuste.
00021  * - Oba pola hasła są zgodne.
00022  * - Hasło ma co najmniej 6 znaków.
00023  *
00024  * Po sukcesie nawiguje bezpośrednio do [MainActivity], czyszcząc stos back.
00025  * Kliknięcie linku "przejdź do logowania" wywołuje `finish()` aby wrócić do
00026  * [LoginActivity].
00027  */
00028 class RegisterActivity : AppCompatActivity() {
00029     private lateinit var auth: FirebaseAuth
00030
00031     override fun onCreate(savedInstanceState: Bundle?) {
00032         super.onCreate(savedInstanceState)
00033         window.statusBarColor = android.graphics.Color.parseColor("#FBD70E")
00034         androidx.core.view.WindowCompat.getInsetsController(window,
00035             window.decorView).isAppearanceLightStatusBars = true
00036         setContentView(R.layout.activity_register)
00037
00038         auth = FirebaseAuth.getInstance()
00039
00040         val emailET = findViewById<TextInputEditText>(R.id.emailEditText)
00041         val passwordET = findViewById<TextInputEditText>(R.id.passwordEditText)
00042         val repeatPasswordET = findViewById<TextInputEditText>(R.id.repeatPasswordEditText)
00043         val registerBtn = findViewById<Button>(R.id.registerButton)
00044         val loginTV = findViewById<TextView>(R.id.goToLoginText)
00045
00046         registerBtn.setOnClickListener {
00047             val email = emailET.text.toString().trim()
00048             val pass = passwordET.text.toString().trim()
00049             val repeatPass = repeatPasswordET.text.toString().trim()
00050
00051             if (email.isEmpty() || pass.isEmpty() || repeatPass.isEmpty()) {
00052                 Toast.makeText(this, "Wypełnij wszystkie pola", Toast.LENGTH_SHORT).show()
00053                 return@setOnClickListener
00054             }
00055
00056             if (pass != repeatPass) {
00057                 Toast.makeText(this, "Hasła nie są identyczne!", Toast.LENGTH_SHORT).show()
00058                 return@setOnClickListener
00059             }
00060
00061             if (pass.length < 6) {
00062                 Toast.makeText(this, "Hasło musi mieć min. 6 znaków", Toast.LENGTH_SHORT).show()
00063                 return@setOnClickListener
00064             }
00065
00066             auth.createUserWithEmailAndPassword(email, pass)
00067                 .addOnCompleteListener(this) { task ->
00068                     if (task.isSuccessful) {
00069                         Toast.makeText(this, "Konto utworzone pomyślnie!", Toast.LENGTH_SHORT).show()
00070                         val intent = Intent(this, MainActivity::class.java)
00071                         intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or
00072                             Intent.FLAG_ACTIVITY_CLEAR_TASK
00073                         startActivity(intent)
00074                         finish()
00075                     } else {
00076                         Toast.makeText(this, "Błąd rejestracji: ${task.exception?.message}",
00077                             Toast.LENGTH_LONG).show()
00078                     }
00079                 }
00080
00081             loginTV.setOnClickListener {

```

```

00081         finish()
00082     }
00083 }
00084 }

```

4.19 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/components/AdmobBanner.kt

Komponent baneru reklamowego AdMob.

Pakiety

- package [modifier](#)

4.19.1 Opis szczegółowy

Komponent baneru reklamowego AdMob.

Definicja w pliku [AdmobBanner.kt](#).

4.20 AdmobBanner.kt

[Idź do dokumentacji tego pliku.](#)

```

00001 /**
00002  * @file AdmobBanner.kt
00003  * @brief Komponent baneru reklamowego AdMob.
00004  */
00005 package com.example.otomotuzplus.ui.components
00006
00007 import androidx.compose.foundation.layout.fillMaxWidth
00008 import androidx.compose.runtime.Composable
00009 import androidx.compose.ui.Modifier
00010 import androidx.compose.ui.viewinterop.AndroidView
00011 import com.google.android.gms.ads.AdRequest
00012 import com.google.android.gms.ads.AdSize
00013 import com.google.android.gms.ads.AdView
00014
00015 /**
00016  * Wyświetla reklamę banerową Google AdMob przy użyciu opakowania interop [AndroidView].
00017  *
00018  * ID jednostki reklamowej ustawione na testowe ID banera AdMob
00019  * ('ca-app-pub-3940256099942544/6300978111'), aby nie generować prawdziwego ruchu
00020  * w buildach deweloperskich. Przed wydaniem zastąp produkcyjnym ID.
00021  *
00022  * @param modifier Opcjonalny [Modifier] stosowany do opakowania [AndroidView];
00023  *   baner zawsze wypełnia dostępną szerokość.
00024  */
00025 @Composable
00026 fun AdmobBanner(modifier: Modifier = Modifier) {
00027     AndroidView(
00028         modifier = modifier.fillMaxWidth(),
00029         factory = { context ->
00030             AdView(context).apply {
00031                 setAdSize(AdSize.BANNER)
00032                 adUnitId = "ca-app-pub-3940256099942544/6300978111"
00033                 loadAd(AdRequest.Builder().build())
00034             }
00035         }
00036     )
00037 }

```

4.21 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/components/CommonComponents.kt

Wspólne komponenty UI: nagłówek ekranu, placeholder, element ustawień.

Pakiety

- package [name](#)

4.21.1 Opis szczegółowy

Wspólne komponenty UI: nagłówek ekranu, placeholder, element ustawień.

Definicja w pliku [CommonComponents.kt](#).

4.22 CommonComponents.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file CommonComponents.kt
00003  * @brief Wspólne komponenty UI: nagłówek ekranu, placeholder, element ustawień.
00004  */
00005 package com.example.otomotuzplus.ui.components
00006
00007 import androidx.compose.foundation.clickable
00008 import androidx.compose.foundation.layout.*
00009 import androidx.compose.material.icons.Icons
00010 import androidx.compose.material.icons.filled.ChevronRight
00011 import androidx.compose.material3.Icon
00012 import androidx.compose.material3.MaterialTheme
00013 import androidx.compose.material3.Text
00014 import androidx.compose.runtime.Composable
00015 import androidx.compose.ui.Alignment
00016 import androidx.compose.ui.Modifier
00017 import androidx.compose.ui.graphics.Color
00018 import androidx.compose.ui.graphics.vector.ImageVector
00019 import androidx.compose.ui.text.font.FontWeight
00020 import androidx.compose.ui.unit.Dp
00021 import androidx.compose.ui.unit.dp
00022 import androidx.compose.ui.unit.sp
00023
00024 /**
00025  * Wiersz nagłówka na pełną szerokość używany na górze każdego głównego ekranu.
00026  *
00027  * Nagłówek zawiera pogrubiony tytuł wyśrodkowany w dostępnej przestrzeni oraz
00028  * opcjonalny slot końcowy (np. przyciski ikon) wyrównany do prawej krawędzi.
00029  *
00030  * @param title          Tekst wyświetlany jako tytuł ekranu.
00031  * @param modifier       Opcjonalny [Modifier] stosowany do zewnętrznego [Row].
00032  * @param trailingWidth   Stała szerokość zarezerwowana dla [trailingContent]; domyślnie 112 dp.
00033  * @param trailingContent Zawartość kompozycji umieszczona w slotie końcowym (np. przyciski ikon).
00034  */
00035 @Composable
00036 fun ScreenHeader(
00037     title: String,
00038     modifier: Modifier = Modifier,
00039     trailingWidth: Dp = 112.dp,
00040     trailingContent: @Composable RowScope.() -> Unit = {}
00041 ) {
00042     Row(
00043         modifier = modifier
00044         .fillMaxWidth()
00045         .padding(horizontal = 16.dp),
00046         verticalAlignment = Alignment.CenterVertically
00047     ) {
00048         Spacer(modifier = Modifier.width(28.dp))
00049         Text(
00050             text = title,
00051             modifier = Modifier.weight(1f),
```

```

00052         fontSize = 24.sp,
00053         fontWeight = FontWeight.Bold,
00054         color = MaterialTheme.colorScheme.onBackground
00055     )
00056     Box(
00057         modifier = Modifier.width(trailingWidth),
00058         contentAlignment = Alignment.CenterEnd
00059     ) {
00060         Row(
00061             horizontalArrangement = Arrangement.End,
00062             verticalAlignment = Alignment.CenterVertically,
00063             content = trailingContent
00064         )
00065     }
00066 }
00067 }
00068
00069 /**
00070  * Wyśrodkowany zastępczy tekst używany podczas tworzenia dla niezaimplementowanych ekranów.
00071  *
00072  * @param name Tekst do wyświetlenia na środku ekranu.
00073  */
00074 @Composable
00075 fun PlaceholderScreen(name: String) {
00076     Column(
00077         modifier = Modifier.fillMaxSize(),
00078         horizontalAlignment = Alignment.CenterHorizontally,
00079         verticalArrangement = Arrangement.Center
00080     ) {
00081         Text(
00082             text = name,
00083             fontSize = 24.sp,
00084             fontWeight = FontWeight.Bold,
00085             color = MaterialTheme.colorScheme.onBackground
00086         )
00087     }
00088 }
00089
00090 /**
00091  * Klikalny wiersz ustawień z wiodącą ikoną, tytułem i końcowym znacznikiem strzałki.
00092  *
00093  * Używany wyłącznie w [com.example.otomotuzplus.ui.screens.settings.SettingsScreen].
00094  *
00095  * @param title Etykieta tekstowa wyświetlana w wierszu.
00096  * @param icon Wiodąca ikona wyświetlana po lewej stronie tytułu.
00097  * @param onClick Callback wywoływany gdy wiersz zostanie kliknięty.
00098  */
00099 @Composable
00100 fun SettingsClickItem(
00101     title: String,
00102     icon: ImageVector,
00103     onClick: () -> Unit
00104 ) {
00105     Row(
00106         modifier = Modifier
00107             .fillMaxWidth()
00108             .clickable { onClick() }
00109             .padding(vertical = 12.dp),
00110         verticalAlignment = Alignment.CenterVertically
00111     ) {
00112         Icon(
00113             imageVector = icon,
00114             contentDescription = null,
00115             tint = MaterialTheme.colorScheme.onBackground
00116         )
00117         Spacer(modifier = Modifier.width(16.dp))
00118         Text(
00119             text = title,
00120             color = MaterialTheme.colorScheme.onBackground,
00121             modifier = Modifier.weight(1f)
00122         )
00123         Icon(
00124             imageVector = Icons.Default.ChevronRight,
00125             contentDescription = null,
00126             tint = Color.Gray
00127         )
00128     }
00129 }

```

4.23 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/components/ListingCard.kt

Karta ogłoszenia pojazdu używana na ekranach Home, Search i Favorites.

Pakiety

- package [ListingCardData](#)

Zmienne

- Object [ListingCardData.title](#)
Karta Material3 wyświetlająca skrócone podsumowanie ogłoszenia pojazdu.
- Object [ListingCardData.year](#)
- Object [ListingCardData.mileageText](#)
- Object [ListingCardData.fuelText](#)
- Object [ListingCardData.bodyTypeText](#)
- Object [ListingCardData.driveTypeText](#)
- Object [ListingCardData.locationText](#)
- Object [ListingCardData.priceText](#)
- Object [ListingCardData.isFavorite](#)
- Object [ListingCardData.onFavoriteClick](#)
- Object [ListingCardData.coverImageUrl](#)
- Object [ListingCardData.metadataLine1](#)
- Object [ListingCardData.metadataLine2](#)
- Object [ListingCardData.price](#)

4.23.1 Opis szczegółowy

Karta ogłoszenia pojazdu używana na ekranach Home, Search i Favorites.

Definicja w pliku [ListingCard.kt](#).

4.24 ListingCard.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file ListingCard.kt
00003  * @brief Karta ogłoszenia pojazdu używana na ekranach Home, Search i Favorites.
00004  */
00005 package com.example.otomotuzplus.ui.components
00006
00007 import androidx.compose.foundation.background
00008 import androidx.compose.foundation.layout.Arrangement
00009 import androidx.compose.foundation.layout.Box
00010 import androidx.compose.foundation.layout.Column
00011 import androidx.compose.foundation.layout.Spacer
00012 import androidx.compose.foundation.layout.fillMaxSize
00013 import androidx.compose.foundation.layout.fillMaxWidth
00014 import androidx.compose.foundation.layout.height
00015 import androidx.compose.foundation.layout.padding
00016 import androidx.compose.material.icons.Icons
00017 import androidx.compose.material.icons.filled.Favorite
00018 import androidx.compose.material.icons.filled.FavoriteBorder
00019 import androidx.compose.material3.Card
00020 import androidx.compose.material3.CardDefaults
00021 import androidx.compose.material3.Icon
00022 import androidx.compose.material3.IconButton
00023 import androidx.compose.material3.MaterialTheme
00024 import androidx.compose.material3.Surface
00025 import androidx.compose.material3.Text
00026 import androidx.compose.runtime.Composable
00027 import androidx.compose.ui.Alignment
00028 import androidx.compose.ui.Modifier
00029 import androidx.compose.ui.graphics.Color
00030 import androidx.compose.ui.text.font.FontWeight
00031 import androidx.compose.ui.text.style.TextOverflow
00032 import androidx.compose.ui.unit.dp
```

```

00033 import androidx.compose.ui.unit.sp
00034 import com.example.otomotuzplus.ui.theme.BrandGold
00035 import com.example.otomotuzplus.ui.theme.Slate400
00036 import coil.compose.AsyncImage
00037 import androidx.compose.ui.layout.ContentScale
00038
00039 /**
00040  * Model widoku dla pojedynczej karty ogłoszenia.
00041  *
00042  * Wszystkie pola tekstowe powinny już zawierać sufiksy jednostek
00043  * (np. `"98 400 km"`, `"19 000 zł"`), ponieważ sufiks rozwiązywany jest
00044  * z [AppStrings] przed zbudowaniem karty.
00045  *
00046  * @property title      Tytuł ogłoszenia (np. "Opel Corsa 1.2").
00047  * @property year       Ciąg roku produkcji.
00048  * @property mileageText Przebieg z sufiksem jednostki.
00049  * @property fuelText    Zlokalizowana etykieta rodzaju paliwa.
00050  * @property bodyTypeText Zlokalizowana etykieta skrzyni biegów (ponownie użyta w układzie karty).
00051  * @property driveTypeText Ciąg pojemności silnika z sufiksem jednostki (ponownie użyty).
00052  * @property locationText Czytelna nazwa miasta lub obszaru.
00053  * @property priceText   Cena z sufiksem waluty.
00054  * @property isFavorite  Czy bieżący użytkownik dodał ogłoszenie do ulubionych.
00055  * @property onFavoriteClick Callback wywoływany gdy użytkownik kliknie ikonę serduszka.
00056  * @property coverImageUrl Opcjonalny URL Firebase Storage dla pierwszego zdjęcia ogłoszenia.
00057  */
00058 data class ListingCardData(
00059     val title: String,
00060     val year: String,
00061     val mileageText: String,
00062     val fuelText: String,
00063     val bodyTypeText: String,
00064     val driveTypeText: String,
00065     val locationText: String,
00066     val priceText: String,
00067     val isFavorite: Boolean,
00068     val onFavoriteClick: () -> Unit,
00069     val coverImageUrl: String? = null
00070 )
00071
00072 /**
00073  * Karta Material3 wyświetlająca skrócone podsumowanie ogłoszenia pojazdu.
00074  *
00075  * Karta pokazuje zdjęcie okładki (lub tło zastępcze), przycisk przełącznika
00076  * ulubionych w prawym górnym rogu i dwa wiersze metadanych pod tytułem.
00077  *
00078  * @param item      Dane ogłoszenia do wyświetlenia.
00079  * @param modifier Opcjonalny [Modifier] stosowany do zewnętrznej [Card].
00080  */
00081 @Composable
00082 fun ListingCard(
00083     item: ListingCardData,
00084     modifier: Modifier = Modifier
00085 ) {
00086     val title = item.title.trim()
00087     val metadataLine1 = listOf(item.year, item.mileageText, item.fuelText)
00088         .map { it.trim() }
00089         .filter { it.isNotEmpty() }
00090         .joinToString(" • ")
00091     val metadataLine2 = listOf(item.bodyTypeText, item.driveTypeText, item.locationText)
00092         .map { it.trim() }
00093         .filter { it.isNotEmpty() }
00094         .joinToString(" • ")
00095     val price = item.priceText.trim()
00096
00097     Card(
00098         modifier = modifier.fillMaxWidth(),
00099         shape = androidx.compose.foundation.shape.RoundedCornerShape(18.dp),
00100         colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
00101         elevation = CardDefaults.cardElevation(defaultElevation = 3.dp)
00102     ) {
00103         Column {
00104             Box(
00105                 modifier = modifier
00106                     .fillMaxWidth()
00107                     .height(155.dp)
00108                     .background(MaterialTheme.colorScheme.surfaceVariant)
00109             ) {
00110                 if (!item.coverImageUrl.isNullOrEmpty()) {
00111                     AsyncImage(
00112                         model = item.coverImageUrl,
00113                         contentDescription = "Okładka ogłoszenia",
00114                         modifier = Modifier.fillMaxSize(),
00115                         contentScale = ContentScale.Crop
00116                     )
00117                 }
00118                 Surface(
00119                     shape = androidx.compose.foundation.shape.RoundedCornerShape(14.dp),

```



```

00120             color = MaterialTheme.colorScheme.surface.copy(alpha = 0.92f),
00121             tonalElevation = 2.dp,
00122             modifier = Modifier
00123                 .align(Alignment.TopEnd)
00124                 .padding(10.dp)
00125         ) {
00126             IconButton(onClick = item.onFavoriteClick) {
00127                 Icon(
00128                     imageVector = if (item.isFavorite) Icons.Default.Favorite else
00129                     Icons.Default.FavoriteBorder,
00130                     contentDescription = null,
00131                     tint = if (item.isFavorite) Color(0xFFDC2626) else
00132                     MaterialTheme.colorScheme.onSurface,
00133                     modifier = Modifier.padding(2.dp)
00134                 )
00135             }
00136         }
00137     Column(modifier = Modifier.padding(14.dp), verticalArrangement =
00138         Arrangement.spacedBy(0.dp)) {
00139         if (title.isNotEmpty()) {
00140             Text(
00141                 text = title,
00142                 color = MaterialTheme.colorScheme.onSurface,
00143                 fontWeight = FontWeight.Bold,
00144                 fontSize = 18.sp,
00145                 maxLines = 1,
00146                 overflow = TextOverflow.Ellipsis
00147             )
00148         }
00149         if (metadataLine1.isNotEmpty()) {
00150             Spacer(modifier = Modifier.height(8.dp))
00151             Text(
00152                 text = metadataLine1,
00153                 color = Slate400,
00154                 fontSize = 14.sp
00155             )
00156         }
00157         if (metadataLine2.isNotEmpty()) {
00158             Spacer(modifier = Modifier.height(6.dp))
00159             Text(
00160                 text = metadataLine2,
00161                 color = Slate400,
00162                 fontSize = 14.sp
00163             )
00164         }
00165     }
00166     if (price.isNotEmpty()) {
00167         Spacer(modifier = Modifier.height(12.dp))
00168         Text(
00169             text = price,
00170             color = BrandGold,
00171             fontWeight = FontWeight.ExtraBold,
00172             fontSize = 21.sp
00173         )
00174     }
00175 }
00176 }
00177 }
00178 }
00179 }
00180

```

4.25 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/models/AppStrings.kt

Interfejs i implementacje i18n – polskie i angielskie ciągi UI.

4.25.1 Opis szczegółowy

Interfejs i implementacje i18n – polskie i angielskie ciągi UI.

Definicja w pliku [AppStrings.kt](#).

4.26 AppStrings.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file AppStrings.kt
00003  * @brief Interfejs i implementacje i18n - polskie i angielskie ciągi UI.
00004  */
00005 package com.example.otomotuzplus.ui.models
00006
00007 /**
00008  * Wszystkie widoczne dla użytkownika ciągi znaków dla jednego języka UI.
00009  *
00010  * Aplikacja używa dwóch instancji singletonów -- [PolishStrings] i [EnglishStrings] --
00011  * wybieranych w czasie działania na podstawie języka zapisanego w
00012  * [com.example.otomotuzplus.data.PreferenceManager]. Dodanie nowego ciągu
00013  * wymaga dodania pola tutaj i podania wartości w **obu** singletonach.
00014  *
00015  * Podejście celowo omija mechanizm zasobów `res/values-XX/strings.xml` systemu Android,
00016  * dzięki czemu zmiana języka działa natychmiast bez restartu Activity.
00017  */
00018 data class AppStrings(
00019     val home: String,
00020     val search: String,
00021     val add: String,
00022     val favorites: String,
00023     val profile: String,
00024     val searchPlaceholder: String,
00025     val buy: String,
00026     val sell: String,
00027     val findCar: String,
00028     val listYourCar: String,
00029     val popularBrands: String,
00030     val seeAll: String,
00031     val freshArrivals: String,
00032     val featured: String,
00033     val filters: String,
00034     val sortBy: String,
00035     val sortNameAsc: String,
00036     val sortNameDesc: String,
00037     val sortPriceAsc: String,
00038     val sortPriceDesc: String,
00039     val sortYearDesc: String,
00040     val sortMileageAsc: String,
00041     val brand: String,
00042     val model: String,
00043     val minPrice: String,
00044     val maxPrice: String,
00045     val minYear: String,
00046     val maxYear: String,
00047     val maxMileage: String,
00048     val fuelType: String,
00049     val fuelPetrol: String,
00050     val fuelDiesel: String,
00051     val fuelHybrid: String,
00052     val fuelElectric: String,
00053     val transmission: String,
00054     val transmissionAutomatic: String,
00055     val transmissionManual: String,
00056     val bodyType: String,
00057     val bodySedan: String,
00058     val bodySuv: String,
00059     val bodyCoupe: String,
00060     val bodyHatchback: String,
00061     val bodyWagon: String,
00062     val driveType: String,
00063     val driveFwd: String,
00064     val driveRwd: String,
00065     val driveAwd: String,
00066     val drive4x4: String,
00067     val condition: String,
00068     val conditionNew: String,
00069     val conditionUsed: String,
00070     val location: String,
00071     val resetFilters: String,
00072     val resultsCount: String,
00073     val noSearchResults: String,
00074     val myListings: String,
00075     val history: String,
00076     val settings: String,
00077     val logout: String,
00078     val theme: String,
00079     val themeSystem: String,
00080     val themeLight: String,
00081     val themeDark: String,
00082     val language: String,
00083     val notifications: String,
```

```
00084     val appPreferences: String,
00085     val noHistory: String,
00086     val memberSince: String,
00087     val chooseLanguage: String,
00088     val chooseTheme: String,
00089     val listingsStatus: String,
00090     val historyEmpty: String,
00091     val brandLogosPlaceholder: String,
00092     val noListingsYet: String,
00093     val verifiedListing: String,
00094     val seller: String,
00095     val back: String,
00096     val yearProduction: String,
00097     val mileage: String,
00098     val fuel: String,
00099     val gearbox: String,
00100     val engineCapacityLabel: String,
00101     val power: String,
00102     val unitCurrency: String,
00103     val unitKm: String,
00104     val unitCm3: String,
00105     val unitPower: String,
00106     val addListingTitle: String,
00107     val brandAndModel: String,
00108     val price: String,
00109     val year: String,
00110     val city: String,
00111     val addPhotos: String,
00112     val photoSourceTitle: String,
00113     val photoSourceMessage: String,
00114     val photoSourceGallery: String,
00115     val photoSourceCamera: String,
00116     val selectedPhotoContentDescription: String,
00117     val removePhotoContentDescription: String,
00118     val addListingSuccess: String,
00119     val addListingError: String,
00120     val addListingRequiredFields: String,
00121     val uploading: String,
00122     val postalCode: String,
00123     val postalCodeInvalid: String,
00124     val mapViewToggle: String,
00125     val listViewToggle: String,
00126     val searchRadius: String,
00127     val radiusAny: String,
00128     val addListingGeocodingWarning: String,
00129     val phoneNumber: String,
00130     val callSeller: String,
00131     val messageSeller: String,
00132     val noEmail: String,
00133     val notificationsSettingsDescription: String,
00134     val likeNotificationTitle: String,
00135     val likeNotificationMessage: String,
00136     val dealTestNotification: String,
00137     val dealNotification: String,
00138     val dealNotificationDescription: String,
00139     val enable: String,
00140     val maybeLater: String,
00141     val newOffers: String,
00142     val shakePhone: String,
00143     val noPhoneNumber: String
00144 )
00145
00146 val PolishStrings = AppStrings(
00147     home = "Start",
00148     search = "Szukaj",
00149     add = "Dodaj",
00150     favorites = "Ulubione",
00151     profile = "Profil",
00152     searchPlaceholder = "Szukaj samochodów, marek lub modeli",
00153     buy = "Kup",
00154     sell = "Sprzedaj",
00155     findCar = "Znajdź auto",
00156     listYourCar = "Dodaj ogłoszenie",
00157     popularBrands = "Popularne marki",
00158     seeAll = "Zobacz wszystko",
00159     freshArrivals = "Najnowsze oferty",
00160     featured = "Wyróżnione",
00161     filters = "Filtry",
00162     sortBy = "Sortuj",
00163     sortNameAsc = "Nazwa A-Z",
00164     sortNameDesc = "Nazwa Z-A",
00165     sortPriceAsc = "Cena rosnąco",
00166     sortPriceDesc = "Cena malejąco",
00167     sortYearDesc = "Rok od najnowszych",
00168     sortMileageAsc = "Przebieg rosnąco",
00169     brand = "Marka",
00170     model = "Model",
```

```
00171 minPrice = "Cena od",
00172 maxPrice = "Cena do",
00173 minYear = "Rok od",
00174 maxYear = "Rok do",
00175 maxMileage = "Maks. przebieg (km)",
00176 fuelType = "Rodzaj paliwa",
00177 fuelPetrol = "Benzyna",
00178 fuelDiesel = "Diesel",
00179 fuelHybrid = "Hybryda",
00180 fuelElectric = "Elektryczny",
00181 transmission = "Skrzynia biegów",
00182 transmissionAutomatic = "Automatyczna",
00183 transmissionManual = "Manualna",
00184 bodyType = "Typ nadwozia",
00185 bodySedan = "Sedan",
00186 bodySuv = "SUV",
00187 bodyCoupe = "Coupe",
00188 bodyHatchback = "Hatchback",
00189 bodyWagon = "Kombi",
00190 driveType = "Napęd",
00191 driveFwd = "Przedni (FWD)",
00192 driveRwd = "Tylny (RWD)",
00193 driveAwd = "Na wszystkie koła (AWD)",
00194 drive4x4 = "4x4",
00195 condition = "Stan",
00196 conditionNew = "Nowy",
00197 conditionUsed = "Używany",
00198 location = "Lokalizacja",
00199 resetFilters = "Wyczyść",
00200 resultsCount = "Wyniki: %d",
00201 noSearchResults = "Brak ofert spełniających kryteria.",
00202 myListings = "Moje ogłoszenia",
00203 history = "Historia",
00204 settings = "Ustawienia",
00205 logout = "Wyloguj się",
00206 theme = "Motyw",
00207 themeSystem = "Systemowy",
00208 themeLight = "Jasny",
00209 themeDark = "Ciemny",
00210 language = "Język",
00211 notifications = "Powiadomienia",
00212 appPreferences = "Preferencje aplikacji",
00213 noHistory = "Brak historii",
00214 memberSince = "Członek od %s",
00215 chooseLanguage = "Wybierz język",
00216 chooseTheme = "Wybierz motyw",
00217 listingsStatus = "Aktywne",
00218 historyEmpty = "Brak historii",
00219 brandLogosPlaceholder = "kiedyś zamienimy je",
00220 noListingsYet = "Nie dodałeś jeszcze żadnych ogłoszeń.",
00221 verifiedListing = "Zweryfikowane ogłoszenie",
00222 seller = "Sprzedający",
00223 back = "Wróć",
00224 yearProduction = "Rok produkcji",
00225 mileage = "Przebieg",
00226 fuel = "Paliwo",
00227 gearbox = "Skrzynia",
00228 engineCapacityLabel = "Pojemność",
00229 power = "Moc",
00230 unitCurrency = "zł",
00231 unitKm = "km",
00232 unitCm3 = "cm3",
00233 unitPower = "KM",
00234 addListingTitle = "Dodaj ogłoszenie",
00235 brandAndModel = "Marka i model",
00236 price = "Cena",
00237 year = "Rocznik",
00238 city = "Miasto",
00239 addPhotos = "Dodaj zdjęcia",
00240 photoSourceTitle = "Wybierz źródło zdjęcia",
00241 photoSourceMessage = "Skąd chcesz dodać zdjęcia do swojego ogłoszenia?",
00242 photoSourceGallery = "Galeria",
00243 photoSourceCamera = "Aparat",
00244 selectedPhotoContentDescription = "Wybrane zdjęcie",
00245 removePhotoContentDescription = "Usuń zdjęcie",
00246 addListingSuccess = "Ogłoszenie dodane!",
00247 addListingError = "Błąd: %s",
00248 addListingRequiredFields = "Podaj przynajmniej markę i cenę!",
00249 uploading = "Wysyłanie...",
00250 noEmail = "brak emaila",
00251 notificationsSettingsDescription = "Powiadomienia z ogłoszeniami, które mogą cię zainteresować",
00252 likeNotificationTitle = "Polubienie!",
00253 likeNotificationMessage = "Twoje auto %s zostało właśnie polubione!",
00254 dealTestNotification = "Wyślij testowe powiadomienie",
00255 dealNotification = "Powiadomienie o okazjach",
00256 dealNotificationDescription = "Ciagle przegapiasz tanie Passaty? Włącz powiadomienia, aby
otrzymywać info o nowych autach i promocjach!",
```

```
00257     enable = "Włącz",
00258     maybeLater = "Może później",
00259     newOffers = "Nowe oferty",
00260     postalCode = "Kod pocztowy",
00261     postalCodeInvalid = "Niepoprawny kod pocztowy (format: XX-XXX)",
00262     mapViewToggle = "Mapa",
00263     listViewToggle = "Lista",
00264     searchRadius = "Promień",
00265     radiusAny = "Dowolny",
00266     addListingGeocodingWarning = "Dodano bez lokalizacji na mapie",
00267     phoneNumber = "Numer telefonu",
00268     callSeller = "Zadzwoń",
00269     messageSeller = "Wyślij SMS",
00270     shakePhone = "Potrzęsij telefonem, aby odkryć kontakt",
00271     noPhoneNumber = "Brak numeru telefonu w ogłoszeniu"
00272 )
00273
00274 val EnglishStrings = AppStrings(
00275     home = "Home",
00276     search = "Search",
00277     add = "Add",
00278     favorites = "Favorites",
00279     profile = "Profile",
00280     searchPlaceholder = "Search cars, brands, or models",
00281     buy = "Buy",
00282     sell = "Sell",
00283     findCar = "Find a Car",
00284     listYourCar = "List Your Car",
00285     popularBrands = "Popular Brands",
00286     seeAll = "See All",
00287     freshArrivals = "Fresh Arrivals",
00288     featured = "FEATURED",
00289     filters = "Filters",
00290     sortBy = "Sort by",
00291     sortNameAsc = "Name A-Z",
00292     sortNameDesc = "Name Z-A",
00293     sortPriceAsc = "Price low to high",
00294     sortPriceDesc = "Price high to low",
00295     sortYearDesc = "Newest year",
00296     sortMileageAsc = "Mileage low to high",
00297     brand = "Brand",
00298     model = "Model",
00299     minPrice = "Min price",
00300     maxPrice = "Max price",
00301     minYear = "Min year",
00302     maxYear = "Max year",
00303     maxMileage = "Max mileage (km)",
00304     fuelType = "Fuel type",
00305     fuelPetrol = "Petrol",
00306     fuelDiesel = "Diesel",
00307     fuelHybrid = "Hybrid",
00308     fuelElectric = "Electric",
00309     transmission = "Transmission",
00310     transmissionAutomatic = "Automatic",
00311     transmissionManual = "Manual",
00312     bodyType = "Body type",
00313     bodySedan = "Sedan",
00314     bodySuv = "SUV",
00315     bodyCoupe = "Coupe",
00316     bodyHatchback = "Hatchback",
00317     bodyWagon = "Wagon",
00318     driveType = "Drive type",
00319     driveFwd = "Front-wheel drive (FWD)",
00320     driveRwd = "Rear-wheel drive (RWD)",
00321     driveAwd = "All-wheel drive (AWD)",
00322     drive4x4 = "4x4",
00323     condition = "Condition",
00324     conditionNew = "New",
00325     conditionUsed = "Used",
00326     location = "Location",
00327     resetFilters = "Reset",
00328     resultsCount = "Results: %d",
00329     noSearchResults = "No offers match your criteria.",
00330     myListings = "My listings",
00331     history = "History",
00332     settings = "Settings",
00333     logout = "Logout",
00334     theme = "Theme",
00335     themeSystem = "System",
00336     themeLight = "Light",
00337     themeDark = "Dark",
00338     language = "Language",
00339     notifications = "Notifications",
00340     appPreferences = "App preferences",
00341     noHistory = "No history",
00342     memberSince = "Member since %s",
00343     chooseLanguage = "Choose language",
```

```

00344     chooseTheme = "Choose theme",
00345     listingsStatus = "Active",
00346     historyEmpty = "No history found",
00347     brandLogosPlaceholder = "we will replace them later",
00348     noListingsYet = "You have not added any listings yet.",
00349     verifiedListing = "Verified listing",
00350     seller = "Seller",
00351     back = "Back",
00352     yearProduction = "Year of production",
00353     mileage = "Mileage",
00354     fuel = "Fuel",
00355     gearbox = "Gearbox",
00356     engineCapacityLabel = "Engine capacity",
00357     power = "Power",
00358     unitCurrency = "PLN",
00359     unitKm = "km",
00360     unitCm3 = "cm3",
00361     unitPower = "HP",
00362     addListingTitle = "Add listing",
00363     brandAndModel = "Brand and model",
00364     price = "Price",
00365     year = "Year",
00366     city = "City",
00367     addPhotos = "Add photos",
00368     photoSourceTitle = "Choose photo source",
00369     photoSourceMessage = "Where do you want to add photos for your listing from?",
00370     photoSourceGallery = "Gallery",
00371     photoSourceCamera = "Camera",
00372     selectedPhotoContentDescription = "Selected photo",
00373     removePhotoContentDescription = "Remove photo",
00374     addListingSuccess = "Listing added!",
00375     addListingError = "Error: %s",
00376     addListingRequiredFields = "Provide at least brand and price!",
00377     uploading = "Uploading...",
00378     noEmail = "no email",
00379     postalCode = "Postal code",
00380     postalCodeInvalid = "Invalid postal code (format: XX-XXX)",
00381     mapViewToggle = "Map",
00382     listViewToggle = "List",
00383     searchRadius = "Radius",
00384     radiusAny = "Any",
00385     addListingGeocodingWarning = "Added without map location",
00386     phoneNumber = "Phone number",
00387     callSeller = "Call",
00388     messageSeller = "Send SMS",
00389     notificationsSettingsDescription = "Notifications with listings that might interest you",
00390     likeNotificationTitle = "Like!",
00391     likeNotificationMessage = "Your car %s was just liked!",
00392     dealTestNotification = "Send test notification",
00393     dealNotification = "Deal notification",
00394     dealNotificationDescription = "Missing out on the best deals? Turn on the notifications!",
00395     enable = "Enable",
00396     maybeLater = "Maybe later",
00397     newOffers = "New offers",
00398     shakePhone = "Shake your phone to reveal contact information",
00399     noPhoneNumber = "There is no phone number provided in the listing"
00400 )

```

4.27 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/models/ListingCatalog.kt

Katalogi statyczne: marki, modele, rodzaje paliwa; przykładowe dane.

4.27.1 Opis szczegółowy

Katalogi statyczne: marki, modele, rodzaje paliwa; przykładowe dane.

Definicja w pliku [ListingCatalog.kt](#).

4.28 ListingCatalog.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file ListingCatalog.kt
00003  * @brief Katalogi statyczne: marki, modele, rodzaje paliwa; przykładowe dane.
00004  */
00005 package com.example.otomotuzplus.ui.models
00006
00007 /**
00008  * Strukturalny model widoku ogłoszenia pojazdu używany w katalogu danych przykładowych.
00009  *
00010  * W przeciwieństwie do [com.example.otomotuzplus.models.CarAd], właściwości numeryczne
00011  * przechowywane jako właściwe typy (Int), ponieważ klasa nigdy nie przechodzi przez
00012  * Firestore -- używana wyłącznie dla statycznych danych przykładowych w [sampleListings].
00013  *
00014  * @property name      Czytelny tytuł ogłoszenia.
00015  * @property brand      Nazwa marki producenta.
00016  * @property model      Oznaczenie modelu.
00017  * @property price      Cena wywoławcza w PLN.
00018  * @property year       Rok produkcji.
00019  * @property mileageKm  Przebieg w kilometrach.
00020  * @property fuelType   Klucz rodzaju paliwa (np. `petrol`, `diesel`, `electric`).
00021  * @property transmission Klucz rodzaju skrzyni biegów (`automatic` lub `manual`).
00022  * @property bodyType   Klucz typu nadwozia (np. `suv`, `sedan`, `coupe`).
00023  * @property driveType  Klucz napędu (`fwd`, `rwd`, `awd`).
00024  * @property carCondition Klucz stanu (`new` lub `used`).
00025  * @property location   Polska nazwa miasta.
00026  */
00027 data class CarListing(
00028     val name: String,
00029     val brand: String,
00030     val model: String,
00031     val price: Int,
00032     val year: Int,
00033     val mileageKm: Int,
00034     val fuelType: String,
00035     val transmission: String,
00036     val bodyType: String,
00037     val driveType: String,
00038     val carCondition: String,
00039     val location: String
00040 )
00041
00042 /**
00043  * Generuje złożony klucz ulubionych dla [CarListing].
00044  *
00045  * Format klucza `<name>|<year>` odzwierciedla format używany dla kluczy
00046  * ulubionych opartych na [CarAd] w [OtomotUZplusApp], umożliwiając
00047  * koegzystencję obu zbiorów na tej samej liście `favoriteCars`.
00048  */
00049 fun CarListing.favoriteKey(): String = "$name|$year"
00050
00051 /**
00052  * Zwraca zakodowaną na stałe listę sześciu przykładowych ogłoszeń pojazdu
00053  * do podglądów UI i testów deweloperskich. Nie wyświetlana użytkownikowi produkcyjnie.
00054  *
00055  * @return Lista obiektów [CarListing] obejmująca paliwo benzynowe, diesel, hybrydę
00056  * i elektryczne w różnych przedziałach cenowych i typach nadwozia.
00057  */
00058 fun sampleListings(): List<CarListing> = listOf(
00059     CarListing(
00060         name = "Audi RS6 Avant",
00061         brand = "Audi",
00062         model = "RS6",
00063         price = 690000,
00064         year = 2023,
00065         mileageKm = 8500,
00066         fuelType = "petrol",
00067         transmission = "automatic",
00068         bodyType = "wagon",
00069         driveType = "awd",
00070         carCondition = "used",
00071         location = "Warszawa"
00072     ),
00073     CarListing(
00074         name = "BMW M4 Competition xDrive",
00075         brand = "BMW",
00076         model = "M4",
00077         price = 429000,
00078         year = 2022,
00079         mileageKm = 15000,
00080         fuelType = "petrol",
00081         transmission = "automatic",
00082         bodyType = "coupe",
00083         driveType = "awd",
```

```

00084         carCondition = "used",
00085         location = "Krakow"
00086     ),
00087     CarListing(
00088         name = "Mercedes C 300",
00089         brand = "Mercedes",
00090         model = "C 300",
00091         price = 268000,
00092         year = 2021,
00093         mileageKm = 39000,
00094         fuelType = "hybrid",
00095         transmission = "automatic",
00096         bodyType = "sedan",
00097         driveType = "rwd",
00098         carCondition = "used",
00099         location = "Wroclaw"
00100     ),
00101     CarListing(
00102         name = "Porsche 911 Carrera S",
00103         brand = "Porsche",
00104         model = "911",
00105         price = 585000,
00106         year = 2020,
00107         mileageKm = 42000,
00108         fuelType = "petrol",
00109         transmission = "automatic",
00110         bodyType = "coupe",
00111         driveType = "rwd",
00112         carCondition = "used",
00113         location = "Poznan"
00114     ),
00115     CarListing(
00116         name = "Volkswagen Tiguan 2.0 TDI",
00117         brand = "VW",
00118         model = "Tiguan",
00119         price = 156000,
00120         year = 2022,
00121         mileageKm = 27000,
00122         fuelType = "diesel",
00123         transmission = "automatic",
00124         bodyType = "suv",
00125         driveType = "fwd",
00126         carCondition = "used",
00127         location = "Gdansk"
00128     ),
00129     CarListing(
00130         name = "Tesla Model Y Performance",
00131         brand = "Tesla",
00132         model = "Model Y",
00133         price = 312000,
00134         year = 2024,
00135         mileageKm = 5000,
00136         fuelType = "electric",
00137         transmission = "automatic",
00138         bodyType = "suv",
00139         driveType = "awd",
00140         carCondition = "new",
00141         location = "Lodz"
00142     )
00143 )
00144

```

4.29 Dokumentacja pliku `app/src/main/java/com/example/otomotuzplus/ui/models/ServerValueTranslations.kt`

Funkcje pomocnicze mapujące surowe wartości pól Firestore na zlokalizowane ciągi wyświetlania dla rodzaju paliwa i skrzyni biegów.

Pakiety

- package `strings`

Funkcje

- Object `localizeGearboxType` (Object `rawValue`, Object `strings`)
Konwertuje surową wartość skrzyni biegów z Firestore na zlokalizowany ciąg wyświetlania.
- Object `fuelTypeKey` (Object `rawValue`)
Normalizuje surowy ciąg rodzaju paliwa do niezależnego od języka klucza kanonicznego.
- Object `gearboxTypeKey` (Object `rawValue`)
Normalizuje surowy ciąg skrzyni biegów do niezależnego od języka klucza kanonicznego.

4.29.1 Opis szczegółowy

Funkcje pomocnicze mapujące surowe wartości pól Firestore na zlokalizowane ciągi wyświetlania dla rodzaju paliwa i skrzyni biegów.

Dokumenty Firestore przechowują wartości paliwa i skrzyni w języku użytym przez sprzedającego (polskim lub angielskim). Funkcje te normalizują surową wartość do klucza kanonicznego, a następnie rozwiązują ją przez aktywną instancję [AppStrings].

Definicja w pliku [ServerValueTranslations.kt](#).

4.29.2 Dokumentacja funkcji

`fuelTypeKey()`

```
Object fuelTypeKey (  
    Object rawValue )
```

Normalizuje surowy ciąg rodzaju paliwa do niezależnego od języka klucza kanonicznego.

Rozpoznawane klucze: "petrol", "diesel", "hybrid", "electric". Używane przez [localizeFuelType] i logikę dopasowania filtrów wyszukiwania.

Parametry

<code>rawValue</code>	Surowa wartość rodzaju paliwa (bez rozróżniania wielkości liter, po polsku lub angielsku).
-----------------------	--

Zwraca

Ciąg klucza kanonicznego lub `null` jeśli wartość nierozpoznana.

Definicja w linii 47 pliku [ServerValueTranslations.kt](#).

`gearboxTypeKey()`

```
Object gearboxTypeKey (  
    Object rawValue )
```

Normalizuje surowy ciąg skrzyni biegów do niezależnego od języka klucza kanonicznego.

Rozpoznawane klucze: "automatic", "manual". Obejmuje popularne skróty takie jak "DSG", "CVT", "↔ MT".

Parametry

<i>rawValue</i>	Surowa wartość skrzyni biegów (bez rozróżniania wielkości liter, po polsku lub angielsku).
-----------------	--

Zwraca

Ciąg klucza kanonicznego lub `null` jeśli wartość nierozpoznana.

Definicja w linii 59 pliku [ServerValueTranslations.kt](#).

localizeGearboxType()

```
Object localizeGearboxType (
    Object rawValue,
    Object strings )
```

Konwertuje surową wartość skrzyni biegów z Firestore na zlokalizowany ciąg wyświetlania.

Parametry

<i>rawValue</i>	Surowy ciąg skrzyni biegów (np. "Manualna", "automatic", "DSG").
<i>strings</i>	Aktywna instancja [AppStrings].

Zwraca

Zlokalizowana etykieta skrzyni biegów lub [rawValue] jeśli nierozpoznana.

Definicja w linii 35 pliku [ServerValueTranslations.kt](#).

4.30 ServerValueTranslations.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file ServerValueTranslations.kt
00003  * @brief Funkcje pomocnicze mapujące surowe wartości pól Firestore na
00004  *        zlokalizowane ciągi wyświetlania dla rodzaju paliwa i skrzyni biegów.
00005  *
00006  * Dokumenty Firestore przechowują wartości paliwa i skrzyni w języku użytym
00007  * przez sprzedającego (polskim lub angielskim). Funkcje te normalizują surową
00008  * wartość do klucza kanonicznego, a następnie rozwiązują ją przez aktywną
00009  * instancję [AppStrings].
00010  */
00011 package com.example.otomotuzplus.ui.models
00012
00013 import java.util.Locale
00014
00015 /**
00016  * Konwertuje surową wartość rodzaju paliwa z Firestore na zlokalizowany ciąg wyświetlania.
00017  *
00018  * Konwersja nie rozróżnia wielkości liter i toleruje pisownię polską i angielską.
00019  * Nieznane wartości zwracane bez zmian.
00020  *
00021  * @param rawValue Surowy ciąg rodzaju paliwa z Firestore (np. `"Benzyna"` lub `"petrol"`).
00022  * @param strings Aktywna instancja [AppStrings] zawierająca zlokalizowaną etykietę.
00023  * @return Zlokalizowana etykieta rodzaju paliwa lub [rawValue] jeśli klucz nie został rozpoznany.
00024  */
00025 fun localizeFuelType(rawValue: String, strings: AppStrings): String {
00026     val value = rawValue.trim()
00027     return when (fuelTypeKey(value)) {
```

```

00028         "petrol" -> strings.fuelPetrol
00029         "diesel" -> strings.fuelDiesel
00030         "hybrid" -> strings.fuelHybrid
00031         "electric" -> strings.fuelElectric
00032         else -> value
00033     }
00034 }
00035
00036 /**
00037  * Konwertuje surową wartość skrzyni biegów z Firestore na zlokalizowany ciąg wyświetlania.
00038  *
00039  * @param rawValue Surowy ciąg skrzyni biegów (np. "Manualna", "automatic", "DSG").
00040  * @param strings Aktywna instancja [AppStrings].
00041  * @return Zlokalizowana etykieta skrzyni biegów lub [rawValue] jeśli nierozpoznana.
00042  */
00043 fun localizeGearboxType(rawValue: String, strings: AppStrings): String {
00044     val value = rawValue.trim()
00045     return when (gearboxTypeKey(value)) {
00046         "automatic" -> strings.transmissionAutomatic
00047         "manual" -> strings.transmissionManual
00048         else -> value
00049     }
00050 }
00051
00052 /**
00053  * Normalizuje surowy ciąg rodzaju paliwa do niezależnego od języka klucza kanonicznego.
00054  *
00055  * Rozpoznawane klucze: "petrol", "diesel", "hybrid", "electric".
00056  * Używane przez [localizeFuelType] i logikę dopasowania filtrów wyszukiwania.
00057  *
00058  * @param rawValue Surowa wartość rodzaju paliwa (bez rozróżniania wielkości liter, po polsku lub
00059  * angielsku).
00060  * @return Ciąg klucza kanonicznego lub "null" jeśli wartość nierozpoznana.
00061  */
00062 fun fuelTypeKey(rawValue: String): String? {
00063     return when (rawValue.trim().lowercase(Locale.ROOT)) {
00064         "benzyna", "petrol", "gasoline", "benzin" -> "petrol"
00065         "diesel", "olej napedowy" -> "diesel"
00066         "hybryda", "hybrid", "phev", "hev" -> "hybrid"
00067         "elektryczny", "electric", "ev" -> "electric"
00068         else -> null
00069     }
00070 }
00071 /**
00072  * Normalizuje surowy ciąg skrzyni biegów do niezależnego od języka klucza kanonicznego.
00073  *
00074  * Rozpoznawane klucze: "automatic", "manual".
00075  * Obejmuje popularne skróty takie jak "DSG", "CVT", "MT".
00076  *
00077  * @param rawValue Surowa wartość skrzyni biegów (bez rozróżniania wielkości liter, po polsku lub
00078  * angielsku).
00079  * @return Ciąg klucza kanonicznego lub "null" jeśli wartość nierozpoznana.
00080  */
00081 fun gearboxTypeKey(rawValue: String): String? {
00082     return when (rawValue.trim().lowercase(Locale.ROOT)) {
00083         "automatyczna", "automatic", "auto", "at", "cvt", "dct", "dsg" -> "automatic"
00084         "manualna", "manual", "man", "mt" -> "manual"
00085         else -> null
00086     }
00087 }

```

4.31 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/ui/navigation/NavDestinations.kt

Enum nawigacyjny AppDestinations i model elementu paska nawigacji.

4.31.1 Opis szczegółowy

Enum nawigacyjny AppDestinations i model elementu paska nawigacji.

Definicja w pliku [NavDestinations.kt](#).

4.32 NavDestinations.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file NavDestinations.kt
00003  * @brief Enum nawigacyjny AppDestinations i model elementu paska nawigacji.
00004  */
00005 package com.example.otomotuzplus.ui.navigation
00006
00007 import androidx.compose.ui.graphics.vector.ImageVector
00008
00009 /**
00010  * Główne miejsca docelowe nawigacji wyświetlane na dolnym pasku nawigacyjnym.
00011  *
00012  * `MainActivity` używa tego enuma jako głównego stanu nawigacji; steruje
00013  * zarówno wyświetlanym ekranem jak i zaznaczonym elementem paska nav.
00014  * Ekran nakładkowe ([com.example.otomotuzplus.ui.screens.details.AdDetailScreen]
00015  * i [com.example.otomotuzplus.ui.screens.settings.SettingsScreen]) nie są
00016  * tu reprezentowane -- kontrolowane osobnym stanem boolowskim lub nullable.
00017  */
00018 enum class AppDestinations { HOME, SEARCH, ADD, FAVORITES, PROFILE }
00019
00020 /**
00021  * Deskryptor pojedynczego elementu w
00022  * [androidx.compose.material3.adaptive.navigation.NavigationSuiteScaffold].
00023  *
00024  * @property destination [AppDestinations] do którego nawiguje ten element.
00025  * @property label Zlokalizowana etykieta wyświetlana pod ikoną.
00026  * @property selectedIcon Ikona renderowana gdy miejsce docelowe jest aktywne.
00027  * @property unselectedIcon Ikona renderowana gdy miejsce docelowe jest nieaktywne.
00028  */
00029 data class NavigationItem(
00030     val destination: AppDestinations,
00031     val label: String,
00032     val selectedIcon: ImageVector,
00033     val unselectedIcon: ImageVector
00034 )
```

4.33 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/screens/add/AddScreen.kt

Ekran tworzenia nowego ogłoszenia pojazdu z geokodowaniem i przesyłaniem zdjęć.

Pakiety

- package [context](#)

Funkcje

- Object [context.originalToTransformed](#) (Object offset)
- Object [context.transformedToOriginal](#) (Object offset)
- Object [context.customTextFieldColors](#) ()
- Object [context.createTempImageUri](#) (Object context)

Tworzy plik tymczasowy i zwraca URI [FileProvider] odpowiedni do przekazania do `ActivityResultContracts.TakePicture`.

Zmienne

- Object [context.repository](#)
- Object [context.coroutineScope](#)
- Object [context.title](#)
- Object [context.priceText](#)
- Object [context.locationText](#)
- Object [context.postalCode](#)
- Object [context.year](#)
- Object [context.mileageText](#)
- Object [context.fuelText](#)
- Object [context.gearboxText](#)
- Object [context.engineCapacity](#)
- Object [context.powerText](#)
- Object [context.phoneNumber](#)
- Object [context.isUploading](#)
- Object [context.selectedImageUri](#)
- Object [context.showPhotoSourceDialog](#)
- Object [context.templImageUri](#)
- Object [context.cameraLauncher](#)
- Object [context.multiplePhotoPickerLauncher](#)
- Object [context.s](#)
- Object [context.formatted](#)
- Object [context.isPetrol](#)
- Object [context.isDiesel](#)
- Object [context.isManual](#)
- Object [context.isAuto](#)
- Object [context.uri](#)
- Object [context.validPostal](#)
- Object [context.coords](#)
- Object [context.currentUser](#)
- Object [context.sellerId](#)
- Object [context.sellerEmail](#)
- Object [context.newCar](#)
- Object [context.msg](#)

4.33.1 Opis szczegółowy

Ekran tworzenia nowego ogłoszenia pojazdu z geokodowaniem i przesyłaniem zdjęć.

Definicja w pliku [AddScreen.kt](#).

4.34 AddScreen.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file AddScreen.kt
00003  * @brief Ekran tworzenia nowego ogłoszenia pojazdu z geokodowaniem i przesyłaniem zdjęć.
00004  */
00005 package com.example.otomotuzplus.ui.screens.add
00006
00007 import android.widget.Toast
00008 import androidx.compose.foundation.background
00009 import androidx.compose.foundation.layout.*
00010 import androidx.compose.foundation.rememberScrollState
00011 import androidx.compose.foundation.shape.RoundedCornerShape
```

```

00012 import androidx.compose.foundation.text.KeyboardOptions
00013 import androidx.compose.foundation.verticalScroll
00014 import androidx.compose.material3.*
00015 import androidx.compose.runtime.*
00016 import androidx.compose.ui.Modifier
00017 import androidx.compose.ui.graphics.Color
00018 import androidx.compose.ui.platform.LocalContext
00019 import androidx.compose.ui.text.font.FontWeight
00020 import androidx.compose.ui.text.AnnotatedString
00021 import androidx.compose.ui.text.input.KeyboardType
00022 import androidx.compose.ui.text.input.OffsetMapping
00023 import androidx.compose.ui.text.input.TransformedText
00024 import androidx.compose.ui.text.input.VisualTransformation
00025 import androidx.compose.ui.unit.dp
00026 import androidx.compose.ui.unit.sp
00027 import com.example.otomotuzplus.data.FirebaseRepository
00028 import com.example.otomotuzplus.data.geocodePostalCode
00029 import com.example.otomotuzplus.models.CarAd
00030 import com.example.otomotuzplus.ui.models.AppStrings
00031 import kotlinx.coroutines.launch
00032 import androidx.compose.runtime.rememberCoroutineScope
00033 import com.example.otomotuzplus.ui.theme.BrandGold
00034 import java.io.File
00035 import androidx.core.content.FileProvider
00036 import android.net.Uri
00037 import androidx.activity.compose.rememberLauncherForActivityResult
00038 import androidx.activity.result.contract.ActivityResultContracts
00039 import androidx.compose.foundation.layout.PaddingValues
00040 import androidx.compose.foundation.lazy.LazyRow
00041 import androidx.compose.material.icons.Icons
00042 import androidx.compose.material.icons.filled.Close
00043 import androidx.compose.material.icons.filled.PhotoCamera
00044 import androidx.compose.ui.draw.clip
00045 import coil.compose.AsyncImage
00046
00047 /**
00048  * Ekran tworzenia nowego ogłoszenia pojazdu.
00049  *
00050  * ## Pola formularza
00051  * Marka i model, cena, rok, przebieg, miasto, kod pocztowy, rodzaj paliwa (grupa przycisków),
00052  * skrzynia biegów (grupa przycisków), pojemność silnika, moc, numer telefonu.
00053  *
00054  * ## Dodawanie zdjęć
00055  * Zdjęcia są wybierane z galerii systemowej (do 5) lub wykonywane aparatem
00056  * urządzenia przez plik tymczasowy [FileProvider]. Miniatury są wyświetlane
00057  * w poziomym wierszu z przyciskiem usunięcia na każdej.
00058  *
00059  * ## Przebieg wysyłania
00060  * 1. Waliduje wymagane pola (tytuł i cena) oraz format kodu pocztowego.
00061  * 2. Geokoduje kod pocztowy przez [geocodePostalCode] (korutyna na dyspozytorze IO).
00062  * 3. Przesyła wybrane zdjęcia równolegle przez [FirebaseRepository.uploadImages].
00063  * 4. Zapisuje dokument [CarAd] przez [FirebaseRepository.addCar].
00064  * 5. Wyświetla Toast i wywołuje [onNavigateBack] po sukcesie.
00065  *
00066  * @param strings Aktywne [AppStrings] dla zlokalizowanych etykiet.
00067  * @param onNavigateBack Callback wywołany po pomyślnym przesłaniu lub anulowaniu.
00068  */
00069 @OptIn(ExperimentalMaterial3Api::class)
00070 @Composable
00071 fun AddScreen(
00072     strings: AppStrings,
00073     onNavigateBack: () -> Unit
00074 ) {
00075     val context = LocalContext.current
00076     val repository = remember { FirebaseRepository() }
00077     val coroutineScope = rememberCoroutineScope()
00078     var title by remember { mutableStateOf("") }
00079     var priceText by remember { mutableStateOf("") }
00080     var locationText by remember { mutableStateOf("") }
00081     var postalCode by remember { mutableStateOf("") }
00082     var year by remember { mutableStateOf("") }
00083     var mileageText by remember { mutableStateOf("") }
00084     var fuelText by remember(strings) { mutableStateOf(strings.fuelPetrol) }
00085     var gearboxText by remember(strings) { mutableStateOf(strings.transmissionManual) }
00086     var engineCapacity by remember { mutableStateOf("") }
00087     var powerText by remember { mutableStateOf("") }
00088     var phoneNumber by remember { mutableStateOf("") }
00089     var isUploading by remember { mutableStateOf(false) }
00090     var selectedImageUris by remember { mutableStateOf<List<Uri>>(emptyList()) }
00091     var showPhotoSourceDialog by remember { mutableStateOf(false) }
00092     var tempImageUri by remember { mutableStateOf<Uri?>(null) }
00093     val cameraLauncher = rememberLauncherForActivityResult(
00094         contract = ActivityResultContracts.TakePicture()
00095     ) { success ->
00096         if (success && tempImageUri != null) {
00097             selectedImageUris = selectedImageUris + tempImageUri!!
00098         }
00099     }

```

```

00099     }
00100     val multiplePhotoPickerLauncher = androidx.activity.compose.rememberLauncherForActivityResult (
00101         contract =
00102             androidx.activity.result.contract.ActivityResultContracts.PickMultipleVisualMedia(maxItems = 5)
00103     ) { uris ->
00104         if (uris.isNotEmpty()) {
00105             selectedImageUris = selectedImageUris + uris
00106         }
00107     }
00108     Column(
00109         modifier = Modifier
00110             .fillMaxSize()
00111             .background(MaterialTheme.colorScheme.background)
00112             .padding(16.dp)
00113             .verticalScroll(rememberScrollState()),
00114         verticalArrangement = Arrangement.spacedBy(14.dp)
00115     ) {
00116         Text(
00117             text = strings.addListingTitle,
00118             color = MaterialTheme.colorScheme.onBackground,
00119             fontSize = 24.sp,
00120             fontWeight = FontWeight.Bold,
00121             modifier = Modifier.padding(bottom = 8.dp, top = 24.dp)
00122         )
00123         OutlinedTextField(
00124             value = title,
00125             onChange = { title = it },
00126             label = { Text(strings.brandAndModel) },
00127             modifier = Modifier.fillMaxWidth(),
00128             singleLine = true,
00129             colors = customTextFieldColors()
00130         )
00131     }
00132     Row(
00133         modifier = Modifier.fillMaxWidth(),
00134         horizontalArrangement = Arrangement.spacedBy(12.dp)
00135     ) {
00136         OutlinedTextField(
00137             value = priceText,
00138             onChange = { priceText = it.filter { char -> char.isDigit() } },
00139             label = { Text(strings.price) },
00140             suffix = { Text(strings.unitCurrency) },
00141             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
00142             modifier = Modifier.weight(1f),
00143             singleLine = true,
00144             colors = customTextFieldColors()
00145         )
00146         OutlinedTextField(
00147             value = year,
00148             onChange = { year = it.filter { char -> char.isDigit() } },
00149             label = { Text(strings.year) },
00150             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
00151             modifier = Modifier.weight(1f),
00152             singleLine = true,
00153             colors = customTextFieldColors()
00154         )
00155     }
00156     Row(
00157         modifier = Modifier.fillMaxWidth(),
00158         horizontalArrangement = Arrangement.spacedBy(12.dp)
00159     ) {
00160         OutlinedTextField(
00161             value = mileageText,
00162             onChange = { mileageText = it.filter { char -> char.isDigit() } },
00163             label = { Text(strings.mileage) },
00164             suffix = { Text(strings.unitKm) },
00165             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
00166             modifier = Modifier.weight(1f),
00167             singleLine = true,
00168             colors = customTextFieldColors()
00169         )
00170         OutlinedTextField(
00171             value = locationText,
00172             onChange = { locationText = it },
00173             label = { Text(strings.city) },
00174             modifier = Modifier.weight(1f),
00175             singleLine = true,
00176             colors = customTextFieldColors()
00177         )
00178     }
00179     OutlinedTextField(
00180         value = postalCode,
00181         onChange = { postalCode = it.filter { c -> c.isDigit() }.take(5) },
00182     )

```

```

00185         label = { Text(strings.postalCode) },
00186         placeholder = { Text("00-000") },
00187         modifier = Modifier.fillMaxWidth(),
00188         singleLine = true,
00189         keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
00190         visualTransformation = remember {
00191             VisualTransformation { raw ->
00192                 val s = raw.text
00193                 val formatted = if (s.length <= 2) s else "${s.take(2)}-${s.drop(2)}"
00194                 TransformedText(
00195                     text = AnnotatedString(formatted),
00196                     offsetMapping = object : OffsetMapping {
00197                         override fun originalToTransformed(offset: Int) =
00198                             if (offset <= 2) offset else offset + 1
00199                         override fun transformedToOriginal(offset: Int) =
00200                             if (offset <= 2) offset else (offset - 1).coerceAtMost(s.length)
00201                     }
00202                 )
00203             }
00204         },
00205         colors = customTextFieldColors()
00206     )
00207
00208     Column(modifier = Modifier.fillMaxWidth()) {
00209         Text(
00210             text = strings.fuelType,
00211             color = MaterialTheme.colorScheme.onBackground.copy(alpha = 0.7f),
00212             fontSize = 14.sp,
00213             modifier = Modifier.padding(start = 4.dp, bottom = 4.dp)
00214         )
00215         Row(
00216             modifier = Modifier.fillMaxWidth(),
00217             horizontalArrangement = Arrangement.spacedBy(12.dp)
00218         ) {
00219             val isPetrol = fuelText == strings.fuelPetrol
00220             Button(
00221                 onClick = { fuelText = strings.fuelPetrol },
00222                 modifier = Modifier.weight(1f).height(50.dp),
00223                 colors = ButtonDefaults.buttonColors(
00224                     containerColor = if (isPetrol) BrandGold else
00225                         MaterialTheme.colorScheme.surface,
00226                     contentColor = if (isPetrol) Color.Black else
00227                         MaterialTheme.colorScheme.onSurface
00228                 ),
00229                 shape = RoundedCornerShape(12.dp)
00230             ) {
00231                 Text(strings.fuelPetrol, fontWeight = if (isPetrol) FontWeight.Bold else
00232                     FontWeight.Normal)
00233             }
00234             val isDiesel = fuelText == strings.fuelDiesel
00235             Button(
00236                 onClick = { fuelText = strings.fuelDiesel },
00237                 modifier = Modifier.weight(1f).height(50.dp),
00238                 colors = ButtonDefaults.buttonColors(
00239                     containerColor = if (isDiesel) BrandGold else
00240                         MaterialTheme.colorScheme.surface,
00241                     contentColor = if (isDiesel) Color.Black else
00242                         MaterialTheme.colorScheme.onSurface
00243                 ),
00244                 shape = RoundedCornerShape(12.dp)
00245             ) {
00246                 Text(strings.fuelDiesel, fontWeight = if (isDiesel) FontWeight.Bold else
00247                     FontWeight.Normal)
00248             }
00249         }
00250
00251         Column(modifier = Modifier.fillMaxWidth()) {
00252             Text(
00253                 text = strings.transmission,
00254                 color = MaterialTheme.colorScheme.onBackground.copy(alpha = 0.7f),
00255                 fontSize = 14.sp,
00256                 modifier = Modifier.padding(start = 4.dp, bottom = 4.dp)
00257             )
00258             Row(
00259                 modifier = Modifier.fillMaxWidth(),
00260                 horizontalArrangement = Arrangement.spacedBy(12.dp)
00261             ) {
00262                 val isManual = gearboxText == strings.transmissionManual
00263                 Button(
00264                     onClick = { gearboxText = strings.transmissionManual },
00265                     modifier = Modifier.weight(1f).height(50.dp),
00266                     colors = ButtonDefaults.buttonColors(
00267                         containerColor = if (isManual) BrandGold else
00268                             MaterialTheme.colorScheme.surface,
00269                         contentColor = if (isManual) Color.Black else

```



```

MaterialTheme.colorScheme.onSurface
00265         ),
00266         shape = RoundedCornerShape(12.dp)
00267     ) {
00268         Text(strings.transmissionManual, fontWeight = if (isManual) FontWeight.Bold else
FontWeight.Normal)
00269     }
00270
00271     val isAuto = gearboxText == strings.transmissionAutomatic
00272     Button(
00273         onClick = { gearboxText = strings.transmissionAutomatic },
00274         modifier = Modifier.weight(1f).height(50.dp),
00275         colors = ButtonDefaults.buttonColors(
00276             containerColor = if (isAuto) BrandGold else MaterialTheme.colorScheme.surface,
00277             contentColor = if (isAuto) Color.Black else
MaterialTheme.colorScheme.onSurface
00278         ),
00279         shape = RoundedCornerShape(12.dp)
00280     ) {
00281         Text(strings.transmissionAutomatic, fontWeight = if (isAuto) FontWeight.Bold else
FontWeight.Normal)
00282     }
00283     }
00284 }
00285
00286 Row(
00287     modifier = Modifier.fillMaxWidth(),
00288     horizontalArrangement = Arrangement.spacedBy(12.dp)
00289 ) {
00290     OutlinedTextField(
00291         value = engineCapacity,
00292         onValueChange = { engineCapacity = it.filter { char -> char.isDigit() } },
00293         label = { Text(strings.engineCapacityLabel) },
00294         suffix = { Text(strings.unitCm3) },
00295         keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
00296         modifier = Modifier.weight(1f),
00297         singleLine = true,
00298         colors = customTextFieldColors()
00299     )
00300     OutlinedTextField(
00301         value = powerText,
00302         onValueChange = { powerText = it.filter { char -> char.isDigit() } },
00303         label = { Text(strings.power) },
00304         suffix = { Text(strings.unitPower) },
00305         keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
00306         modifier = Modifier.weight(1f),
00307         singleLine = true,
00308         colors = customTextFieldColors()
00309     )
00310 }
00311
00312 OutlinedTextField(
00313     value = phoneNumber,
00314     onValueChange = { phoneNumber = it.filter { c -> c.isDigit() || c == '+' || c == ' ' || c
== '-' } },
00315     label = { Text(strings.phoneNumber) },
00316     keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone),
00317     modifier = Modifier.fillMaxWidth(),
00318     singleLine = true,
00319     colors = customTextFieldColors()
00320 )
00321
00322 Spacer(modifier = Modifier.height(16.dp))
00323
00324 Button(
00325     onClick = { showPhotoSourceDialog = true },
00326     modifier = Modifier.fillMaxWidth().height(50.dp),
00327     colors = ButtonDefaults.buttonColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant),
00328     shape = RoundedCornerShape(12.dp)
00329 ) {
00330     Icon(
00331         Icons.Default.PhotoCamera,
00332         contentDescription = null,
00333         tint = MaterialTheme.colorScheme.onSurface
00334     )
00335     Spacer(modifier = Modifier.width(8.dp))
00336     Text(strings.addPhotos, color = MaterialTheme.colorScheme.onSurface)
00337 }
00338
00339 if (selectedImageUris.isNotEmpty()) {
00340     Spacer(modifier = Modifier.height(12.dp))
00341     LazyRow(
00342         horizontalArrangement = Arrangement.spacedBy(8.dp),
00343         contentPadding = PaddingValues(horizontal = 4.dp)
00344     ) {
00345         items(selectedImageUris.size) { index ->

```

```

00346         val uri = selectedImageUris[index]
00347     Box {
00348         AsyncImage(
00349             model = uri,
00350             contentDescription = strings.selectedPhotoContentDescription,
00351             modifier = Modifier
00352                 .size(80.dp)
00353                 .clip(RoundedCornerShape(8.dp)),
00354             contentScale = androidx.compose.ui.layout.ContentScale.Crop
00355         )
00356         IconButton(
00357             onClick = { selectedImageUris = selectedImageUris - uri },
00358             modifier = Modifier
00359                 .align(androidx.compose.ui.Alignment.TopEnd)
00360                 .size(24.dp)
00361                 .background(Color.Black.copy(alpha = 0.5f),
00362                     androidx.compose.foundation.shape.CircleShape)
00363             ) {
00364             Icon(
00365                 Icons.Default.Close,
00366                 contentDescription = strings.removePhotoContentDescription,
00367                 tint = Color.White,
00368                 modifier = Modifier.size(16.dp)
00369             )
00370         }
00371     }
00372 }
00373
00374 Spacer(modifier = Modifier.height(24.dp))
00375
00376 Button(
00377     onClick = {
00378         val validPostal = postalCode.length == 5
00379         when {
00380             title.isBlank() || priceText.isBlank() ->
00381                 Toast.makeText(context, strings.addListingRequiredFields,
00382                     Toast.LENGTH_SHORT).show()
00383             !validPostal ->
00384                 Toast.makeText(context, strings.postalCodeInvalid, Toast.LENGTH_SHORT).show()
00385             else -> {
00386                 isUploading = true
00387                 coroutineScope.launch {
00388                     val coords =
00389                         geocodePostalCode("${postalCode.take(2)}-${postalCode.drop(2)}")
00390                     repository.uploadImages(selectedImageUris) { uploadedUrls ->
00391                         val currentUser =
00392                             com.google.firebase.auth.FirebaseAuth.getInstance().currentUser
00393                         val sellerId = currentUser?.uid ?: "unknown"
00394                         val sellerEmail = currentUser?.email ?: "Brak emaila"
00395                         val newCar = CarAd(
00396                             title = title,
00397                             priceText = priceText,
00398                             locationText = locationText,
00399                             postalCode = "${postalCode.take(2)}-${postalCode.drop(2)}",
00400                             latitude = coords?.first ?: 0.0,
00401                             longitude = coords?.second ?: 0.0,
00402                             year = year,
00403                             mileageText = mileageText,
00404                             fuelText = fuelText,
00405                             gearboxText = gearboxText,
00406                             engineCapacity = engineCapacity,
00407                             powerText = powerText,
00408                             imageUrls = uploadedUrls,
00409                             sellerId = sellerId,
00410                             sellerEmail = sellerEmail,
00411                             phoneNumber = phoneNumber.trim()
00412                         )
00413                         repository.addCar(
00414                             car = newCar,
00415                             onSuccess = {
00416                                 isUploading = false
00417                                 val msg = if (coords == null)
00418                                     "${strings.addListingSuccess}"
00419                                 else strings.addListingSuccess
00420                                 Toast.makeText(context, msg, Toast.LENGTH_SHORT).show()
00421                                 onNavigateBack()
00422                             },
00423                             onFailure = { e ->
00424                                 isUploading = false
00425                                 Toast.makeText(context,
00426                                     strings.addListingError.format(e.message ?: ""), Toast.LENGTH_LONG).show()
00427                             }
00428                         )
00429                     }
00430                 }
00431             }
00432         }
00433     }

```

```

00427         }
00428     }
00429 },
00430     modifier = Modifier
00431         .fillMaxWidth()
00432         .height(56.dp),
00433     shape = RoundedCornerShape(16.dp),
00434     colors = ButtonDefaults.buttonColors(containerColor = BrandGold),
00435     enabled = !isUploading
00436 ) {
00437     Text(
00438         text = if (isUploading) strings.uploading else strings.addListingTitle,
00439         color = Color.Black,
00440         fontSize = 18.sp,
00441         fontWeight = FontWeight.Bold
00442     )
00443 }
00444 Spacer(modifier = Modifier.height(32.dp))
00445 }
00446
00447 if (showPhotoSourceDialog) {
00448     AlertDialog(
00449         onDismissRequest = { showPhotoSourceDialog = false },
00450         title = { Text(strings.photoSourceTitle) },
00451         text = { Text(strings.photoSourceMessage) },
00452         confirmButton = {
00453             TextButton(onClick = {
00454                 showPhotoSourceDialog = false
00455                 multiplePhotoPickerLauncher.launch(
00456                     androidx.activity.result.PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia.ImageOnly)
00457                 )
00458             }) {
00459                 Text(strings.photoSourceGallery)
00460             }
00461         },
00462         dismissButton = {
00463             TextButton(onClick = {
00464                 showPhotoSourceDialog = false
00465                 val uri = createTempImageUri(context)
00466                 tempImageUri = uri
00467                 cameraLauncher.launch(uri)
00468             }) {
00469                 Text(strings.photoSourceCamera)
00470             }
00471         }
00472     )
00473 }
00474 }
00475
00476 @OptIn(ExperimentalMaterial3Api::class)
00477 @Composable
00478 private fun customTextFieldColors() = OutlinedTextFieldDefaults.colors(
00479     focusedBorderColor = BrandGold,
00480     unfocusedBorderColor = MaterialTheme.colorScheme.outline,
00481     focusedContainerColor = MaterialTheme.colorScheme.surface,
00482     unfocusedContainerColor = MaterialTheme.colorScheme.surface,
00483     focusedTextColor = MaterialTheme.colorScheme.onSurface,
00484     unfocusedTextColor = MaterialTheme.colorScheme.onSurface,
00485     cursorColor = BrandGold
00486 )
00487 /**
00488  * Tworzy plik tymczasowy i zwraca URI [FileProvider] odpowiedni do
00489  * przekazania do `ActivityResultContracts.TakePicture`.
00490  *
00491  * Plik jest tworzony w [Context.externalCacheDir] i oznaczony do usunięcia
00492  * przy zamknięciu JVM. Po zapisaniu przez aplikację aparatu URI jest dodawane
00493  * do listy `selectedImageUris` wewnątrz [AddScreen].
00494  *
00495  * @param context Kontekst Activity lub aplikacji.
00496  * @return URI `content://` obsługiwane przez FileProvider `com.example.otomotuzplus.provider`.
00497  */
00498 fun createTempImageUri(context: android.content.Context): Uri {
00499     val tempFile = File.createTempFile("temp_car_image_", ".jpg", context.externalCacheDir).apply {
00500         createNewFile()
00501         deleteOnExit()
00502     }
00503     return FileProvider.getUriForFile(
00504         context,
00505         "com.example.otomotuzplus.provider",
00506         tempFile
00507     )
00508 }

```

4.35 Dokumentacja pliku

`app/src/main/java/com/example/otomotuzplus/ui/screens/details/AdDetail↔
Screen.kt`

Pełnoekranowy widok szczegółów ogłoszenia z gestowym ujawnianiem kontaktu.

Pakiety

- package `context`

Funkcje

- Object `context.onSensorChanged` (Object event)
- Object `context.onAccuracyChanged` (Object sensor, Object accuracy)
- Object `context.SpecItem` (Object icon, Object `title`, Object subtitle, Object modifier)

Pojedynczy wiersz w siatce specyfikacji pojazdu.

Zmienne

- Object `context.location`
- Object `context.price`
- Object `context.seller`
- Object `context.phone`
- Object `context.isContactVisible`
- Object `context.sensorManager`
- Object `context.accelerometer`
- Object `context.listener`
- Object `context.specs`
- Object `context.pagerState`

4.35.1 Opis szczegółowy

Pełnoekranowy widok szczegółów ogłoszenia z gestowym ujawnianiem kontaktu.

Definicja w pliku `AdDetailScreen.kt`.

4.36 AdDetailScreen.kt

[Idź do dokumentacji tego pliku.](#)

```

00001 /**
00002  * @file AdDetailScreen.kt
00003  * @brief Pełnoekranowy widok szczegółów ogłoszenia z gestowym ujawnianiem kontaktu.
00004  */
00005 package com.example.otomotuzplus.ui.screens.details
00006
00007 import android.content.Context
00008 import android.content.Intent
00009 import android.hardware.Sensor
00010 import android.hardware.SensorEvent
00011 import android.hardware.SensorEventListener
00012 import android.hardware.SensorManager
00013 import android.net.Uri
00014 import androidx.compose.foundation.background
00015 import androidx.compose.foundation.clickable
00016 import androidx.compose.foundation.layout.*
00017 import androidx.compose.foundation.rememberScrollState
00018 import androidx.compose.foundation.shape.RoundedCornerShape
00019 import androidx.compose.foundation.verticalScroll
00020 import androidx.compose.foundation.pager.HorizontalPager
00021 import androidx.compose.foundation.pager.rememberPagerState
00022 import androidx.compose.material.icons.Icons
00023 import androidx.compose.material.icons.filled.*
00024 import androidx.compose.material3.*
00025 import androidx.compose.runtime.*
00026 import androidx.compose.ui.Alignment
00027 import androidx.compose.ui.Modifier
00028 import androidx.compose.ui.graphics.Color
00029 import androidx.compose.ui.graphics.vector.ImageVector
00030 import androidx.compose.ui.platform.LocalContext
00031 import androidx.compose.ui.text.font.FontWeight
00032 import androidx.compose.ui.unit.dp
00033 import androidx.compose.ui.unit.sp
00034 import androidx.compose.ui.layout.ContentScale
00035 import coil.compose.AsyncImage
00036 import com.example.otomotuzplus.models.CarAd
00037 import com.example.otomotuzplus.ui.models.AppStrings
00038 import com.example.otomotuzplus.ui.models.localizeFuelType
00039 import com.example.otomotuzplus.ui.models.localizeGearboxType
00040 import com.example.otomotuzplus.ui.theme.BrandGold
00041 import kotlin.math.sqrt
00042
00043 /**
00044  * Pełnoekranowy widok szczegółów pojedynczego ogłoszenia pojazdu.
00045  *
00046  * ## Układ
00047  * 1. Przezroczysty [TopAppBar] ze strzałką powrotu.
00048  * 2. Poziomy pager obrazów ([HorizontalPager]) gdy dostępne są zdjęcia, lub
00049  *    ikona zastępcza.
00050  * 3. Zaokrąglona karta zawierająca:
00051  *    - Etykiety "Zweryfikowane ogłoszenie".
00052  *    - Tytuł, lokalizację i cenę.
00053  *    - Siatkę specyfikacji ([SpecItem]): rok, przebieg, paliwo, skrzynia, silnik, moc.
00054  *    - Wiersz identyfikatora sprzedawcy.
00055  *    - Sekcję kontaktową (patrz poniżej).
00056  *
00057  * ## Dotknięcie pokrętle (shake) do ujawnienia kontaktu
00058  * Numer telefonu sprzedawcy jest ukryty za gestem potrząśnięcia. [SensorEventListener]
00059  * zarejestrowany na [Sensor.TYPE_ACCELEROMETER] monitoruje przyspieszenie netto
00060  * (z wyłączeniem grawitacji). Gdy wartość przekroczy  $5 \text{ m/s}^2$  przyciski telefonu
00061  * i SMS są ujawniane. Czujnik jest wyrejestrowany w `onDispose` aby zapobiec
00062  * nadmiernemu zużyciu baterii.
00063  *
00064  * @param car Obiekt [CarAd] do wyświetlenia.
00065  * @param strings Aktywne [AppStrings] dla zlokalizowanych etykiet.
00066  * @param onBackClick Callback wywoływany po dotknięciu strzałki powrotu.
00067  */
00068 @OptIn(ExperimentalMaterial3Api::class)
00069 @Composable
00070 fun AdDetailScreen(
00071     car: CarAd,
00072     strings: AppStrings,
00073     onBackClick: () -> Unit
00074 ) {
00075     val context = LocalContext.current
00076     val title = car.title.trim()
00077     val location = car.locationText.trim()
00078     val price = car.priceText.trim()
00079     val seller = car.sellerEmail.ifEmpty { car.sellerId }.trim()
00080     val phone = car.phoneNumber.filter { it.isDigit() || it == '+' }
00081
00082     var isContactVisible by remember { mutableStateOf(false) }
00083

```

```

00084     DisposableEffect(Unit) {
00085         val sensorManager = context.getSystemService(Context.SENSOR_SERVICE) as SensorManager
00086         val accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
00087
00088         val listener = object : SensorEventListener {
00089             override fun onSensorChanged(event: SensorEvent) {
00090                 val x = event.values[0]
00091                 val y = event.values[1]
00092                 val z = event.values[2]
00093
00094                 val acceleration = sqrt((x * x + y * y + z * z).toDouble()) - 9.81
00095                 if (acceleration > 5) {
00096                     isContactVisible = true
00097                 }
00098             }
00099             override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
00100         }
00101
00102         sensorManager.registerListener(listener, accelerometer, SensorManager.SENSOR_DELAY_UI)
00103
00104         onDispose {
00105             sensorManager.unregisterListener(listener)
00106         }
00107     }
00108
00109     val specs = buildList {
00110         if (car.year.isNotBlank()) add(DetailSpec(Icons.Default.DateRange, car.year.trim(),
00111 strings.yearProduction))
00112         if (car.mileageText.isNotBlank()) add(DetailSpec(Icons.Default.Speed,
00113 "${car.mileageText.trim()} ${strings.unitKm}", strings.mileage))
00114         if (car.fuelText.isNotBlank()) add(DetailSpec(Icons.Default.LocalGasStation,
00115 localizeFuelType(car.fuelText, strings), strings.fuel))
00116         if (car.gearboxText.isNotBlank()) add(DetailSpec(Icons.Default.Settings,
00117 localizeGearboxType(car.gearboxText, strings), strings.gearbox))
00118         if (car.engineCapacity.isNotBlank()) add(DetailSpec(Icons.Default.Build,
00119 "${car.engineCapacity.trim()} ${strings.unitCm3}", strings.engineCapacityLabel))
00120         if (car.powerText.isNotBlank()) add(DetailSpec(Icons.Default.ElectricBolt,
00121 "${car.powerText.trim()} ${strings.unitPower}", strings.power))
00122     }
00123
00124     Column(
00125         modifier = Modifier
00126             .fillMaxSize()
00127             .background(MaterialTheme.colorScheme.background)
00128             .verticalScroll(rememberScrollState())
00129     ) {
00130         TopAppBar(
00131             title = { Text("") },
00132             navigationIcon = {
00133                 IconButton(onClick = onBackClick) {
00134                     Icon(Icons.Default.ArrowBackIosNew, contentDescription = strings.back, tint =
00135 MaterialTheme.colorScheme.onBackground)
00136                 }
00137             },
00138             colors = TopAppBarDefaults.topAppBarColors(containerColor = Color.Transparent)
00139         )
00140
00141         if (car.imageUrls.isNotEmpty()) {
00142             val pagerState = rememberPagerState(pageCount = { car.imageUrls.size })
00143             HorizontalPager(
00144                 state = pagerState,
00145                 modifier = Modifier.fillMaxWidth().height(250.dp)
00146             ) { page ->
00147                 AsyncImage(
00148                     model = car.imageUrls[page],
00149                     contentDescription = null,
00150                     modifier = Modifier.fillMaxSize(),
00151                     contentScale = ContentScale.Crop
00152                 )
00153             } else {
00154                 Box(
00155                     modifier = Modifier
00156                         .fillMaxWidth()
00157                         .height(250.dp)
00158                         .background(MaterialTheme.colorScheme.surfaceVariant),
00159                     contentAlignment = Alignment.Center
00160                 ) {
00161                     Icon(
00162                         imageVector = Icons.Default.DirectionsCar,
00163                         contentDescription = null,
00164                         modifier = Modifier.size(80.dp),
00165                         tint = Color.Gray
00166                     )
00167                 }
00168             }
00169         }
00170     }
00171
00172     Column(

```

```

00164         modifier = Modifier
00165             .fillMaxWidth()
00166             .padding(16.dp)
00167             .background(MaterialTheme.colorScheme.surface, RoundedCornerShape(16.dp))
00168             .padding(16.dp)
00169     ) {
00170         Text(text = strings.verifiedListing, color = Color.Gray, fontSize = 12.sp)
00171         Spacer(modifier = Modifier.height(4.dp))
00172
00173         Row(
00174             modifier = Modifier.fillMaxWidth(),
00175             horizontalArrangement = Arrangement.SpaceBetween,
00176             verticalAlignment = Alignment.CenterVertically
00177         ) {
00178             Column(modifier = Modifier.weight(1f)) {
00179                 if (title.isNotEmpty()) {
00180                     Text(text = title, fontSize = 24.sp, fontWeight = FontWeight.Bold, color =
MaterialTheme.colorScheme.onSurface)
00181                 }
00182                 if (location.isNotEmpty()) {
00183                     Text(text = location, color = Color.Gray, fontSize = 14.sp)
00184                 }
00185             }
00186             if (price.isNotEmpty()) {
00187                 Text(
00188                     text = "$price ${strings.unitCurrency}",
00189                     fontSize = 20.sp,
00190                     fontWeight = FontWeight.ExtraBold,
00191                     color = BrandGold
00192                 )
00193             }
00194         }
00195
00196         if (specs.isNotEmpty()) {
00197             Spacer(modifier = Modifier.height(24.dp))
00198             specs.chunked(2).forEachIndexed { rowIndex, rowItems ->
00199                 Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.spacedBy(8.dp)) {
00200                     rowItems.forEach { spec ->
00201                         SpecItem(icon = spec.icon, title = spec.title, subtitle = spec.subtitle,
modifier = Modifier.weight(1f))
00202                     }
00203                     if (rowItems.size == 1) Spacer(modifier = Modifier.weight(1f))
00204                 }
00205                 if (rowIndex < specs.chunked(2).lastIndex) Spacer(modifier =
Modifier.height(12.dp))
00206             }
00207         }
00208
00209         if (seller.isNotEmpty()) {
00210             Spacer(modifier = Modifier.height(24.dp))
00211             Text(
00212                 text = strings.seller,
00213                 fontWeight = FontWeight.Bold,
00214                 fontSize = 18.sp,
00215                 color = MaterialTheme.colorScheme.onSurface
00216             )
00217             Spacer(modifier = Modifier.height(8.dp))
00218             Row(
00219                 modifier = Modifier
00220                     .fillMaxWidth()
00221                     .background(MaterialTheme.colorScheme.surfaceVariant,
RoundedCornerShape(12.dp))
00222                     .padding(12.dp),
00223                 verticalAlignment = Alignment.CenterVertically
00224             ) {
00225                 Icon(
00226                     Icons.Default.AccountCircle,
00227                     contentDescription = null,
00228                     modifier = Modifier.size(40.dp),
00229                     tint = MaterialTheme.colorScheme.onSurface
00230                 )
00231                 Spacer(modifier = Modifier.width(12.dp))
00232                 Text(
00233                     text = seller,
00234                     fontSize = 16.sp,
00235                     fontWeight = FontWeight.Medium,
00236                     color = MaterialTheme.colorScheme.onSurface
00237                 )
00238             }
00239         }
00240
00241         Spacer(modifier = Modifier.height(24.dp))
00242
00243         if (phone.isNotEmpty()) {
00244             if (isContactVisible) {
00245                 Row(

```

```

00246             modifier = Modifier.fillMaxWidth(),
00247             horizontalArrangement = Arrangement.spacedBy(8.dp)
00248         ) {
00249             Button(
00250                 onClick = {
00251                     try {
00252                         context.startActivity(Intent(Intent.ACTION_DIAL,
00253                             Uri.parse("tel:$phone")))
00254                     } catch (_: Exception) { }
00255                 },
00256                 modifier = Modifier.weight(1f).height(55.dp),
00257                 colors = ButtonDefaults.buttonColors(containerColor = BrandGold),
00258                 shape = RoundedCornerShape(14.dp)
00259             ) {
00260                 Icon(Icons.Default.Phone, contentDescription = null, tint = Color.Black)
00261                 Spacer(modifier = Modifier.width(8.dp))
00262                 Text(strings.callSeller, color = Color.Black, fontWeight =
00263                     FontWeight.Bold)
00264             }
00265             Button(
00266                 onClick = {
00267                     try {
00268                         context.startActivity(Intent(Intent.ACTION_SENDTO,
00269                             Uri.parse("sms:$phone")))
00270                     } catch (_: Exception) { }
00271                 },
00272                 modifier = Modifier.weight(1f).height(55.dp),
00273                 colors = ButtonDefaults.buttonColors(
00274                     containerColor = MaterialTheme.colorScheme.secondaryContainer,
00275                     contentColor = MaterialTheme.colorScheme.onSecondaryContainer
00276                 ),
00277                 shape = RoundedCornerShape(14.dp)
00278             ) {
00279                 Icon(Icons.Default.Message, contentDescription = null)
00280                 Spacer(modifier = Modifier.width(8.dp))
00281                 Text(strings.messageSeller, fontWeight = FontWeight.Bold)
00282             }
00283         } else {
00284             Box(
00285                 modifier = Modifier
00286                     .fillMaxWidth()
00287                     .background(BrandGold.copy(alpha = 0.1f), RoundedCornerShape(12.dp))
00288                     .padding(16.dp),
00289                 contentAlignment = Alignment.Center
00290             ) {
00291                 Text(
00292                     text = strings.shakePhone,
00293                     color = BrandGold,
00294                     fontSize = 14.sp,
00295                     fontWeight = FontWeight.Bold
00296                 )
00297             }
00298         } else {
00299             Box(
00300                 modifier = Modifier
00301                     .fillMaxWidth()
00302                     .background(MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f),
00303                         RoundedCornerShape(12.dp))
00304                     .padding(16.dp),
00305                 contentAlignment = Alignment.Center
00306             ) {
00307                 Text(
00308                     text = strings.noPhoneNumber,
00309                     color = MaterialTheme.colorScheme.onSurfaceVariant,
00310                     fontSize = 14.sp
00311                 )
00312             }
00313         }
00314         Spacer(modifier = Modifier.height(32.dp))
00315     }
00316 }
00317
00318 /**
00319  * Pojedynczy wiersz w siatce specyfikacji pojazdu.
00320  *
00321  * Wyświetla wiodącą ikonę, pogrubioną wartość główną ([title]) oraz etykietę
00322  * pomocniczą ([subtitle]) poniżej.
00323  *
00324  * @param icon      Ikona reprezentująca kategorię specyfikacji.
00325  * @param title     Wartość główna (np. "15 000 km").
00326  * @param subtitle  Etykieta kategorii (np. "Przebieg").
00327  * @param modifier  Opcjonalny [Modifier] stosowany do kontenera wiersza.
00328  */

```



```

00329 @Composable
00330 fun SpecItem(icon: ImageVector, title: String, subtitle: String, modifier: Modifier = Modifier) {
00331     Row(modifier = modifier, verticalAlignment = Alignment.CenterVertically) {
00332         Icon(icon, contentDescription = null, tint = Color.Gray, modifier = Modifier.size(24.dp))
00333         Spacer(modifier = Modifier.width(8.dp))
00334         Column {
00335             Text(text = title, fontWeight = FontWeight.Bold, fontSize = 14.sp, color =
MaterialTheme.colorScheme.onSurface)
00336             Text(text = subtitle, fontSize = 12.sp, color = Color.Gray)
00337         }
00338     }
00339 }
00340
00341 private data class DetailSpec(
00342     val icon: ImageVector,
00343     val title: String,
00344     val subtitle: String
00345 )

```

4.37 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/screens/favorites/FavoritesScreen.kt

Ekran ulubionych ogłoszeń filtrowanych z bazy Firestore.

Pakiety

- package [favoriteItems](#)

Zmienne

- Object [favoriteItems.key](#)
- Object [favoriteItems.carKey](#)
- Object [favoriteItems.value](#)

4.37.1 Opis szczegółowy

Ekran ulubionych ogłoszeń filtrowanych z bazy Firestore.

Definicja w pliku [FavoritesScreen.kt](#).

4.38 FavoritesScreen.kt

[Idź do dokumentacji tego pliku.](#)

```

00001 /**
00002  * @file FavoritesScreen.kt
00003  * @brief Ekran ulubionych ogłoszeń filtrowanych z bazy Firestore.
00004  */
00005 package com.example.otomotuzplus.ui.screens.favorites
00006
00007 import androidx.compose.foundation.background
00008 import androidx.compose.foundation.clickable
00009 import androidx.compose.foundation.layout.Arrangement
00010 import androidx.compose.foundation.layout.Spacer
00011 import androidx.compose.foundation.layout.fillMaxSize
00012 import androidx.compose.foundation.layout.fillMaxWidth
00013 import androidx.compose.foundation.layout.height
00014 import androidx.compose.foundation.layout.padding
00015 import androidx.compose.foundation.lazy.LazyColumn
00016 import androidx.compose.foundation.lazy.items
00017 import androidx.compose.foundation.shape.RoundedCornerShape
00018 import androidx.compose.material3.Card

```

```

00019 import androidx.compose.material3.CardDefaults
00020 import androidx.compose.material3.MaterialTheme
00021 import androidx.compose.material3.Text
00022 import androidx.compose.runtime.Composable
00023 import androidx.compose.ui.Modifier
00024 import androidx.compose.ui.text.font.FontWeight
00025 import androidx.compose.ui.unit.dp
00026 import com.example.otomotuzplus.models.CarAd
00027 import com.example.otomotuzplus.ui.components.ListingCard
00028 import com.example.otomotuzplus.ui.components.ListingCardData
00029 import com.example.otomotuzplus.ui.components.ScreenHeader
00030 import com.example.otomotuzplus.ui.models.AppStrings
00031 import com.example.otomotuzplus.ui.models.localizeFuelType
00032 import com.example.otomotuzplus.ui.models.localizeGearboxType
00033 import com.example.otomotuzplus.ui.theme.Slate400
00034
00035 /**
00036  * Ekran wyświetlający ogłoszenia pojazdów oznaczone przez użytkownika jako ulubione.
00037  *
00038  * Stan ulubionych jest zarządzany w [OtomotUZplusApp] jako [List] kluczy złożonych
00039  * w formie "<documentId>|<title>". Ekran filtruje [allCarsFromDb] względem tych kluczy
00040  * i renderuje [ListingCard] dla każdego dopasowania.
00041  *
00042  * Karta pustego stanu jest wyświetlana gdy brak ulubionych.
00043  *
00044  * @param strings Aktywne [AppStrings] dla zlokalizowanych etykiet.
00045  * @param favoriteCars Lista kluczy ulubionych samochodów z rodzica.
00046  * @param onFavoriteToggle Callback do usunięcia samochodu z ulubionych.
00047  * @param allCarsFromDb Pełna bieżąca lista obiektów [CarAd] z Firestore.
00048  * @param onCarClick Callback z dotkniętym [CarAd] otwierający ekran szczegółów.
00049  * @param modifier Opcjonalny zewnętrzny [Modifier].
00050  */
00051 @Composable
00052 fun FavoritesScreen(
00053     strings: AppStrings,
00054     favoriteCars: List<String>,
00055     onFavoriteToggle: (String) -> Unit,
00056     allCarsFromDb: List<CarAd>,
00057     onCarClick: (CarAd) -> Unit,
00058     modifier: Modifier = Modifier
00059 ) {
00060     val favoriteItems = allCarsFromDb.filter { car ->
00061         val key = "${car.id}|${car.title}"
00062         favoriteCars.contains(key)
00063     }
00064
00065     LazyColumn(
00066         modifier = modifier
00067             .fillMaxSize()
00068             .background(MaterialTheme.colorScheme.background),
00069         verticalArrangement = Arrangement.spacedBy(14.dp)
00070     ) {
00071         item {
00072             Spacer(modifier = Modifier.height(8.dp))
00073             ScreenHeader(title = strings.favorites)
00074         }
00075         item {
00076             Text(
00077                 text = strings.resultsCount.format(favoriteItems.size),
00078                 modifier = Modifier.padding(horizontal = 16.dp),
00079                 color = Slate400,
00080                 fontWeight = FontWeight.Medium
00081             )
00082         }
00083
00084         if (favoriteItems.isEmpty()) {
00085             item {
00086                 Card(
00087                     modifier = Modifier
00088                         .fillMaxWidth()
00089                         .padding(horizontal = 16.dp),
00090                     shape = RoundedCornerShape(18.dp),
00091                     colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface)
00092                 ) {
00093                     Text(
00094                         text = strings.noSearchResults,
00095                         modifier = Modifier.padding(20.dp),
00096                         color = Slate400
00097                     )
00098                 }
00099             }
00100         } else {
00101             items(favoriteItems) { car ->
00102                 val carKey = "${car.id}|${car.title}"
00103
00104                 ListingCard(

```

```

00105         item = ListingCardData(
00106             title = car.title,
00107             year = car.year,
00108             mileageText = car.mileageText.withSuffix(strings.unitKm),
00109             fuelText = localizeFuelType(car.fuelText, strings),
00110             bodyTypeText = localizeGearboxType(car.gearboxText, strings),
00111             driveTypeText = car.engineCapacity.withSuffix(strings.unitCm3),
00112             locationText = car.locationText,
00113             priceText = car.priceText.withSuffix(strings.unitCurrency),
00114             coverImageUrl = car.imageUrls.firstOrNull(),
00115             isFavorite = true,
00116             onFavoriteClick = { onFavoriteToggle(carKey) }
00117         ),
00118         modifier = Modifier
00119             .padding(horizontal = 16.dp)
00120             .clickable { onCarClick(car) }
00121     )
00122 }
00123 }
00124
00125     item {
00126         Spacer(modifier = Modifier.height(10.dp))
00127     }
00128 }
00129 }
00130
00131 private fun String.withSuffix(suffix: String): String {
00132     val value = trim()
00133     return if (value.isEmpty()) "" else "$value $suffix"
00134 }
00135

```

4.39 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/ui/screens/home/HomeScreen.kt

Ekran główny – karuzela marek i lista najnowszych ogłoszeń.

Pakiety

- package [query](#)

Zmienne

- Object [query.context](#)
- Object [query.brands](#)
- Object [query.arrivals](#)
- Object [query.originalCarFromDb](#)
- Object [query.carKey](#)
- Object [query.config](#)
- Object [query.screenWidth](#)
- Object [query.placeholderSize](#)
- Object [query.textSize](#)
- Object [query.tileHeight](#)
- Object [query.padding](#)
- Object [query.titleSize](#)
- Object [query.subtitleSize](#)
- Object [query.spacingIconToTitle](#)
- Object [query.spacingTitleToSubtitle](#)
- Object [query.value](#)

4.39.1 Opis szczegółowy

Ekran główny – karuzela marek i lista najnowszych ogłoszeń.

Definicja w pliku [HomeScreen.kt](#).

4.40 HomeScreen.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file HomeScreen.kt
00003  * @brief Ekran główny - karuzela marek i lista najnowszych ogłoszeń.
00004  */
00005 package com.example.otomotuzplus.ui.screens.home
00006
00007 import androidx.compose.foundation.background
00008 import androidx.compose.foundation.border
00009 import androidx.compose.foundation.clickable
00010 import androidx.compose.foundation.text.KeyboardActions
00011 import androidx.compose.foundation.text.KeyboardOptions
00012 import androidx.compose.foundation.layout.Arrangement
00013 import androidx.compose.foundation.layout.Box
00014 import androidx.compose.foundation.layout.Column
00015 import androidx.compose.foundation.layout.Row
00016 import androidx.compose.foundation.layout.heightIn
00017 import androidx.compose.foundation.layout.Spacer
00018 import androidx.compose.foundation.layout.fillMaxSize
00019 import androidx.compose.foundation.layout.fillMaxWidth
00020 import androidx.compose.foundation.layout.height
00021 import androidx.compose.foundation.layout.padding
00022 import androidx.compose.foundation.layout.size
00023 import androidx.compose.foundation.layout.width
00024 import androidx.compose.foundation.lazy.LazyColumn
00025 import androidx.compose.foundation.lazy.LazyRow
00026 import androidx.compose.foundation.lazy.items
00027 import androidx.compose.foundation.lazy.itemsIndexed
00028 import androidx.compose.foundation.shape.CircleShape
00029 import androidx.compose.foundation.shape.RoundedCornerShape
00030 import androidx.compose.material.icons.Icons
00031 import androidx.compose.material.icons.filled.AddCircle
00032 import androidx.compose.material.icons.filled.DirectionsCar
00033 import androidx.compose.material.icons.filled.Favorite
00034 import androidx.compose.material.icons.filled.FavoriteBorder
00035 import androidx.compose.material.icons.filled.Notifications
00036 import androidx.compose.material.icons.filled.Search
00037 import androidx.compose.material.icons.filled.ShoppingCart
00038 import androidx.compose.material3.Card
00039 import androidx.compose.material3.CardDefaults
00040 import androidx.compose.material3.ExperimentalMaterial3Api
00041 import androidx.compose.material3.Icon
00042 import androidx.compose.material3.IconButton
00043 import androidx.compose.material3.MaterialTheme
00044 import androidx.compose.material3.OutlinedTextField
00045 import androidx.compose.material3.OutlinedTextFieldDefaults
00046 import androidx.compose.material3.Surface
00047 import androidx.compose.material3.Text
00048 import androidx.compose.material3.TextButton
00049 import androidx.compose.runtime.Composable
00050 import androidx.compose.runtime.getValue
00051 import androidx.compose.runtime.mutableStateOf
00052 import androidx.compose.runtime.remember
00053 import androidx.compose.runtime.saveable.rememberSaveable
00054 import androidx.compose.runtime.setValue
00055 import androidx.compose.ui.Alignment
00056 import androidx.compose.ui.Modifier
00057 import androidx.compose.ui.graphics.Color
00058 import androidx.compose.ui.platform.LocalConfiguration
00059 import androidx.compose.ui.text.font.FontWeight
00060 import androidx.compose.ui.text.input.ImeAction
00061 import androidx.compose.ui.text.style.TextOverflow
00062 import androidx.compose.ui.unit.dp
00063 import androidx.compose.ui.unit.sp
00064 import com.example.otomotuzplus.ui.components.ScreenHeader
00065 import com.example.otomotuzplus.ui.components.ListingCard
00066 import com.example.otomotuzplus.ui.components.ListingCardData
00067 import com.example.otomotuzplus.ui.models.AppStrings
00068 import com.example.otomotuzplus.ui.models.localizeFuelType
00069 import com.example.otomotuzplus.ui.models.localizeGearboxType
00070 import com.example.otomotuzplus.ui.models.sampleListings
```

```

00071 import com.example.otomotuzplus.ui.theme.BrandGold
00072 import com.example.otomotuzplus.ui.theme.DarkSlate800
00073 import com.example.otomotuzplus.ui.theme.Slate400
00074 import com.example.otomotuzplus.ui.components.AdmobBanner
00075
00076 /**
00077  * Ekran główny wyświetlany gdy wybrano [AppDestinations.HOME].
00078  *
00079  * Wyświetla kolejno:
00080  * 1. [ScreenHeader] z przyciskiem ikony powiadomień.
00081  * 2. Pasek wyszukiwania tekstowego przesyłany przez akcję IME "Szukaj".
00082  * 3. Dwa [ActionTile]: "Kup" (nawiguje do Szukaj) i "Sprzedaj" (nawiguje do Dodaj).
00083  * 4. Poziomy przewijany rząd [BrandPlaceholderTile] do szybkiego filtrowania marki.
00084  * 5. Pionowa lista [ListingCard] załadowanych z Firestore, z [AdmobBanner]
00085  * wstawianym po każdej trzeciej karcie.
00086  *
00087  * @param strings Aktywna instancja [AppStrings] do lokalizacji.
00088  * @param carsFromDb Lista ogłoszeń [CarAd] na żywo z Firestore.
00089  * @param modifier Opcjonalny zewnętrzny [Modifier].
00090  * @param onNavigateToSearch Callback do otwarcia zakładki Szukaj bez filtrów.
00091  * @param onNavigateToAdd Callback do otwarcia zakładki Dodaj.
00092  * @param onSearchSubmit Callback z przesłanym ciągiem zapytania.
00093  * @param onBrandSelect Callback z nazwą marki gdy kliknięto kafelek marki.
00094  * @param onSeeAllClick Callback dla przycisku "Zobacz wszystko" w sekcji marek.
00095  * @param onNotificationsClick Callback dla przycisku ikony powiadomień.
00096  * @param favoriteCars Zestaw kluczy ulubionych samochodów przechowywany przez rodzica.
00097  * @param onFavoriteToggle Callback do przełączania stanu ulubionych samochodu.
00098  * @param onCarClick Callback z klikniętym [CarAd] do otwarcia [AdDetailScreen].
00099  */
00100 @OptIn(ExperimentalMaterial3Api::class)
00101 @Composable
00102 fun HomeScreen(
00103     strings: AppStrings,
00104     carsFromDb: List<com.example.otomotuzplus.models.CarAd>,
00105     modifier: Modifier = Modifier,
00106     onNavigateToSearch: () -> Unit = {},
00107     onNavigateToAdd: () -> Unit = {},
00108     onSearchSubmit: (String) -> Unit = {},
00109     onBrandSelect: (String) -> Unit = {},
00110     onSeeAllClick: () -> Unit = {},
00111     onNotificationsClick: () -> Unit = {},
00112     favoriteCars: List<String> = emptyList(),
00113     onFavoriteToggle: (String) -> Unit = {},
00114     onCarClick: (com.example.otomotuzplus.models.CarAd) -> Unit = {}
00115 ) {
00116     var query by rememberSaveable { mutableStateOf("") }
00117     val context = androidx.compose.ui.platform.LocalContext.current
00118
00119     val brands = remember {
00120         listOf("Porsche", "BMW", "Mercedes", "Audi", "VW")
00121     }
00122
00123     val arrivals = remember(carsFromDb, strings) {
00124         carsFromDb.map { listing ->
00125             ListingCardData(
00126                 title = listing.title,
00127                 year = listing.year,
00128                 mileageText = listing.mileageText.withSuffix(strings.unitKm),
00129                 fuelText = localizeFuelType(listing.fuelText, strings),
00130                 bodyTypeText = localizeGearboxType(listing.gearboxText, strings),
00131                 driveTypeText = listing.engineCapacity.withSuffix(strings.unitCm3),
00132                 locationText = listing.locationText,
00133                 priceText = listing.priceText.withSuffix(strings.unitCurrency),
00134                 coverImageUrl = listing.imageUrls.firstOrNull(),
00135                 isFavorite = false,
00136                 onFavoriteClick = {}
00137             )
00138         }
00139     }
00140
00141     LazyColumn(
00142         modifier = modifier
00143             .fillMaxSize()
00144             .background(MaterialTheme.colorScheme.background),
00145         verticalArrangement = Arrangement.spacedBy(14.dp)
00146     ) {
00147         item {
00148             Spacer(modifier = Modifier.height(8.dp))
00149             ScreenHeader(title = "OtomotUZ++") {
00150                 Surface(
00151                     shape = RoundedCornerShape(12.dp),
00152                     color = MaterialTheme.colorScheme.surface,
00153                     tonalElevation = 2.dp,
00154                     shadowElevation = 1.dp
00155                 ) {
00156                     IconButton(onClick = onNotificationsClick) {
00157                         Icon(

```

```

00158             imageVector = Icons.Default.Notifications,
00159             contentDescription = null,
00160             tint = MaterialTheme.colorScheme.onSurface
00161         )
00162     }
00163 }
00164 }
00165 }
00166
00167 item {
00168     SearchBar(
00169         query = query,
00170         onQueryChange = { query = it },
00171         placeholder = strings.searchPlaceholder,
00172         onSubmit = onSearchSubmit
00173     )
00174 }
00175
00176 item {
00177     Row(
00178         modifier = Modifier
00179             .fillMaxWidth()
00180             .padding(horizontal = 16.dp),
00181         horizontalArrangement = Arrangement.spacedBy(14.dp)
00182     ) {
00183         ActionTile(
00184             modifier = Modifier.weight(1f),
00185             title = strings.buy,
00186             subtitle = strings.findCar,
00187             icon = Icons.Default.ShoppingCart,
00188             background = BrandGold,
00189             foreground = Color(0xFF111111),
00190             onClick = onNavigateToSearch
00191         )
00192         ActionTile(
00193             modifier = Modifier.weight(1f),
00194             title = strings.sell,
00195             subtitle = strings.listYourCar,
00196             icon = Icons.Default.AddCircle,
00197             background = DarkSlate800,
00198             foreground = Color.White,
00199             onClick = onNavigateToAdd
00200         )
00201     }
00202 }
00203
00204 item {
00205     SectionHeader(
00206         title = strings.popularBrands,
00207         actionLabel = strings.seeAll,
00208         onActionClick = onSeeAllClick
00209     )
00210 }
00211
00212 item {
00213     LazyRow(
00214         horizontalArrangement = Arrangement.spacedBy(14.dp),
00215         contentPadding = androidx.compose.foundation.layout.PaddingValues(horizontal = 16.dp)
00216     ) {
00217         items(brands) { brand ->
00218             BrandPlaceholderTile(
00219                 brand = brand,
00220                 onClick = { onBrandSelect(brand) }
00221             )
00222         }
00223     }
00224 }
00225
00226 item {
00227     SectionHeader(title = strings.freshArrivals, actionLabel = null)
00228 }
00229
00230 itemsIndexed(arrivals) { index, carData ->
00231     val originalCarFromDb = carsFromDb[index]
00232
00233     val carKey = "${originalCarFromDb.id}|${originalCarFromDb.title}"
00234
00235     ListingCard(
00236         item = carData.copy(
00237             isFavorite = favoriteCars.contains(carKey),
00238             onFavoriteClick = { onFavoriteToggle(carKey) }
00239         ),
00240         modifier = Modifier
00241             .padding(horizontal = 16.dp)
00242             .clickable {
00243                 onCarClick(originalCarFromDb)
00244             }
00245     )
00246 }

```

```

00245         )
00246         if ((index + 1) % 3 == 0) {
00247             Spacer(modifier = Modifier.height(14.dp))
00248             AdmobBanner(modifier = Modifier.padding(horizontal = 16.dp))
00249         }
00250     }
00251     item {
00252         Spacer(modifier = Modifier.height(8.dp))
00253     }
00254 }
00255 }
00256 }
00257
00258 @Composable
00259 private fun BrandPlaceholderTile(
00260     brand: String,
00261     onClick: () -> Unit
00262 ) {
00263     Column(
00264         horizontalAlignment = Alignment.CenterHorizontally,
00265         modifier = Modifier.width(68.dp)
00266     ) {
00267         Card(
00268             onClick = onClick,
00269             modifier = Modifier.size(58.dp),
00270             shape = RoundedCornerShape(16.dp),
00271             colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
00272             elevation = CardDefaults.cardElevation(defaultElevation = 1.dp)
00273         ) {
00274             Box(
00275                 modifier = Modifier
00276                     .fillMaxSize()
00277                     .border(1.dp, MaterialTheme.colorScheme.outlineVariant,
00278                         RoundedCornerShape(16.dp)),
00279                 contentAlignment = Alignment.Center
00280             ) {
00281                 Text(
00282                     text = brand.firstOrNull()?.uppercase() ?: "?",
00283                     color = MaterialTheme.colorScheme.onSurface,
00284                     fontWeight = FontWeight.Bold,
00285                     fontSize = 20.sp
00286                 )
00287             }
00288             Spacer(modifier = Modifier.height(8.dp))
00289             Text(
00290                 text = brand,
00291                 fontSize = 13.sp,
00292                 color = MaterialTheme.colorScheme.onBackground,
00293                 maxLines = 1,
00294                 overflow = TextOverflow.Ellipsis
00295             )
00296         }
00297     }
00298
00299 @OptIn(ExperimentalMaterial3Api::class)
00300 @Composable
00301 private fun SearchBar(
00302     query: String,
00303     onQueryChange: (String) -> Unit,
00304     placeholder: String,
00305     onSubmit: (String) -> Unit
00306 ) {
00307     val config = LocalConfiguration.current
00308     val screenWidth = config.screenWidthDp
00309     val placeholderSize = if (screenWidth < 360) 12.sp else 14.sp
00310     val textSize = if (screenWidth < 360) 13.sp else 15.sp
00311
00312     OutlinedTextField(
00313         value = query,
00314         onValueChange = onQueryChange,
00315         modifier = Modifier
00316             .fillMaxWidth()
00317             .padding(horizontal = 16.dp),
00318         placeholder = { Text(text = placeholder, color = Slate400, fontSize = placeholderSize) },
00319         leadingIcon = { Icon(Icons.Default.Search, contentDescription = null) },
00320         singleLine = true,
00321         textStyle = androidx.compose.material3.LocalTextStyle.current.copy(fontSize = textSize),
00322         keyboardOptions = KeyboardOptions(imeAction = ImeAction.Search),
00323         keyboardActions = KeyboardActions(
00324             onSearch = { onSubmit(query.trim()) }
00325         ),
00326         shape = RoundedCornerShape(18.dp),
00327         colors = OutlinedTextFieldDefaults.colors(
00328             focusedBorderColor = MaterialTheme.colorScheme.outline,
00329             unfocusedBorderColor = MaterialTheme.colorScheme.outline,
00330             focusedContainerColor = MaterialTheme.colorScheme.surface,

```

```

00331         unfocusedContainerColor = MaterialTheme.colorScheme.surface,
00332         focusedTextColor = MaterialTheme.colorScheme.onSurface,
00333         unfocusedTextColor = MaterialTheme.colorScheme.onSurface,
00334         focusedLeadingIconColor = MaterialTheme.colorScheme.onSurfaceVariant,
00335         unfocusedLeadingIconColor = MaterialTheme.colorScheme.onSurfaceVariant,
00336         cursorColor = BrandGold
00337     )
00338 )
00339 }
00340
00341 @Composable
00342 private fun ActionTile(
00343     modifier: Modifier = Modifier,
00344     title: String,
00345     subtitle: String,
00346     icon: androidx.compose.ui.graphics.vector.ImageVector,
00347     background: Color,
00348     foreground: Color,
00349     onClick: () -> Unit = {}
00350 ) {
00351     val config = LocalConfiguration.current
00352     val screenWidth = config.screenWidthDp
00353
00354     val tileHeight: androidx.compose.ui.unit.Dp
00355     val padding: androidx.compose.ui.unit.Dp
00356     val titleSize: androidx.compose.ui.unit.TextUnit
00357     val subtitleSize: androidx.compose.ui.unit.TextUnit
00358     val spacingIconToTitle: androidx.compose.ui.unit.Dp
00359     val spacingTitleToSubtitle: androidx.compose.ui.unit.Dp
00360
00361     when {
00362         screenWidth < 360 -> {
00363             // Very small screens
00364             tileHeight = 85.dp
00365             padding = 8.dp
00366             titleSize = 10.sp
00367             subtitleSize = 14.sp
00368             spacingIconToTitle = 4.dp
00369             spacingTitleToSubtitle = 2.dp
00370         }
00371         screenWidth < 380 -> {
00372             // Small screens (Realme RMX2202 ~360-380dp)
00373             tileHeight = 100.dp
00374             padding = 10.dp
00375             titleSize = 11.sp
00376             subtitleSize = 15.sp
00377             spacingIconToTitle = 6.dp
00378             spacingTitleToSubtitle = 2.dp
00379         }
00380         screenWidth < 420 -> {
00381             // Medium-small (Nothing Phone 2 ~380-410dp)
00382             tileHeight = 125.dp
00383             padding = 12.dp
00384             titleSize = 13.sp
00385             subtitleSize = 18.sp
00386             spacingIconToTitle = 8.dp
00387             spacingTitleToSubtitle = 2.dp
00388         }
00389         screenWidth < 600 -> {
00390             // Medium screens
00391             tileHeight = 120.dp
00392             padding = 11.dp
00393             titleSize = 14.sp
00394             subtitleSize = 19.sp
00395             spacingIconToTitle = 8.dp
00396             spacingTitleToSubtitle = 2.dp
00397         }
00398         else -> {
00399             // Large screens (tablets)
00400             tileHeight = 150.dp
00401             padding = 14.dp
00402             titleSize = 13.sp
00403             subtitleSize = 18.sp
00404             spacingIconToTitle = 10.dp
00405             spacingTitleToSubtitle = 3.dp
00406         }
00407     }
00408
00409     Card(
00410         onClick = onClick,
00411         modifier = modifier.heightIn(min = tileHeight),
00412         shape = RoundedCornerShape(22.dp),
00413         colors = CardDefaults.cardColors(containerColor = background),
00414         elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
00415     ) {
00416         Column(
00417             modifier = Modifier

```



```

00418         .fillMaxSize()
00419         .padding(padding),
00420         verticalArrangement = Arrangement.Top,
00421         horizontalAlignment = Alignment.Start
00422     ) {
00423         Surface(
00424             shape = CircleShape,
00425             color = foreground.copy(alpha = 0.18f)
00426         ) {
00427             Icon(
00428                 imageVector = icon,
00429                 contentDescription = null,
00430                 tint = foreground,
00431                 modifier = Modifier.padding(6.dp)
00432             )
00433         }
00434         Spacer(modifier = Modifier.height(spacingIconToTitle))
00435         Column {
00436             Text(
00437                 text = title,
00438                 color = foreground.copy(alpha = 0.72f),
00439                 fontSize = titleSize,
00440                 fontWeight = FontWeight.SemiBold,
00441                 modifier = Modifier.fillMaxWidth()
00442             )
00443             Spacer(modifier = Modifier.height(spacingTitleToSubtitle))
00444             Text(
00445                 text = subtitle,
00446                 color = foreground,
00447                 fontSize = subtitleSize,
00448                 fontWeight = FontWeight.Bold,
00449                 modifier = Modifier.fillMaxWidth()
00450             )
00451         }
00452     }
00453 }
00454 }
00455
00456 @Composable
00457 private fun SectionHeader(
00458     title: String,
00459     actionLabel: String?,
00460     onClick: () -> Unit = {}
00461 ) {
00462     Row(
00463         modifier = Modifier
00464             .fillMaxWidth()
00465             .padding(horizontal = 16.dp),
00466         verticalAlignment = Alignment.CenterVertically
00467     ) {
00468         Text(
00469             text = title,
00470             modifier = Modifier.weight(1f),
00471             color = MaterialTheme.colorScheme.onBackground,
00472             fontSize = 19.sp,
00473             fontWeight = FontWeight.Bold
00474         )
00475         if (actionLabel != null) {
00476             TextButton(onClick = onClick) {
00477                 Text(
00478                     text = actionLabel,
00479                     color = BrandGold,
00480                     fontWeight = FontWeight.SemiBold
00481                 )
00482             }
00483         }
00484     }
00485 }
00486
00487 private fun String.withSuffix(suffix: String): String {
00488     val value = trim()
00489     return if (value.isEmpty()) "" else "$value $suffix"
00490 }
00491
00492
00493
00494

```

4.41 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/screens/profile/ProfileScreen.kt

Ekran profilu użytkownika z zakładkami Moje ogłoszenia i Historia.

Pakiety

- package [selectedTab](#)

Funkcje

- Object [selectedTab.MyListingsSection](#) (Object strings, Object [myCars](#), Object onCarClick)
Treść zakładki "Moje ogłoszenia" wewnątrz [ProfileScreen].
- Object [selectedTab.CarListingCard](#) (Object car, Object strings, Object onClick)
Zwarta karta używana w zakładce "Moje ogłoszenia".

Zmienne

- Object [selectedTab.tabs](#)
- Object [selectedTab.user](#)
- Object [selectedTab.userEmail](#)
- Object [selectedTab.creationTimestamp](#)
- Object [selectedTab.memberYear](#)
- Object [selectedTab.myCars](#)
- Object [selectedTab.isSelected](#)
- Object [selectedTab.value](#)

4.41.1 Opis szczegółowy

Ekran profilu użytkownika z zakładkami Moje ogłoszenia i Historia.

Definicja w pliku [ProfileScreen.kt](#).

4.42 ProfileScreen.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file ProfileScreen.kt
00003  * @brief Ekran profilu użytkownika z zakładkami Moje ogłoszenia i Historia.
00004  */
00005 package com.example.otomotuzplus.ui.screens.profile
00006
00007 import androidx.compose.foundation.background
00008 import androidx.compose.foundation.clickable
00009 import androidx.compose.foundation.layout.*
00010 import androidx.compose.foundation.lazy.LazyColumn
00011 import androidx.compose.foundation.lazy.items
00012 import androidx.compose.foundation.shape.CircleShape
00013 import androidx.compose.foundation.shape.RoundedCornerShape
00014 import androidx.compose.material.icons.Icons
00015 import androidx.compose.material.icons.filled.DirectionsCar
00016 import androidx.compose.material.icons.filled.Settings
00017 import androidx.compose.material3.*
00018 import androidx.compose.runtime.*
00019 import androidx.compose.runtime.saveable.rememberSaveable
00020 import androidx.compose.ui.Alignment
00021 import androidx.compose.ui.Modifier
00022 import androidx.compose.ui.draw.clip
00023 import androidx.compose.ui.graphics.Color
00024 import androidx.compose.ui.text.font.FontWeight
00025 import androidx.compose.ui.unit.dp
00026 import androidx.compose.ui.unit.sp
00027 import com.example.otomotuzplus.models.CarAd
00028 import com.example.otomotuzplus.ui.models.AppStrings
00029 import com.google.firebase.auth.FirebaseAuth
00030 import java.util.Calendar
00031 import coil.compose.AsyncImage
```

```

00032 import androidx.compose.ui.layout.ContentScale
00033
00034 /**
00035  * Ekran profilu użytkownika wyświetlający informacje o koncie i własne ogłoszenia sprzedawcy.
00036  *
00037  * Odczytuje aktualnie zalogowanego użytkownika z [FirebaseAuth] w celu uzyskania adresu
00038  * e-mail i roku założenia konta. Filtruje [allCarsFromDb] wg `car.sellerId == userEmail`
00039  * aby wypełnić zakładkę "Moje ogłoszenia".
00040  *
00041  * Dostępne są dwie zakładki: "Moje ogłoszenia" ([MyListingsSection]) i "Historia"
00042  * (placeholder -- zawsze pusta).
00043  *
00044  * Ikona ustawień w nagłówku nawiguje do
00045  * [com.example.otomotuzplus.ui.screens.settings.SettingsScreen] przez [onSettingsClick].
00046  *
00047  * @param onSettingsClick Callback otwierający nakładkę ustawień.
00048  * @param strings Aktywne [AppStrings] dla zlokalizowanych etykiet.
00049  * @param allCarsFromDb Pełna bieżąca lista obiektów [CarAd] z Firestore.
00050  * @param onCarClick Callback z dotkniętym [CarAd] otwierający ekran szczegółów.
00051  */
00052 @Composable
00053 fun ProfileScreen(
00054     onSettingsClick: () -> Unit,
00055     strings: AppStrings,
00056     allCarsFromDb: List<CarAd>,
00057     onCarClick: (CarAd) -> Unit
00058 ) {
00059     var selectedTab by rememberSaveable { mutableIntStateOf(0) }
00060     val tabs = listOf(strings.myListings, strings.history)
00061
00062     val user = FirebaseAuth.getInstance().currentUser
00063     val userEmail = user?.email ?: "Brak emaila"
00064
00065     val creationTimestamp = user?.metadata?.creationTimestamp ?: 0L
00066     val memberYear = if (creationTimestamp > 0) {
00067         Calendar.getInstance().apply { timeInMillis = creationTimestamp
00068         }.get(Calendar.YEAR).toString()
00069     } else {
00070         "2026"
00071     }
00072     val myCars = allCarsFromDb.filter { it.sellerId == userEmail }
00073
00074     Column(
00075         modifier = Modifier
00076             .fillMaxSize()
00077             .background(MaterialTheme.colorScheme.background)
00078     ) {
00079         // Header
00080         Box(
00081             modifier = Modifier
00082                 .fillMaxWidth()
00083                 .padding(16.dp)
00084                 .clip(RoundedCornerShape(24.dp))
00085                 .background(MaterialTheme.colorScheme.surface)
00086                 .padding(20.dp)
00087         ) {
00088             Row(
00089                 verticalAlignment = Alignment.CenterVertically,
00090                 modifier = Modifier.fillMaxWidth()
00091             ) {
00092                 Column(modifier = Modifier.weight(1f)) {
00093                     Text(
00094                         text = userEmail,
00095                         fontSize = 18.sp,
00096                         fontWeight = FontWeight.Bold,
00097                         color = MaterialTheme.colorScheme.onSurface
00098                     )
00099                     Text(
00100                         text = strings.memberSince.format(memberYear),
00101                         fontSize = 14.sp,
00102                         color = Color.Gray
00103                     )
00104                 }
00105                 Column(
00106                     horizontalAlignment = Alignment.CenterHorizontally,
00107                     modifier = Modifier.clickable { onSettingsClick() }
00108                 ) {
00109                     Box(
00110                         modifier = Modifier
00111                             .size(40.dp)
00112                             .clip(CircleShape)
00113                             .background(MaterialTheme.colorScheme.primary.copy(alpha = 0.15f)),
00114                         contentAlignment = Alignment.Center
00115                     ) {
00116                         Icon(
00117

```

```

00118             Icons.Default.Settings,
00119             contentDescription = strings.settings,
00120             tint = MaterialTheme.colorScheme.primary,
00121             modifier = Modifier.size(24.dp)
00122         )
00123     }
00124     Text(
00125         text = strings.settings,
00126         fontSize = 12.sp,
00127         color = Color.Gray,
00128         modifier = Modifier.padding(top = 4.dp)
00129     )
00130 }
00131 }
00132 }
00133 // Tabs
00134 Row(
00135     modifier = Modifier
00136         .fillMaxWidth()
00137         .padding(horizontal = 16.dp)
00138         .clip(RoundedCornerShape(12.dp))
00139         .background(MaterialTheme.colorScheme.surface)
00140         .padding(4.dp)
00141 ) {
00142     tabs.forEachIndexed { index, title ->
00143         val isSelected = selectedTab == index
00144         Box(
00145             modifier = Modifier
00146                 .weight(1f)
00147                 .clip(RoundedCornerShape(8.dp))
00148                 .background(if (isSelected) MaterialTheme.colorScheme.primary else
00149 Color.Transparent)
00150                 .clickable { selectedTab = index }
00151                 .padding(vertical = 10.dp),
00152             contentAlignment = Alignment.Center
00153         ) {
00154             Text(
00155                 text = title,
00156                 color = if (isSelected) MaterialTheme.colorScheme.onPrimary else Color.Gray,
00157                 fontWeight = if (isSelected) FontWeight.Bold else FontWeight.Normal,
00158                 fontSize = 14.sp
00159             )
00160         }
00161     }
00162 }
00163 Spacer(modifier = Modifier.height(16.dp))
00164 if (selectedTab == 0) {
00165     MyListingsSection(strings = strings, myCars = myCars, onCarClick = onCarClick)
00166 } else {
00167     Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
00168         Text(strings.historyEmpty, color = Color.Gray)
00169     }
00170 }
00171 }
00172 }
00173 }
00174 }
00175 /**
00176  * Treść zakładki "Moje ogłoszenia" wewnątrz [ProfileScreen].
00177  *
00178  * Renderuje [LazyColumn] kart [CarListingCard], lub komunikat pustego stanu gdy
00179  * użytkownik nie ma żadnych ogłoszeń.
00180  *
00181  * @param strings Aktywne [AppStrings] dla zlokalizowanych etykiet.
00182  * @param myCars Ogłoszenia należące do bieżącego użytkownika.
00183  * @param onCarClick Callback z dotkniętym [CarAd] otwierający ekran szczegółów.
00184  */
00185 @Composable
00186 fun MyListingsSection(strings: AppStrings, myCars: List<CarAd>, onCarClick: (CarAd) -> Unit) {
00187     if (myCars.isEmpty()) {
00188         Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
00189             Text(text = strings.noListingsYet, color = Color.Gray)
00190         }
00191     } else {
00192         LazyColumn(
00193             modifier = Modifier.fillMaxSize(),
00194             contentPadding = PaddingValues(16.dp),
00195             verticalArrangement = Arrangement.spacedBy(16.dp)
00196         ) {
00197             items(myCars) { car ->
00198                 CarListingCard(car = car, strings = strings, onClick = { onCarClick(car) })
00199             }
00200         }
00201     }
00202 }
00203 }

```

```

00204
00205 /**
00206  * Zwarta karta używana w zakładce "Moje ogłoszenia".
00207  *
00208  * Wyświetla miniaturę zdjęcia (lub ikonę zastępczą), cenę, tytuł, linię specyfikacji
00209  * rok + przebieg oraz odznakę statusu "Aktywne".
00210  *
00211  * @param car      Obiekt [CarAd] do reprezentowania.
00212  * @param strings  Aktywne [AppStrings] dla zlokalizowanych przyrostków jednostek i etykiet.
00213  * @param onClick  Callback wywoływany po dotknięciu karty.
00214  */
00215 @Composable
00216 fun CarListingCard(car: CarAd, strings: AppStrings, onClick: () -> Unit) {
00217     val title = car.title.trim()
00218     val price = car.priceText.withSuffix(strings.unitCurrency)
00219     val specsLine = listOf(
00220         car.year.trim(),
00221         car.mileageText.withSuffix(strings.unitKm)
00222     ).filter { it.isNotEmpty() }
00223     .joinToString(", ")
00224
00225     Card(
00226         modifier = Modifier
00227             .fillMaxWidth()
00228             .clickable { onClick() },
00229         shape = RoundedCornerShape(16.dp),
00230         colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
00231         border = androidx.compose.foundation.BorderStroke(1.dp, MaterialTheme.colorScheme.outline)
00232     ) {
00233         Row(
00234             modifier = Modifier.padding(12.dp),
00235             verticalAlignment = Alignment.CenterVertically
00236         ) {
00237             Box(
00238                 modifier = Modifier
00239                     .size(100.dp)
00240                     .clip(RoundedCornerShape(12.dp))
00241                     .background(MaterialTheme.colorScheme.surfaceVariant),
00242                 contentAlignment = Alignment.Center
00243             ) {
00244                 if (car.imageUrls.isNotEmpty()) {
00245                     AsyncImage(
00246                         model = car.imageUrls.first(),
00247                         contentDescription = "Miniatura ogłoszenia",
00248                         modifier = Modifier.fillMaxSize(),
00249                         contentScale = ContentScale.Crop
00250                     )
00251                 } else {
00252                     Icon(
00253                         imageVector = Icons.Default.DirectionsCar,
00254                         contentDescription = null,
00255                         modifier = Modifier.size(40.dp),
00256                         tint = Color.Gray
00257                     )
00258                 }
00259             }
00260             Spacer(modifier = Modifier.width(16.dp))
00261             Column {
00262                 if (price.isNotEmpty()) {
00263                     Text(
00264                         text = price,
00265                         fontSize = 18.sp,
00266                         fontWeight = FontWeight.Bold,
00267                         color = MaterialTheme.colorScheme.onSurface
00268                     )
00269                 }
00270                 if (title.isNotEmpty()) {
00271                     Text(
00272                         text = title,
00273                         fontSize = 16.sp,
00274                         color = MaterialTheme.colorScheme.onSurface
00275                     )
00276                 }
00277                 if (specsLine.isNotEmpty()) {
00278                     Text(
00279                         text = specsLine,
00280                         fontSize = 14.sp,
00281                         color = Color.Gray
00282                     )
00283                 }
00284             }
00285             Text(
00286                 text = strings.listingsStatus,
00287                 fontSize = 12.sp,
00288                 color = Color(0xFF10B981),
00289                 modifier = Modifier.padding(top = 4.dp)
00290             )
00291         }
00292     }

```

```
00291         )
00292     }
00293 }
00294 }
00295 }
00296
00297 private fun String.withSuffix(suffix: String): String {
00298     val value = trim()
00299     return if (value.isEmpty()) "" else "$value $suffix"
00300 }
00301
```

4.43 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/screens/search/CarClusterOverlay.kt

Nakładka OSMDroid renderująca znaczniki ogłoszeń z grupowaniem siatkowym.

Pakiety

- package [CarClusterOverlay](#)

Funkcje

- Object [draw](#) (Object canvas, Object mapView, Object shadow)
- Object [onSingleTapConfirmed](#) (Object e, Object mapView)
- Object [computeClusters](#) (Object zoom)

Zmienne

- Object [CarClusterOverlay.cars](#)
- Object [CarClusterOverlay.onSingleCarTap](#)
- Object [CarClusterOverlay.onClusterTap](#)
- Object [textPaint](#)

4.43.1 Opis szczegółowy

Nakładka OSMDroid renderująca znaczniki ogłoszeń z grupowaniem siatkowym.

Definicja w pliku [CarClusterOverlay.kt](#).

4.43.2 Dokumentacja funkcji

computeClusters()

```
Object computeClusters (
    Object zoom )
```

Definicja w linii 78 pliku [CarClusterOverlay.kt](#).

draw()

```
Object draw (
    Object canvas,
    Object mapView,
    Object shadow )
```

Definicja w linii 72 pliku [CarClusterOverlay.kt](#).

onSingleTapConfirmed()

```
Object onSingleTapConfirmed (
    Object e,
    Object mapView )
```

Definicja w linii 75 pliku [CarClusterOverlay.kt](#).

4.43.3 Dokumentacja zmiennych**textPaint**

```
Object textPaint
```

Definicja w linii 69 pliku [CarClusterOverlay.kt](#).

4.44 CarClusterOverlay.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file CarClusterOverlay.kt
00003  * @brief Nakładka OSMDroid renderująca znaczniki ogłoszeń z grupowaniem siatkowym.
00004  */
00005 package com.example.otomotuzplus.ui.screens.search
00006
00007 import android.graphics.Canvas
00008 import android.graphics.Paint
00009 import android.graphics.Typeface
00010 import android.view.MotionEvent
00011 import com.example.otomotuzplus.models.CarAd
00012 import org.osmdroid.util.GeoPoint
00013 import org.osmdroid.views.MapView
00014 import org.osmdroid.views.overlay.Overlay
00015 import kotlin.math.abs
00016 import kotlin.math.pow
00017 import kotlin.math.sqrt
00018
00019 /**
00020  * OSMDroid [Overlay] renderujący znaczniki ogłoszeń pojazdów z grupowaniem siatkowym.
00021  *
00022  * ## Algorytm grupowania
00023  * Na każdym poziomie powiększenia mapa jest podzielona na siatkę geograficzną, której
00024  * rozmiar komórki zależy od bieżącego zoomu (grubsze siatki przy niższym zoomie).
00025  * Samochody w tej samej komórce są łączone w jeden [Cluster], którego pozycja to
00026  * średnia współrzędnych jego członków. Lista klastrów jest przeliczana tylko gdy
00027  * zoom zmienia się o więcej niż 0.4 poziomy (histereza zapobiegająca przeliczaniu
00028  * przy każdym przewinięciu).
00029  *
00030  * ## Interakcja
00031  * - Klaster jednosamochodowy → wywoływana jest [onSingleCarTap].
00032  * - Klaster wielosamochodowy → [onClusterTap] jest wywoływany z listą samochodów klastra;
00033  *   [SearchScreen] zawęza aktywny filtr geo do promienia ograniczającego klastra
00034  *   i przełącza z powrotem na widok listy.
00035  *
00036  * @param cars Wszystkie obiekty [CarAd] przechodzące przez aktywne filtry.
```

```

00037 * Samochody ze współrzędnymi (0, 0) są wykluczone z mapy.
00038 * @param onSingleCarTap Callback dla dotknięcia pojedynczego znacznika samochodu.
00039 * @param onClusterTap Callback dla dotknięcia klastra wielu samochodów.
00040 */
00041 class CarClusterOverlay(
00042     private val cars: List<CarAd>,
00043     private val onSingleCarTap: (CarAd) -> Unit,
00044     private val onClusterTap: (List<CarAd>) -> Unit
00045 ) : Overlay() {
00046
00047     /**
00048      * Grupa geograficznie bliskich wpisów [CarAd] renderowanych jako jeden znacznik.
00049      *
00050      * @property geoCenter Środek geograficzny (średnia lat/lng wszystkich [cars]).
00051      * @property cars Wszystkie obiekty [CarAd] w tym klastrze.
00052      * @property screenX Pikselowa współrzędna X na kanwie; aktualizowana przy każdym wywołaniu
00053      * @property screenY Pikselowa współrzędna Y na kanwie; aktualizowana przy każdym wywołaniu
00054      */
00055     data class Cluster(
00056         val geoCenter: GeoPoint,
00057         val cars: List<CarAd>,
00058         var screenX: Float = 0f,
00059         var screenY: Float = 0f
00060     )
00061
00062     private var clusters: List<Cluster> = emptyList()
00063     private var lastZoom = -1.0
00064
00065     private val markerPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
00066         color = android.graphics.Color.parseColor("#E6BD37")
00067         style = Paint.Style.FILL
00068     }
00069     private val strokePaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
00070         color = android.graphics.Color.parseColor("#B8942A")
00071         style = Paint.Style.STROKE
00072         strokeWidth = 3f
00073     }
00074     private val textPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
00075         color = android.graphics.Color.parseColor("#1E1E1E")
00076         textAlign = Paint.Align.CENTER
00077         typeface = Typeface.DEFAULT_BOLD
00078     }
00079
00080     override fun draw(canvas: Canvas?, mapView: MapView?, shadow: Boolean) {
00081         if (shadow || canvas == null || mapView == null) return
00082         val zoom = mapView.zoomLevelDouble
00083         if (abs(zoom - lastZoom) > 0.4 || lastZoom < 0) {
00084             clusters = computeClusters(zoom)
00085             lastZoom = zoom
00086         }
00087         val density = mapView.resources.displayMetrics.density
00088         val projection = mapView.projection
00089         val pt = android.graphics.Point()
00090
00091         for (cluster in clusters) {
00092             projection.toPixels(cluster.geoCenter, pt)
00093             cluster.screenX = pt.x.toFloat()
00094             cluster.screenY = pt.y.toFloat()
00095
00096             if (cluster.cars.size == 1) {
00097                 val r = 14f * density
00098                 canvas.drawCircle(cluster.screenX, cluster.screenY, r, markerPaint)
00099                 canvas.drawCircle(cluster.screenX, cluster.screenY, r, strokePaint)
00100             } else {
00101                 val r = (10f + minOf(cluster.cars.size.toFloat() * 1.5f, 22f)) * density
00102                 canvas.drawCircle(cluster.screenX, cluster.screenY, r, markerPaint)
00103                 canvas.drawCircle(cluster.screenX, cluster.screenY, r, strokePaint)
00104                 textPaint.textSize = 12f * density
00105                 canvas.drawText(
00106                     cluster.cars.size.toString(),
00107                     cluster.screenX,
00108                     cluster.screenY + textPaint.textSize / 3f,
00109                     textPaint
00110                 )
00111             }
00112         }
00113     }
00114
00115     override fun onSingleTapConfirmed(e: MotionEvent?, mapView: MapView?): Boolean {
00116         if (e == null || mapView == null) return false
00117         val density = mapView.resources.displayMetrics.density
00118         val tapX = e.x
00119         val tapY = e.y
00120
00121         for (cluster in clusters) {

```



```

00122         val r = if (cluster.cars.size == 1) 20f * density
00123             else (12f + minOf(cluster.cars.size.toFloat() * 1.5f, 24f)) * density
00124         val dx = tapX - cluster.screenX
00125         val dy = tapY - cluster.screenY
00126         if (sqrt(dx * dx + dy * dy) < r) {
00127             if (cluster.cars.size == 1) onSingleCarTap(cluster.cars[0])
00128             else onClusterTap(cluster.cars)
00129             return true
00130         }
00131     }
00132     return false
00133 }
00134
00135 private fun computeClusters(zoom: Double): List<Cluster> {
00136     val precision = when {
00137         zoom >= 14 -> 3
00138         zoom >= 11 -> 2
00139         zoom >= 8 -> 1
00140         zoom >= 5 -> 0
00141         zoom >= 3 -> -1
00142         else -> -2
00143     }
00144     val factor = 10.0.pow(precision.toDouble())
00145     val buckets = mutableMapOf<Pair<Long, Long>, MutableList<CarAd>>()
00146
00147     for (car in cars) {
00148         if (car.latitude == 0.0 && car.longitude == 0.0) continue
00149         val key = Pair(
00150             (car.latitude * factor).toLong(),
00151             (car.longitude * factor).toLong()
00152         )
00153         buckets.getOrPut(key) { mutableListOf() }.add(car)
00154     }
00155
00156     return buckets.map { (_, group) ->
00157         val avgLat = group.sumOf { it.latitude } / group.size
00158         val avgLng = group.sumOf { it.longitude } / group.size
00159         Cluster(GeoPoint(avgLat, avgLng), group)
00160     }
00161 }
00162 }

```

4.45 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/ui/screens/search/CarMapView.kt

Komponent Compose opakowujący mapę OSMDroid ze zgrupowanymi znacznikami.

Pakiety

- package [context](#)

Funkcje

- Object [context.createUserLocationBitmap](#) (Object context)

Zmienne

- Object [context.hasLocationPermission](#)
- Object [context.permissionLauncher](#)
- Object [context.mapView](#)
- Object [context.myLocationOverlayHolder](#)
- Object [context.lm](#)
- Object [context.loc](#)
- Object [context.locationBitmap](#)
- Object [context.overlay](#)

4.45.1 Opis szczegółowy

Komponent Compose opakowujący mapę OSMDroid ze zgrupowanymi znacznikami.

Definicja w pliku [CarMapView.kt](#).

4.46 CarMapView.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file CarMapView.kt
00003  * @brief Komponent Compose opakowujący mapę OSMDroid ze zgrupowanymi znacznikami.
00004  */
00005 package com.example.otomotuzplus.ui.screens.search
00006
00007 import android.Manifest
00008 import android.content.Context
00009 import android.content.pm.PackageManager
00010 import android.graphics.Bitmap
00011 import android.graphics.Paint
00012 import android.location.LocationManager
00013 import androidx.activity.compose.rememberLauncherForActivityResult
00014 import androidx.activity.result.contract.ActivityResultContracts
00015 import androidx.compose.foundation.layout.Box
00016 import androidx.compose.foundation.layout.Column
00017 import androidx.compose.foundation.layout.Spacer
00018 import androidx.compose.foundation.layout.fillMaxSize
00019 import androidx.compose.foundation.layout.height
00020 import androidx.compose.foundation.layout.padding
00021 import androidx.compose.foundation.layout.size
00022 import androidx.compose.material.icons.Icons
00023 import androidx.compose.material.icons.filled.Add
00024 import androidx.compose.material.icons.filled.Remove
00025 import androidx.compose.material3.FloatingActionButton
00026 import androidx.compose.material3.Icon
00027 import androidx.compose.material3.MaterialTheme
00028 import androidx.compose.runtime.Composable
00029 import androidx.compose.runtime.DisposableEffect
00030 import androidx.compose.runtime.LaunchedEffect
00031 import androidx.compose.runtime.getValue
00032 import androidx.compose.runtime.mutableStateOf
00033 import androidx.compose.runtime.remember
00034 import androidx.compose.runtime.setValue
00035 import androidx.compose.ui.Alignment
00036 import androidx.compose.ui.Modifier
00037 import androidx.compose.ui.draw.clip
00038 import androidx.compose.ui.graphics.RectangleShape
00039 import androidx.compose.ui.platform.LocalContext
00040 import androidx.compose.ui.unit.dp
00041 import androidx.compose.ui.viewinterop.AndroidView
00042 import androidx.core.content.ContextCompat
00043 import com.example.otomotuzplus.models.CarAd
00044 import org.osmdroid.config.Configuration
00045 import org.osmdroid.tileprovider.tilesource.TileSourceFactory
00046 import org.osmdroid.util.BoundingBox
00047 import org.osmdroid.util.GeoPoint
00048 import org.osmdroid.views.MapView
00049 import org.osmdroid.views.overlay.mylocation.GpsMyLocationProvider
00050 import org.osmdroid.views.overlay.mylocation.MyLocationNewOverlay
00051
00052 /**
00053  * Komponent Compose opakowujący OSMDroid [MapView] wyświetlający ogłoszenia pojazdów
00054  * jako zgrupowane znaczniki na mapie.
00055  *
00056  * ## Konfiguracja mapy
00057  * - Źródło kafelków: MAPNIK (standard OpenStreetMap).
00058  * - Zoom wielodotykowy włączony; wbudowane kontrolki zoomu wyłączone (zamiast nich własne FABs).
00059  * - Granice przewijania ograniczone do Europy.
00060  * - Centrum początkowe: Polska (52°N, 20°E) na zoomie 6.
00061  *
00062  * ## Uprawnienia
00063  * `ACCESS_FINE_LOCATION` jest żądane podczas kompozycji jeśli nie zostało jeszcze przyznane.
00064  * Po przyznaniu mapa jest przesuwana do ostatniej znanej lokalizacji urządzenia i dodawany
00065  * jest [MyLocationNewOverlay].
00066  *
00067  * ## Cykl życia
00068  * [DisposableEffect] wywołuje `onResume` / `onPause` / `onDetach` na bazowym [MapView]
00069  * zgodnie z cyklem życia Compose i wyłącza nakładkę lokalizacji przy usuwaniu.
00070  */
```

```

00071 * @param cars          Przefiltrowana lista obiektów [CarAd] do wyświetlenia jako znaczniki.
00072 * @param onSingleCarTap Callback wywoływany gdy użytkownik dotknie znacznika jednego samochodu.
00073 * @param onClusterTap   Callback wywoływany ze wszystkimi samochodami w klastrze po dotknięciu.
00074 * @param modifier       Opcjonalny zewnętrzny [Modifier].
00075 */
00076 @Composable
00077 fun CarMapView(
00078     cars: List<CarAd>,
00079     onSingleCarTap: (CarAd) -> Unit,
00080     onClusterTap: (List<CarAd>) -> Unit,
00081     modifier: Modifier = Modifier
00082 ) {
00083     val context = LocalContext.current
00084
00085     var hasLocationPermission by remember {
00086         mutableStateOf(
00087             ContextCompat.checkSelfPermission(
00088                 context, Manifest.permission.ACCESS_FINE_LOCATION
00089             ) == PackageManager.PERMISSION_GRANTED
00090         )
00091     }
00092
00093     val permissionLauncher = rememberLauncherForActivityResult(
00094         ActivityResultContracts.RequestPermission()
00095     ) { granted -> hasLocationPermission = granted }
00096
00097     LaunchedEffect(Unit) {
00098         if (!hasLocationPermission) {
00099             permissionLauncher.launch(Manifest.permission.ACCESS_FINE_LOCATION)
00100         }
00101     }
00102
00103     val mapView = remember {
00104         Configuration.getInstance().apply {
00105             load(context, context.getSharedPreferences("osmdroid", Context.MODE_PRIVATE))
00106             userAgentValue = "OtomotUZplus/1.0"
00107         }
00108         MapView(context).apply {
00109             setTileSource(TileSourceFactory.MAPNIK)
00110             setMultiTouchControls(true)
00111             setBuiltInZoomControls(false)
00112             isTilesScaledToDpi = true
00113             isHorizontalMapRepetitionEnabled = false
00114             isVerticalMapRepetitionEnabled = false
00115             minZoomLevel = 4.0
00116             maxZoomLevel = 19.0
00117             setScrollableAreaLimitDouble(BoundingBox(72.0, 50.0, 34.0, -25.0))
00118             setLayerType(android.view.View.LAYER_TYPE_HARDWARE, null)
00119             controller.setZoom(6.0)
00120             controller.setCenter(GeoPoint(52.0, 20.0))
00121         }
00122     }
00123
00124     val myLocationOverlayHolder = remember { arrayOfNulls<MyLocationNewOverlay>(1) }
00125
00126     LaunchedEffect(hasLocationPermission) {
00127         if (hasLocationPermission) {
00128             try {
00129                 val lm = context.getSystemService(Context.LOCATION_SERVICE) as LocationManager
00130                 @Suppress("MissingPermission")
00131                 val loc = lm.getLastKnownLocation(LocationManager.GPS_PROVIDER)
00132                     ?: lm.getLastKnownLocation(LocationManager.NETWORK_PROVIDER)
00133                     ?: lm.getLastKnownLocation(LocationManager.PASSIVE_PROVIDER)
00134                 if (loc != null) {
00135                     mapView.controller.setCenter(GeoPoint(loc.latitude, loc.longitude))
00136                     mapView.controller.setZoom(12.0)
00137                 }
00138             } catch (_: SecurityException) {}
00139         }
00140     }
00141
00142     LaunchedEffect(cars, hasLocationPermission) {
00143         mapView.overlays.clear()
00144         myLocationOverlayHolder[0]?.disableMyLocation()
00145         myLocationOverlayHolder[0] = null
00146
00147         mapView.overlays.add(CarClusterOverlay(cars, onSingleCarTap, onClusterTap))
00148
00149         if (hasLocationPermission) {
00150             val locationBitmap = createUserLocationBitmap(context)
00151             val overlay = MyLocationNewOverlay(GpsMyLocationProvider(context), mapView)
00152             overlay.setPersonIcon(locationBitmap)
00153             overlay.setPersonHotspot(locationBitmap.width / 2f, locationBitmap.height / 2f)
00154             overlay.enableMyLocation()
00155             myLocationOverlayHolder[0] = overlay
00156             mapView.overlays.add(overlay)
00157         }
00158     }

```

```

00158         mapView.invalidate()
00159     }
00160
00161     DisposableEffect(Unit) {
00162         mapView.onResume()
00163         onDispose {
00164             myLocationOverlayHolder[0]?.disableMyLocation()
00165             mapView.onPause()
00166             mapView.onDetach()
00167         }
00168     }
00169
00170     Box(modifier = modifier) {
00171         AndroidView(
00172             factory = { mapView },
00173             modifier = Modifier
00174                 .fillMaxSize()
00175                 .clip(RectangleShape)
00176         )
00177         Column(
00178             modifier = Modifier
00179                 .align(Alignment.BottomEnd)
00180                 .padding(bottom = 16.dp, end = 16.dp)
00181         ) {
00182             FloatingActionButton(
00183                 onClick = { mapView.controller.zoomIn() },
00184                 modifier = Modifier.size(48.dp),
00185                 containerColor = MaterialTheme.colorScheme.surface,
00186                 contentColor = MaterialTheme.colorScheme.onSurface
00187             ) {
00188                 Icon(Icons.Default.Add, contentDescription = null)
00189             }
00190             Spacer(Modifier.height(16.dp))
00191             FloatingActionButton(
00192                 onClick = { mapView.controller.zoomOut() },
00193                 modifier = Modifier.size(48.dp),
00194                 containerColor = MaterialTheme.colorScheme.surface,
00195                 contentColor = MaterialTheme.colorScheme.onSurface
00196             ) {
00197                 Icon(Icons.Default.Remove, contentDescription = null)
00198             }
00199         }
00200     }
00201 }
00202
00203 private fun createUserLocationBitmap(context: Context): Bitmap {
00204     val density = context.resources.displayMetrics.density
00205     val size = (density * 20).toInt().coerceAtLeast(16)
00206     val bmp = Bitmap.createBitmap(size, size, Bitmap.Config.ARGB_8888)
00207     val canvas = android.graphics.Canvas(bmp)
00208     val fill = Paint(Paint.ANTI_ALIAS_FLAG).apply {
00209         color = android.graphics.Color.parseColor("#1565C0")
00210         style = Paint.Style.FILL
00211     }
00212     val ring = Paint(Paint.ANTI_ALIAS_FLAG).apply {
00213         color = android.graphics.Color.WHITE
00214         style = Paint.Style.STROKE
00215         strokeWidth = density * 2.5f
00216     }
00217     val cx = size / 2f
00218     val cy = size / 2f
00219     val r = size / 2f - density * 1.5f
00220     canvas.drawCircle(cx, cy, r, fill)
00221     canvas.drawCircle(cx, cy, r, ring)
00222     return bmp
00223 }

```

4.47 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/screens/search/SearchScreen.kt

Ekran wyszukiwania i przeglądania ogłoszeń z filtrami i widokiem mapy.

Pakiety

- package [SearchFilters](#)

Funkcje

- Object [ActiveFiltersRow](#) (Object filters, Object onRemove)
- Object [FiltersCard](#) (Object strings, Object filters, Object onFiltersChange)
- Object [SingleSelectChipRow](#) (Object label, Object options, Object selected, Object onSelectionChange)
- Object [LabeledInput](#) (Object label, Object value, Object onValueChange)

Zmienne

- Object [SearchFilters.brand](#)
- Object [SearchFilters.modelQuery](#)
- Object [SearchFilters.minPrice](#)
- Object [SearchFilters.maxPrice](#)
- Object [SearchFilters.minYear](#)
- Object [SearchFilters.maxYear](#)
- Object [SearchFilters.maxMileage](#)
- Object [SearchFilters.fuelType](#)
- Object [SearchFilters.transmission](#)
- Object [SearchFilters.location](#)
- Object [SearchFilters.radiusKm](#)
- Object [activeFilters](#)
- Object [newShowMap](#)
- Object [config](#)
- Object [screenWidth](#)
- Object [placeholderSize](#)
- Object [textSize](#)
- Object [avgLat](#)
- Object [avgLng](#)
- Object [maxDistKm](#)
- Object [carKey](#)
- Object [labels](#)
- Object [label](#)
- Object [value](#)

4.47.1 Opis szczegółowy

Ekran wyszukiwania i przeglądania ogłoszeń z filtrami i widokiem mapy.

Definicja w pliku [SearchScreen.kt](#).

4.47.2 Dokumentacja funkcji

ActiveFiltersRow()

```
Object ActiveFiltersRow (  
    Object filters,  
    Object onRemove )
```

Definicja w linii 226 pliku [SearchScreen.kt](#).

FiltersCard()

```
Object FiltersCard (
    Object strings,
    Object filters,
    Object onFiltersChange )
```

Definicja w linii 229 pliku [SearchScreen.kt](#).

LabeledInput()

```
Object LabeledInput (
    Object label,
    Object value,
    Object onValueChange )
```

Definicja w linii 235 pliku [SearchScreen.kt](#).

SingleSelectChipRow()

```
Object SingleSelectChipRow (
    Object label,
    Object options,
    Object selected,
    Object onSelectionChange )
```

Definicja w linii 232 pliku [SearchScreen.kt](#).

4.47.3 Dokumentacja zmiennych**activeFilters**

```
Object activeFilters
```

Definicja w linii 175 pliku [SearchScreen.kt](#).

avgLat

```
Object avgLat
```

Definicja w linii 212 pliku [SearchScreen.kt](#).

avgLng

```
Object avgLng
```

Definicja w linii 213 pliku [SearchScreen.kt](#).

carKey

Object carKey

Definicja w linii 218 pliku [SearchScreen.kt](#).

config

Object config

Definicja w linii 193 pliku [SearchScreen.kt](#).

label

Object label

Definicja w linii 276 pliku [SearchScreen.kt](#).

labels

Object labels

Definicja w linii 273 pliku [SearchScreen.kt](#).

maxDistKm

Object maxDistKm

Definicja w linii 214 pliku [SearchScreen.kt](#).

newShowMap

Object newShowMap

Definicja w linii 179 pliku [SearchScreen.kt](#).

placeholderSize

Object placeholderSize

Definicja w linii 195 pliku [SearchScreen.kt](#).

screenWidth

Object screenWidth

Definicja w linii 194 pliku [SearchScreen.kt](#).

textSize

Object textSize

Definicja w linii 196 pliku [SearchScreen.kt](#).

value

Object value

Definicja w linii 281 pliku [SearchScreen.kt](#).

4.48 SearchScreen.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file SearchScreen.kt
00003  * @brief Ekran wyszukiwania i przeglądania ogłoszeń z filtrami i widokiem mapy.
00004  */
00005 package com.example.otomotuzplus.ui.screens.search
00006
00007 import androidx.compose.foundation.background
00008 import androidx.compose.foundation.clickable
00009 import androidx.compose.foundation.layout.Arrangement
00010 import androidx.compose.foundation.layout.Box
00011 import androidx.compose.foundation.layout.Column
00012 import androidx.compose.foundation.layout.Row
00013 import androidx.compose.foundation.layout.Spacer
00014 import androidx.compose.foundation.layout.fillMaxSize
00015 import androidx.compose.foundation.layout.fillMaxWidth
00016 import androidx.compose.foundation.layout.height
00017 import androidx.compose.foundation.layout.padding
00018 import androidx.compose.foundation.layout.width
00019 import androidx.compose.foundation.lazy.LazyColumn
00020 import androidx.compose.foundation.lazy.LazyRow
00021 import androidx.compose.foundation.lazy.items
00022 import androidx.compose.foundation.shape.RoundedCornerShape
00023 import androidx.compose.material.icons.Icons
00024 import androidx.compose.material.icons.automirrored.filled.Sort
00025 import androidx.compose.material.icons.filled.Close
00026 import androidx.compose.material.icons.filled.KeyboardArrowDown
00027 import androidx.compose.material.icons.filled.Map
00028 import androidx.compose.material.icons.filled.Search
00029 import androidx.compose.material.icons.filled.Tune
00030 import androidx.compose.material.icons.filled.ViewList
00031 import androidx.compose.material3.Card
00032 import androidx.compose.material3.CardDefaults
00033 import androidx.compose.material3.DropdownMenu
00034 import androidx.compose.material3.DropdownMenuItem
00035 import androidx.compose.material3.ExperimentalMaterial3Api
00036 import androidx.compose.material3.FilterChip
00037 import androidx.compose.material3.Icon
00038 import androidx.compose.material3.IconButton
00039 import androidx.compose.material3.MaterialTheme
00040 import androidx.compose.material3.OutlinedTextField
00041 import androidx.compose.material3.OutlinedTextFieldDefaults
00042 import androidx.compose.material3.Surface
00043 import androidx.compose.material3.Text
00044 import androidx.compose.material3.TextButton
00045 import androidx.compose.runtime.Composable
00046 import androidx.compose.runtime.LaunchedEffect
00047 import androidx.compose.runtime.getValue
00048 import androidx.compose.runtime.mutableStateOf
00049 import androidx.compose.runtime.remember
00050 import androidx.compose.runtime.saveable.rememberSaveable
00051 import androidx.compose.runtime.setValue
00052 import androidx.compose.ui.Alignment
00053 import androidx.compose.ui.Modifier
00054 import androidx.compose.ui.platform.LocalConfiguration
00055 import androidx.compose.ui.text.font.FontWeight
00056 import androidx.compose.ui.text.style.TextOverflow
00057 import androidx.compose.ui.unit.dp
00058 import androidx.compose.ui.unit.sp
00059 import com.example.otomotuzplus.data.geocodeCity
```



```

00060 import com.example.otomotuzplus.models.CarAd
00061 import com.example.otomotuzplus.ui.components.ListingCard
00062 import com.example.otomotuzplus.ui.components.ListingCardData
00063 import com.example.otomotuzplus.ui.components.ScreenHeader
00064 import com.example.otomotuzplus.ui.models.AppStrings
00065 import com.example.otomotuzplus.ui.models.fuelTypeKey
00066 import com.example.otomotuzplus.ui.models.gearboxTypeKey
00067 import com.example.otomotuzplus.ui.models.localizeFuelType
00068 import com.example.otomotuzplus.ui.models.localizeGearboxType
00069 import com.example.otomotuzplus.ui.theme.BrandGold
00070 import com.example.otomotuzplus.ui.theme.Slate400
00071 import java.util.Locale
00072 import kotlinx.coroutines.delay
00073 import kotlin.math.asin
00074 import kotlin.math.cos
00075 import kotlin.math.pow
00076 import kotlin.math.sin
00077 import kotlin.math.sqrt
00078
00079 /**
00080  * Dostępne opcje sortowania wyników ogłoszeń pojazdów.
00081  *
00082  * * - [NAME_ASC] / [NAME_DESC] -- alfabetycznie według tytułu.
00083  * * - [PRICE_ASC] / [PRICE_DESC] -- numerycznie według cyfr z [CarAd.priceText].
00084  * * - [YEAR_DESC] -- najnowszy rok produkcji jako pierwszy.
00085  * * - [MILEAGE_ASC] -- najniższy przebieg jako pierwszy.
00086  */
00087 enum class SortOption {
00088     NAME_ASC, NAME_DESC, PRICE_ASC, PRICE_DESC, YEAR_DESC, MILEAGE_ASC
00089 }
00090
00091 /**
00092  * Niemutowalna migawka aktywnego stanu filtrów wyszukiwania.
00093  *
00094  * * Wszystkie pola tekstowe domyślnie puste (brak zastosowanego filtra).
00095  * * Granice zakresów numerycznych przechowywane jako ciągi, aby zachować
00096  * * surowe dane wejściowe użytkownika podczas edycji pola tekstowego.
00097  *
00098  * * @property brand Filtr podciągu nazwy marki (bez rozróżniania wielkości liter).
00099  * * @property modelQuery Filtr podciągu nazwy modelu.
00100  * * @property minPrice Dolna granica ceny (tylko cyfry).
00101  * * @property maxPrice Górna granica ceny (tylko cyfry).
00102  * * @property minYear Minimalny rok produkcji (czterocyfrowy ciąg).
00103  * * @property maxYear Maksymalny rok produkcji (czterocyfrowy ciąg).
00104  * * @property maxMileage Maksymalny przebieg (tylko cyfry).
00105  * * @property fuelType Wybrany ciąg wyświetlania rodzaju paliwa lub `null` jeśli bez filtra.
00106  * * @property transmission Wybrany ciąg wyświetlania skrzyni biegów lub `null` jeśli bez filtra.
00107  * * @property location Nazwa miasta używana do filtrowania geolokalizacyjnego.
00108  * * @property radiusKm Promień wyszukiwania w kilometrach; `null` oznacza tylko dopasowanie
00109  * * tekstowe.
00110  */
00111 data class SearchFilters(
00112     val brand: String? = null,
00113     val modelQuery: String = "",
00114     val minPrice: String = "",
00115     val maxPrice: String = "",
00116     val minYear: String = "",
00117     val maxYear: String = "",
00118     val maxMileage: String = "",
00119     val fuelType: String? = null,
00120     val transmission: String? = null,
00121     val location: String = "",
00122     val radiusKm: Int? = null
00123 )
00124
00125 private enum class ActiveFilterKey {
00126     QUERY, BRAND, MODEL, PRICE_RANGE, YEAR_RANGE, MAX_MILEAGE, FUEL, TRANSMISSION, LOCATION,
00127     CLUSTER_AREA
00128 }
00129
00130 private data class ActiveFilterItem(val key: ActiveFilterKey, val label: String)
00131
00132 /**
00133  * Ekran wyszukiwania/przeglądania wyświetlany gdy wybrano [AppDestinations.SEARCH].
00134  *
00135  * * Funkcje:
00136  * * - Wyszukiwanie tekstowe w czasie rzeczywistym z odłożonym rozwiązywaniem geo dla filtrów promienia
00137  * * miasta.
00138  * * - Rozwijana [FiltersCard] z filtrami: marka, model, zakres cen, zakres roku, przebieg,
00139  * * rodzaj paliwa, skrzynia biegów i promień geo.
00140  * * - Chipy aktywnych filtrów z możliwością indywidualnego usunięcia.
00141  * * - Sortowalna lista wyników ([SortOption]).
00142  * * - Przełączanie między widokiem listy a [CarMapView] (mapa OSM z [CarClusterOverlay]).
00143  *
00144  * * Początkowe filtry (ustawione przez głębokie linki z ekranu Home) konsumowane raz przez
00145  * * [onInitialFiltersConsumed], więc nie są ponownie stosowane przy kolejnych rekompozycjach.
00146  */

```

```

00144 * @param strings          Aktywna instancja [AppStrings] do lokalizacji.
00145 * @param modifier          Opcjonalny zewnętrzny [Modifier].
00146 * @param initialQuery      Wstępnie wypełnione zapytanie tekstowe lub `null`.
00147 * @param initialBrand      Wstępnie wybrany filtr marki lub `null`.
00148 * @param initialShowFilters Czy natychmiast otworzyć panel filtrów.
00149 * @param favoriteCars      Klucze ulubionych samochodów od rodzica.
00150 * @param onFavoriteToggle   Callback do przełączania ulubionych.
00151 * @param allCarsFromDb     Pełna lista [CarAd] na żywo z Firestore.
00152 * @param onCarClick        Callback z klikniętym [CarAd] do otwarcia szczegółów.
00153 * @param onInitialFiltersConsumed Wywołany raz po zastosowaniu początkowych filtrów.
00154 */
00155 @OptIn(ExperimentalMaterial3Api::class)
00156 @Composable
00157 fun SearchScreen(
00158     strings: AppStrings,
00159     modifier: Modifier = Modifier,
00160     initialQuery: String? = null,
00161     initialBrand: String? = null,
00162     initialShowFilters: Boolean? = null,
00163     favoriteCars: List<String> = emptyList(),
00164     onFavoriteToggle: (String) -> Unit = {},
00165     allCarsFromDb: List<CarAd>,
00166     onCarClick: (CarAd) -> Unit,
00167     onInitialFiltersConsumed: () -> Unit = {}
00168 ) {
00169     var query by rememberSaveable { mutableStateOf("") }
00170     var showFilters by remember { mutableStateOf(false) }
00171     var showSortMenu by rememberSaveable { mutableStateOf(false) }
00172     var sortOption by rememberSaveable { mutableStateOf(SortOption.NAME_ASC) }
00173     var filters by remember { mutableStateOf(SearchFilters()) }
00174     var showMapView by rememberSaveable { mutableStateOf(false) }
00175
00176     var geocodedCenter by remember { mutableStateOf<Pair<Double, Double>?>(null) }
00177     var clusterCenter by remember { mutableStateOf<Pair<Double, Double>?>(null) }
00178     var clusterRadiusKm by remember { mutableStateOf<Double?>(null) }
00179
00180     var visibleCars by remember { mutableStateOf(allCarsFromDb) }
00181
00182     LaunchedEffect(initialQuery, initialBrand, initialShowFilters) {
00183         val hasPresetQuery = !initialQuery.isNullOrEmpty()
00184         val hasPresetBrand = !initialBrand.isNullOrEmpty()
00185         if (hasPresetQuery || hasPresetBrand || initialShowFilters != null) {
00186             query = initialQuery.orEmpty()
00187             filters = SearchFilters(brand = initialBrand)
00188             sortOption = SortOption.NAME_ASC
00189             showFilters = initialShowFilters ?: false
00190             onInitialFiltersConsumed()
00191         }
00192     }
00193
00194     LaunchedEffect(filters.location, filters.radiusKm) {
00195         geocodedCenter = null
00196         clusterCenter = null
00197         clusterRadiusKm = null
00198         if (filters.location.isNotBlank() && filters.radiusKm != null) {
00199             delay(400)
00200             geocodedCenter = geocodeCity(filters.location)
00201         }
00202     }
00203
00204     LaunchedEffect(query, filters, sortOption, allCarsFromDb, geocodedCenter, clusterCenter,
00205         clusterRadiusKm) {
00206         visibleCars = allCarsFromDb
00207             .filter { it.matches(query, filters, geocodedCenter, clusterCenter, clusterRadiusKm) }
00208             .sortedWith(sortComparator(sortOption))
00209     }
00210
00211     val activeFilters = remember(query, filters, strings, clusterCenter, clusterRadiusKm) {
00212         buildActiveFilterItems(query, filters, strings, clusterCenter, clusterRadiusKm)
00213     }
00214
00215     Column(
00216         modifier = modifier
00217             .fillMaxSize()
00218             .background(MaterialTheme.colorScheme.background)
00219     ) {
00220         Spacer(modifier = Modifier.height(8.dp))
00221
00222         ScreenHeader(
00223             title = strings.search,
00224             trailingWidth = 176.dp
00225         ) {
00226             Surface(
00227                 shape = RoundedCornerShape(12.dp),
00228                 color = MaterialTheme.colorScheme.surface,
00229                 tonalElevation = 2.dp
00230             ) {

```

```

00230         IconButton(onClick = {
00231             val newShowMap = !showMapView
00232             showMapView = newShowMap
00233             if (newShowMap) showFilters = false
00234         }) {
00235             Icon(
00236                 imageVector = if (showMapView) Icons.Default.ViewList else Icons.Default.Map,
00237                 contentDescription = if (showMapView) strings.listViewToggle else
strings.mapViewToggle,
00238                 tint = if (showMapView) BrandGold else MaterialTheme.colorScheme.onSurface
00239             )
00240         }
00241     }
00242     Spacer(modifier = Modifier.width(8.dp))
00243     Surface(
00244         shape = RoundedCornerShape(12.dp),
00245         color = MaterialTheme.colorScheme.surface,
00246         tonalElevation = 2.dp
00247     ) {
00248         IconButton(onClick = { showFilters = !showFilters }) {
00249             Icon(
00250                 imageVector = Icons.Default.Tune,
00251                 contentDescription = strings.filters,
00252                 tint = MaterialTheme.colorScheme.onSurface
00253             )
00254         }
00255     }
00256     Spacer(modifier = Modifier.width(8.dp))
00257     Box {
00258         Surface(
00259             shape = RoundedCornerShape(12.dp),
00260             color = MaterialTheme.colorScheme.surface,
00261             tonalElevation = 2.dp
00262         ) {
00263             IconButton(onClick = { showSortMenu = true }) {
00264                 Icon(
00265                     imageVector = Icons.AutoMirrored.Filled.Sort,
00266                     contentDescription = strings.sortBy,
00267                     tint = MaterialTheme.colorScheme.onSurface
00268                 )
00269             }
00270         }
00271         DropdownMenu(
00272             expanded = showSortMenu,
00273             onDismissRequest = { showSortMenu = false }
00274         ) {
00275             sortOptions(strings).forEach { option ->
00276                 DropdownMenuItem(
00277                     text = { Text(option.label) },
00278                     onClick = {
00279                         sortOption = option.value
00280                         showSortMenu = false
00281                     }
00282                 )
00283             }
00284         }
00285     }
00286 }
00287
00288 if (!showMapView) {
00289     Spacer(modifier = Modifier.height(8.dp))
00290
00291     val config = LocalConfiguration.current
00292     val screenWidth = config.screenWidthDp
00293     val placeholderSize = if (screenWidth < 360) 12.sp else 14.sp
00294     val textSize = if (screenWidth < 360) 13.sp else 15.sp
00295
00296     OutlinedTextField(
00297         value = query,
00298         onValueChange = { query = it },
00299         modifier = Modifier
00300             .fillMaxWidth()
00301             .padding(horizontal = 16.dp),
00302         placeholder = {
00303             Text(strings.searchPlaceholder, color = Slate400, fontSize = placeholderSize)
00304         },
00305         leadingIcon = {
00306             Icon(Icons.Default.Search, contentDescription = null)
00307         },
00308         singleLine = true,
00309         textStyle = androidx.compose.material3.LocalTextStyle.current.copy(fontSize =
textSize),
00310         shape = RoundedCornerShape(16.dp),
00311         colors = OutlinedTextFieldDefaults.colors(
00312             focusedBorderColor = MaterialTheme.colorScheme.outline,
00313             unfocusedBorderColor = MaterialTheme.colorScheme.outline,
00314             focusedContainerColor = MaterialTheme.colorScheme.surface,

```

```

00315         unfocusedContainerColor = MaterialTheme.colorScheme.surface,
00316         cursorColor = BrandGold
00317     )
00318 )
00319
00320     Spacer(modifier = Modifier.height(4.dp))
00321
00322     Row(
00323         modifier = Modifier
00324             .fillMaxWidth()
00325             .padding(horizontal = 16.dp),
00326         verticalAlignment = Alignment.CenterVertically
00327     ) {
00328         Text(
00329             text = strings.resultsCount.format(visibleCars.size),
00330             modifier = Modifier.weight(1f),
00331             color = MaterialTheme.colorScheme.onBackground,
00332             fontWeight = FontWeight.SemiBold
00333         )
00334         TextButton(onClick = {
00335             query = ""
00336             filters = SearchFilters()
00337             sortOption = SortOption.NAME_ASC
00338             geocodedCenter = null
00339             clusterCenter = null
00340             clusterRadiusKm = null
00341         }) {
00342             Text(strings.resetFilters)
00343         }
00344     }
00345 }
00346
00347 if (showFilters) {
00348     FiltersCard(
00349         strings = strings,
00350         filters = filters,
00351         onFiltersChange = { filters = it }
00352     )
00353     Spacer(modifier = Modifier.height(4.dp))
00354 }
00355
00356 if (activeFilters.isNotEmpty()) {
00357     ActiveFiltersRow(
00358         filters = activeFilters,
00359         onRemove = { key ->
00360             when (key) {
00361                 ActiveFilterKey.QUERY -> query = ""
00362                 ActiveFilterKey.BRAND -> filters = filters.copy(brand = null)
00363                 ActiveFilterKey.MODEL -> filters = filters.copy(modelQuery = "")
00364                 ActiveFilterKey.PRICE_RANGE -> filters = filters.copy(minPrice = "", maxPrice
= "")
00365                 ActiveFilterKey.YEAR_RANGE -> filters = filters.copy(minYear = "", maxYear =
""),
00366                 ActiveFilterKey.MAX_MILEAGE -> filters = filters.copy(maxMileage = "")
00367                 ActiveFilterKey.FUEL -> filters = filters.copy(fuelType = null)
00368                 ActiveFilterKey.TRANSMISSION -> filters = filters.copy(transmission = null)
00369                 ActiveFilterKey.LOCATION -> {
00370                     filters = filters.copy(location = "", radiusKm = null)
00371                     geocodedCenter = null
00372                 }
00373                 ActiveFilterKey.CLUSTER_AREA -> {
00374                     clusterCenter = null
00375                     clusterRadiusKm = null
00376                 }
00377             }
00378         }
00379     )
00380     Spacer(modifier = Modifier.height(4.dp))
00381 }
00382
00383 if (showMapView) {
00384     Spacer(modifier = Modifier.height(8.dp))
00385     CarMapView(
00386         cars = visibleCars,
00387         onSingleCarTap = onCarClick,
00388         onClusterTap = { clusterCars ->
00389             val avgLat = clusterCars.map { it.latitude }.average()
00390             val avgLng = clusterCars.map { it.longitude }.average()
00391             val maxDistKm = clusterCars.maxOf {
00392                 haversineKm(avgLat, avgLng, it.latitude, it.longitude)
00393             }
00394             clusterCenter = Pair(avgLat, avgLng)
00395             clusterRadiusKm = maxOf(maxDistKm * 1.5, 1.0)
00396             showMapView = false
00397         },
00398         modifier = Modifier
00399             .weight(1f)

```

```

00400         .fillMaxWidth()
00401     )
00402 } else {
00403     LazyColumn(
00404         modifier = Modifier.weight(1f),
00405         verticalArrangement = Arrangement.spacedBy(14.dp)
00406     ) {
00407         if (visibleCars.isEmpty()) {
00408             item {
00409                 Card(
00410                     modifier = Modifier
00411                         .fillMaxWidth()
00412                         .padding(horizontal = 16.dp),
00413                     shape = RoundedCornerShape(18.dp),
00414                     colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface)
00415                 ) {
00416                     Text(
00417                         text = strings.noSearchResults,
00418                         modifier = Modifier.padding(20.dp),
00419                         color = Slate400
00420                     )
00421                 }
00422             }
00423         } else {
00424             items(visibleCars) { car ->
00425                 val carKey = "${car.id}|${car.title}"
00426                 ListingCard(
00427                     item = ListingCardData(
00428                         title = car.title,
00429                         year = car.year,
00430                         mileageText = car.mileageText.withSuffix(strings.unitKm),
00431                         fuelText = localizeFuelType(car.fuelText, strings),
00432                         bodyTypeText = localizeGearboxType(car.gearboxText, strings),
00433                         driveTypeText = car.engineCapacity.withSuffix(strings.unitCm3),
00434                         locationText = car.locationText,
00435                         priceText = car.priceText.withSuffix(strings.unitCurrency),
00436                         isFavorite = favoriteCars.contains(carKey),
00437                         coverImageUrl = car.imageUrls.firstOrNull(),
00438                         onFavoriteClick = { onFavoriteToggle(carKey) }
00439                     ),
00440                     modifier = Modifier
00441                         .padding(horizontal = 16.dp)
00442                         .clickable { onCarClick(car) }
00443                 )
00444             }
00445         }
00446         item { Spacer(modifier = Modifier.height(10.dp)) }
00447     }
00448 }
00449 }
00450 }
00451
00452 @Composable
00453 private fun ActiveFiltersRow(filters: List<ActiveFilterItem>, onRemove: (ActiveFilterKey) -> Unit) {
00454     LazyRow(
00455         modifier = Modifier.fillMaxWidth(),
00456         contentPadding = androidx.compose.foundation.layout.PaddingValues(horizontal = 16.dp),
00457         horizontalArrangement = Arrangement.spacedBy(8.dp)
00458     ) {
00459         items(filters) { filter ->
00460             Surface(
00461                 shape = RoundedCornerShape(999.dp),
00462                 color = MaterialTheme.colorScheme.surfaceVariant
00463             ) {
00464                 Row(verticalAlignment = Alignment.CenterVertically) {
00465                     Text(
00466                         text = filter.label,
00467                         modifier = Modifier.padding(start = 12.dp, top = 6.dp, bottom = 6.dp),
00468                         color = MaterialTheme.colorScheme.onSurfaceVariant,
00469                         fontSize = 12.sp,
00470                         maxLines = 1,
00471                         overflow = TextOverflow.Ellipsis
00472                     )
00473                     IconButton(onClick = { onRemove(filter.key) }) {
00474                         Icon(Icons.Default.Close, null, tint =
MaterialTheme.colorScheme.onSurfaceVariant)
00475                     }
00476                 }
00477             }
00478         }
00479     }
00480 }
00481
00482 @Composable
00483 private fun FiltersCard(strings: AppStrings, filters: SearchFilters, onFiltersChange: (SearchFilters)
-> Unit) {

```

```

00484     Card(
00485         modifier = Modifier.fillMaxWidth().padding(horizontal = 16.dp),
00486         shape = RoundedCornerShape(20.dp),
00487         colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)
00488     ) {
00489         Column(modifier = Modifier.padding(14.dp), verticalArrangement = Arrangement.spacedBy(10.dp))
00490     {
00491         Text(text = strings.filters, color = MaterialTheme.colorScheme.onSurface, fontWeight =
00492         FontWeight.Bold, fontSize = 17.sp)
00493         LabeledInput(label = strings.brand, value = filters.brand ?: "", onValueChange = {
00494         onFiltersChange(filters.copy(brand = it.ifBlank { null }))) }, placeholder = "np. BMW")
00495         LabeledInput(label = strings.model, value = filters.modelQuery, onValueChange = {
00496         onFiltersChange(filters.copy(modelQuery = it)) }, placeholder = "np. M4")
00497         Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
00498             LabeledInput(label = strings.minPrice, value = filters.minPrice, onValueChange = {
00499             onFiltersChange(filters.copy(minPrice = it.filter(Char::isDigit))) }, modifier = Modifier.weight(1f))
00500             LabeledInput(label = strings.maxPrice, value = filters.maxPrice, onValueChange = {
00501             onFiltersChange(filters.copy(maxPrice = it.filter(Char::isDigit))) }, modifier = Modifier.weight(1f))
00502         }
00503         Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
00504             LabeledInput(label = strings.minYear, value = filters.minYear, onValueChange = {
00505             onFiltersChange(filters.copy(minYear = it.filter(Char::isDigit).take(4))) }, modifier =
00506             Modifier.weight(1f))
00507             LabeledInput(label = strings.maxYear, value = filters.maxYear, onValueChange = {
00508             onFiltersChange(filters.copy(maxYear = it.filter(Char::isDigit).take(4))) }, modifier =
00509             Modifier.weight(1f))
00510         }
00511         LabeledInput(label = strings.maxMileage, value = filters.maxMileage, onValueChange = {
00512         onFiltersChange(filters.copy(maxMileage = it.filter(Char::isDigit))) }, placeholder = "np. 60000")
00513         LabeledInput(label = strings.location, value = filters.location, onValueChange = {
00514         onFiltersChange(filters.copy(location = it, radiusKm = if (it.isBlank()) null else
00515         filters.radiusKm)) }, placeholder = "np. Warszawa")
00516         if (filters.location.isNotBlank()) {
00517             Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
00518                 Text(
00519                     text = strings.searchRadius,
00520                     color = MaterialTheme.colorScheme.onSurface,
00521                     fontWeight = FontWeight.Medium
00522                 )
00523                 LazyRow(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
00524                     val radiusOptions = listOf<Int?>(null, 25, 50, 100, 200)
00525                     items(radiusOptions.size) { i ->
00526                         val km = radiusOptions[i]
00527                         FilterChip(
00528                             selected = filters.radiusKm == km,
00529                             onClick = {
00530                                 onFiltersChange(filters.copy(radiusKm = if (filters.radiusKm ==
00531                                 km) null else km))
00532                             },
00533                             label = { Text(if (km == null) strings.radiusAny else "$km km") }
00534                         )
00535                     }
00536                 }
00537             }
00538         }
00539         SingleSelectChipRow(
00540             label = strings.fuelType,
00541             options = listOf(strings.fuelPetrol, strings.fuelDiesel, strings.fuelElectric,
00542             strings.fuelHybrid),
00543             selected = filters.fuelType,
00544             onSelectionChange = { onFiltersChange(filters.copy(fuelType = it)) }
00545         )
00546         SingleSelectChipRow(
00547             label = strings.transmission,
00548             options = listOf(strings.transmissionAutomatic, strings.transmissionManual),
00549             selected = filters.transmission,
00550             onSelectionChange = { onFiltersChange(filters.copy(transmission = it)) }
00551         )
00552     }
00553 }
00554 @Composable
00555 private fun SingleSelectChipRow(label: String, options: List<String>, selected: String?,
00556 onSelectionChange: (String?) -> Unit) {
00557     Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
00558         Text(text = label, color = MaterialTheme.colorScheme.onSurface, fontWeight =
00559         FontWeight.Medium)
00560         LazyRow(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
00561             items(options) { option ->
00562                 FilterChip(

```

```

00555         selected = selected == option,
00556         onClick = { if (selected == option) onSelectionChange(null) else
onSelectionChange(option) },
00557         label = { Text(option) },
00558         leadingIcon = { if (selected == option) { Icon(Icons.Default.KeyboardArrowDown,
null, tint = MaterialTheme.colorScheme.primary) } }
00559     )
00560 }
00561 }
00562 }
00563 }
00564
00565 @OptIn(ExperimentalMaterial3Api::class)
00566 @Composable
00567 private fun LabeledInput(label: String, value: String, onValueChange: (String) -> Unit, modifier:
Modifier = Modifier, placeholder: String = "") {
00568     OutlinedTextField(
00569         value = value, onValueChange = onValueChange, modifier = modifier.fillMaxWidth(), label = {
Text(label) },
00570         placeholder = { if (placeholder.isNotEmpty()) { Text(placeholder) } }, singleLine = true,
00571         shape = RoundedCornerShape(12.dp),
00572         colors = OutlinedTextFieldDefaults.colors(
            focusedBorderColor = MaterialTheme.colorScheme.outline, unfocusedBorderColor =
MaterialTheme.colorScheme.outline,
00573         focusedContainerColor = MaterialTheme.colorScheme.surface, unfocusedContainerColor =
MaterialTheme.colorScheme.surface
00574     )
00575 )
00576 }
00577
00578 private data class SortLabel(val value: SortOption, val label: String)
00579
00580 private fun sortOptions(strings: AppStrings): List<SortLabel> = listOf(
00581     SortLabel(SortOption.NAME_ASC, strings.sortNameAsc), SortLabel(SortOption.NAME_DESC,
strings.sortNameDesc),
00582     SortLabel(SortOption.PRICE_ASC, strings.sortPriceAsc), SortLabel(SortOption.PRICE_DESC,
strings.sortPriceDesc),
00583     SortLabel(SortOption.YEAR_DESC, strings.sortYearDesc), SortLabel(SortOption.MILEAGE_ASC,
strings.sortMileageAsc)
00584 )
00585
00586 private fun sortComparator(option: SortOption): Comparator<CarAd> = when (option) {
00587     SortOption.NAME_ASC -> compareBy { it.title.lowercase(Locale.getDefault()) }
00588     SortOption.NAME_DESC -> compareByDescending { it.title.lowercase(Locale.getDefault()) }
00589     SortOption.PRICE_ASC -> compareBy { it.priceText.filter(Char::isDigit).toIntOrNull() ?: 0 }
00590     SortOption.PRICE_DESC -> compareByDescending { it.priceText.filter(Char::isDigit).toIntOrNull() ?:
0 }
00591     SortOption.YEAR_DESC -> compareByDescending { it.year.toIntOrNull() ?: 0 }
00592     SortOption.MILEAGE_ASC -> compareBy { it.mileageText.filter(Char::isDigit).toIntOrNull() ?: 0 }
00593 }
00594
00595 private fun haversineKm(lat1: Double, lon1: Double, lat2: Double, lon2: Double): Double {
00596     val R = 6371.0
00597     val dLat = Math.toRadians(lat2 - lat1)
00598     val dLon = Math.toRadians(lon2 - lon1)
00599     val a = sin(dLat / 2).pow(2) + cos(Math.toRadians(lat1)) * cos(Math.toRadians(lat2)) * sin(dLon /
2).pow(2)
00600     return R * 2 * asin(sqrt(a))
00601 }
00602
00603 private fun CarAd.matches(
00604     query: String,
00605     filters: SearchFilters,
00606     geocodedCenter: Pair<Double, Double>?,
00607     clusterCenter: Pair<Double, Double>?,
00608     clusterRadiusKm: Double?
00609 ): Boolean {
00610     val normalizedQuery = query.trim()
00611     val minPrice = filters.minPrice.toIntOrNull()
00612     val maxPrice = filters.maxPrice.toIntOrNull()
00613     val minYear = filters.minYear.toIntOrNull()
00614     val maxYear = filters.maxYear.toIntOrNull()
00615     val maxMileage = filters.maxMileage.toIntOrNull()
00616
00617     val carPrice = priceText.filter(Char::isDigit).toIntOrNull() ?: 0
00618     val carYear = year.toIntOrNull() ?: 0
00619     val carMileage = mileageText.filter(Char::isDigit).toIntOrNull() ?: 0
00620
00621     val queryMatch = normalizedQuery.isBlank() || title.contains(normalizedQuery, ignoreCase = true)
00622
00623     val useClusterFilter = clusterCenter != null && clusterRadiusKm != null
00624     val useGeoFilter = geocodedCenter != null && filters.radiusKm != null
00625
00626     val locationMatch = when {
00627         useClusterFilter -> {
00628             if (latitude == 0.0 && longitude == 0.0) false
00629             else haversineKm(clusterCenter!!.first, clusterCenter.second, latitude, longitude) <=

```

```

        clusterRadiusKm!!
00630     }
00631     useGeoFilter -> {
00632         if (latitude == 0.0 && longitude == 0.0) false
00633         else haversineKm(geocodedCenter!!.first, geocodedCenter.second, latitude, longitude) <=
filters.radiusKm!!
00634     }
00635     filters.location.isNotBlank() -> locationText.contains(filters.location, ignoreCase = true)
00636     else -> true
00637 }
00638
00639     return queryMatch &&
00640         locationMatch &&
00641         (filters.brand.isNullOrBlank() || title.contains(filters.brand, ignoreCase = true)) &&
00642         (filters.modelQuery.isBlank() || title.contains(filters.modelQuery, ignoreCase = true)) &&
00643         (filters.fuelType == null || fuelTypeKey(fuelText) == fuelTypeKey(filters.fuelType)) &&
00644         (filters.transmission == null || gearboxTypeKey(gearboxText) ==
gearboxTypeKey(filters.transmission)) &&
00645         (minPrice == null || carPrice >= minPrice) &&
00646         (maxPrice == null || carPrice <= maxPrice) &&
00647         (minYear == null || carYear >= minYear) &&
00648         (maxYear == null || carYear <= maxYear) &&
00649         (maxMileage == null || carMileage <= maxMileage)
00650 }
00651
00652 private fun buildActiveFilterItems(
00653     query: String,
00654     filters: SearchFilters,
00655     strings: AppStrings,
00656     clusterCenter: Pair<Double, Double>?,
00657     clusterRadiusKm: Double?
00658 ): List<ActiveFilterItem> {
00659     val labels = mutableListOf<ActiveFilterItem>()
00660     if (query.isNotBlank()) labels += ActiveFilterItem(ActiveFilterKey.QUERY, "${strings.search}:
${query.trim()}")
00661     if (!filters.brand.isNullOrBlank()) labels += ActiveFilterItem(ActiveFilterKey.BRAND,
"${strings.brand}: ${filters.brand.trim()}")
00662     if (filters.modelQuery.isNotBlank()) labels += ActiveFilterItem(ActiveFilterKey.MODEL,
"${strings.model}: ${filters.modelQuery.trim()}")
00663     if (filters.minPrice.isNotBlank() || filters.maxPrice.isNotBlank()) {
00664         labels += ActiveFilterItem(ActiveFilterKey.PRICE_RANGE,
"${strings.minPrice}/${strings.maxPrice}: ${filters.minPrice.ifBlank { "-" }
}-${filters.maxPrice.ifBlank { "-" }}")
00665     }
00666     if (filters.minYear.isNotBlank() || filters.maxYear.isNotBlank()) {
00667         labels += ActiveFilterItem(ActiveFilterKey.YEAR_RANGE, "${strings.minYear}/${strings.maxYear}:
${filters.minYear.ifBlank { "-" }}-${filters.maxYear.ifBlank { "-" }}")
00668     }
00669     if (filters.maxMileage.isNotBlank()) labels += ActiveFilterItem(ActiveFilterKey.MAX_MILEAGE,
"${strings.maxMileage}: ${filters.maxMileage}")
00670     if (!filters.fuelType.isNullOrBlank()) labels += ActiveFilterItem(ActiveFilterKey.FUEL,
"${strings.fuelType}: ${filters.fuelType}")
00671     if (!filters.transmission.isNullOrBlank()) labels +=
ActiveFilterItem(ActiveFilterKey.TRANSMISSION, "${strings.transmission}: ${filters.transmission}")
00672     if (filters.location.isNotBlank()) {
00673         val label = if (filters.radiusKm != null)
00674             "${strings.location}: ${filters.location.trim()} (${filters.radiusKm} km)"
00675         else
00676             "${strings.location}: ${filters.location.trim()}"
00677         labels += ActiveFilterItem(ActiveFilterKey.LOCATION, label)
00678     }
00679     if (clusterCenter != null && filters.location.isBlank()) {
00680         labels += ActiveFilterItem(
00681             ActiveFilterKey.CLUSTER_AREA,
00682             "≤ ${clusterRadiusKm?.toInt() ?: "?"} km"
00683         )
00684     }
00685     return labels
00686 }
00687
00688 private fun String.withSuffix(suffix: String): String {
00689     val value = trim()
00690     return if (value.isEmpty()) "" else "$value $suffix"
00691 }

```

4.49 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/screens/settings/SettingsScreen.kt

Ekran ustawień: motyw, język, powiadomienia i wylogowanie.

Pakiety

- package [context](#)

Zmienne

- Object [context.showLanguageDialog](#)
- Object [context.showThemeDialog](#)
- Object [context.openNotificationSettings](#)
- Object [context.intent](#)
- Object [context.label](#)
- Object [context.themeLabel](#)

4.49.1 Opis szczegółowy

Ekran ustawień: motyw, język, powiadomienia i wylogowanie.

Definicja w pliku [SettingsScreen.kt](#).

4.50 SettingsScreen.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file SettingsScreen.kt
00003  * @brief Ekran ustawień: motyw, język, powiadomienia i wylogowanie.
00004  */
00005 package com.example.otomotuzplus.ui.screens.settings
00006
00007 import android.content.Intent
00008 import androidx.compose.foundation.layout.*
00009 import androidx.compose.material.icons.Icons
00010 import androidx.compose.material.icons.automirrored.filled.ArrowBack
00011 import androidx.compose.material.icons.filled.BugReport
00012 import androidx.compose.material.icons.filled.Language
00013 import androidx.compose.material.icons.filled.Notifications
00014 import androidx.compose.material.icons.filled.Palette
00015 import androidx.compose.material3.*
00016 import androidx.compose.runtime.*
00017 import androidx.compose.ui.Modifier
00018 import androidx.compose.ui.graphics.Color
00019 import androidx.compose.ui.platform.LocalContext
00020 import androidx.compose.ui.unit.dp
00021 import com.example.otomotuzplus.LoginActivity
00022 import com.example.otomotuzplus.data.ThemeMode
00023 import com.example.otomotuzplus.ui.components.SettingsClickItem
00024 import com.example.otomotuzplus.ui.models.AppStrings
00025 import com.example.otomotuzplus.utils.NotificationHelper
00026 import com.google.firebase.auth.FirebaseAuth
00027 import com.example.otomotuzplus.ui.components.AdmobBanner
00028
00029 /**
00030  * Ekran ustawień dostępny z [ProfileScreen].
00031  *
00032  * Oferuje:
00033  * - **Motyw** -- otwiera [AlertDialog] do wyboru [ThemeMode.LIGHT], [ThemeMode.DARK]
00034  *   lub [ThemeMode.SYSTEM].
00035  * - **Język** -- otwiera [AlertDialog] do wyboru `"Polski"` lub `"English"`.
00036  * - **Powiadomienia** -- przechodzi do ustawień powiadomień systemu dla tej aplikacji.
00037  * - **Wyślij testowe powiadomienie** -- natychmiast wysyła lokalne powiadomienie dla QA.
00038  * - **Wyloguj** -- wylogowuje przez [FirebaseAuth] i nawiguje z powrotem do [LoginActivity].
00039  *
00040  * [AdmobBanner] jest umieszczony na dole obszaru treści nad przyciskiem wylogowania.
00041  *
00042  * @param onBackPressed Callback zamykający ten ekran i wracający do [ProfileScreen].
00043  * @param themeMode Aktualnie aktywny [ThemeMode].
00044  * @param onThemeChange Callback do utrwalenia i zastosowania nowego [ThemeMode].
00045  * @param currentLanguage Aktualnie aktywny identyfikator języka.
00046  * @param onLanguageChange Callback do utrwalenia i zastosowania nowego języka.
00047  * @param strings Aktywne [AppStrings] dla zlokalizowanych etykiet.
```

```

00048 */
00049 @OptIn(ExperimentalMaterial3Api::class)
00050 @Composable
00051 fun SettingsScreen(
00052     onBackPressed: () -> Unit,
00053     themeMode: ThemeMode,
00054     onThemeChange: (ThemeMode) -> Unit,
00055     currentLanguage: String,
00056     onLanguageChange: (String) -> Unit,
00057     strings: AppStrings
00058 ) {
00059     val context = LocalContext.current
00060     var showLanguageDialog by remember { mutableStateOf(false) }
00061     var showThemeDialog by remember { mutableStateOf(false) }
00062     val openNotificationSettings = {
00063         val intent = Intent().apply {
00064             action = android.provider.Settings.ACTION_APP_NOTIFICATION_SETTINGS
00065             putExtra(android.provider.Settings.EXTRA_APP_PACKAGE, context.packageName)
00066         }
00067         context.startActivity(intent)
00068     }
00069
00070     if (showLanguageDialog) {
00071         AlertDialog(
00072             onDismissRequest = { showLanguageDialog = false },
00073             title = { Text(strings.chooseLanguage, color = MaterialTheme.colorScheme.onSurface) },
00074             text = {
00075                 Column {
00076                     listOf("Polski", "English").forEach { lang ->
00077                         TextButton(
00078                             onClick = {
00079                                 onLanguageChange(lang)
00080                                 showLanguageDialog = false
00081                             },
00082                             modifier = Modifier.fillMaxWidth()
00083                         ) {
00084                             Text(
00085                                 text = lang,
00086                                 modifier = Modifier.fillMaxWidth(),
00087                                 color = MaterialTheme.colorScheme.onSurface
00088                             )
00089                         }
00090                     }
00091                 }
00092             },
00093             confirmButton = {}
00094         )
00095     }
00096
00097     if (showThemeDialog) {
00098         AlertDialog(
00099             onDismissRequest = { showThemeDialog = false },
00100             title = { Text(strings.chooseTheme, color = MaterialTheme.colorScheme.onSurface) },
00101             text = {
00102                 Column {
00103                     ThemeMode.entries.forEach { mode ->
00104                         val label = when (mode) {
00105                             ThemeMode.SYSTEM -> strings.themeSystem
00106                             ThemeMode.LIGHT -> strings.themeLight
00107                             ThemeMode.DARK -> strings.themeDark
00108                         }
00109                         TextButton(
00110                             onClick = {
00111                                 onThemeChange(mode)
00112                                 showThemeDialog = false
00113                             },
00114                             modifier = Modifier.fillMaxWidth()
00115                         ) {
00116                             Text(
00117                                 text = label,
00118                                 modifier = Modifier.fillMaxWidth(),
00119                                 color = MaterialTheme.colorScheme.onSurface
00120                             )
00121                         }
00122                     }
00123                 }
00124             },
00125             confirmButton = {}
00126         )
00127     }
00128
00129     Scaffold(
00130         topBar = {
00131             TopAppBar(
00132                 title = { Text(strings.settings) },
00133                 navigationIcon = {
00134                     IconButton(onClick = onBackPressed) {

```

```

00135         Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Back")
00136     },
00137     ),
00138     colors = TopAppBarDefaults.topAppBarColors(
00139         containerColor = MaterialTheme.colorScheme.background,
00140         titleContentColor = MaterialTheme.colorScheme.onBackground,
00141         navigationIconContentColor = MaterialTheme.colorScheme.onBackground
00142     )
00143 )
00144 },
00145 containerColor = MaterialTheme.colorScheme.background
00146 ) { innerPadding ->
00147     Column(
00148         modifier = Modifier
00149             .padding(innerPadding)
00150             .fillMaxSize()
00151             .padding(16.dp)
00152     ) {
00153         Text(strings.appPreferences, color = Color.Gray)
00154
00155         Spacer(modifier = Modifier.height(16.dp))
00156
00157         val themeLabel = when (themeMode) {
00158             ThemeMode.SYSTEM -> strings.themeSystem
00159             ThemeMode.LIGHT -> strings.themeLight
00160             ThemeMode.DARK -> strings.themeDark
00161         }
00162
00163         SettingsClickItem(
00164             title = "${strings.theme} ($themeLabel)",
00165             icon = Icons.Default.Palette,
00166             onClick = { showThemeDialog = true }
00167         )
00168
00169         SettingsClickItem(
00170             title = "${strings.language} ($currentLanguage)",
00171             icon = Icons.Default.Language,
00172             onClick = { showLanguageDialog = true }
00173         )
00174
00175         SettingsClickItem(
00176             title = strings.notifications,
00177             icon = Icons.Default.Notifications,
00178             onClick = { openNotificationSettings() }
00179         )
00180
00181         SettingsClickItem(
00182             title = strings.dealTestNotification,
00183             icon = Icons.Default.BugReport,
00184             onClick = {
00185                 NotificationHelper.sendNotification(
00186                     context,
00187                     strings.dealNotification,
00188                     strings.dealNotificationDescription
00189                 )
00190             }
00191         )
00192
00193         Spacer(modifier = Modifier.weight(1f))
00194         AdmobBanner()
00195         Spacer(modifier = Modifier.height(24.dp))
00196
00197         Button(
00198             onClick = {
00199                 FirebaseAuth.getInstance().signOut()
00200                 val intent = Intent(context, LoginActivity::class.java).apply {
00201                     flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
00202                 }
00203                 context.startActivity(intent)
00204             },
00205             modifier = Modifier.fillMaxWidth(),
00206             colors = ButtonDefaults.buttonColors(containerColor = Color.Red.copy(alpha = 0.8f))
00207         ) {
00208             Text(strings.logout, color = Color.White)
00209         }
00210     }
00211 }
00212 }

```

4.51 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/ui/theme/Color.kt

Paleta kolorów Material3 aplikacji OtomotUZplus.

Pakiety

- package [BrandGold](#)

Zmienne

- Object [BrandGold.ActionBlue](#)
- Object [BrandGold.DarkText](#)
- Object [BrandGold.DarkSlate950](#)
- Object [BrandGold.DarkSlate900](#)
- Object [BrandGold.DarkSlate800](#)
- Object [BrandGold.DarkSlate700](#)
- Object [BrandGold.Slate400](#)
- Object [BrandGold.Slate300](#)
- Object [BrandGold.LightBackground](#)
- Object [BrandGold.Slate100](#)
- Object [BrandGold.Slate200](#)
- Object [BrandGold.SuccessGreen](#)
- Object [BrandGold.ErrorRed](#)
- Object [BrandGold.WarningOrange](#)

4.51.1 Opis szczegółowy

Paleta kolorów Material3 aplikacji OtomotUZplus.

Definicja w pliku [Color.kt](#).

4.52 Color.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file Color.kt
00003  * @brief Paleta kolorów Material3 aplikacji OtomotUZplus.
00004  */
00005 package com.example.otomotuzplus.ui.theme
00006
00007 import androidx.compose.ui.graphics.Color
00008
00009 /**
00010  * OTOMOTUZ Design System Colors (Aligned with Web Version)
00011  */
00012
00013 // --- Brand Colors ---
00014 val BrandGold = Color(0xFFE6BD37)
00015 val ActionBlue = Color(0xFF4E6AF0)
00016 val DarkText = Color(0xFF1E1E1E)
00017
00018 // --- Dark Mode Palette (Slate Scale) ---
00019 val DarkSlate950 = Color(0xFF030712) // Page Background
00020 val DarkSlate900 = Color(0xFF0F172A) // Card Backgrounds
00021 val DarkSlate800 = Color(0xFF1E293B) // Surface Elements
00022 val DarkSlate700 = Color(0xFF334155) // Borders
00023 val Slate400 = Color(0xFF94A3B8) // Secondary Text
00024 val Slate300 = Color(0xFFCBD5E1) // Tertiary Text
00025
00026 // --- Light Mode Palette ---
00027 val LightBackground = Color(0xFFF8FAFC)
00028 val Slate100 = Color(0xFFF1F5F9)
00029 val Slate200 = Color(0xFFE2E8F0)
00030
00031 // --- Semantic Colors ---
00032 val SuccessGreen = Color(0xFF34A853)
00033 val ErrorRed = Color(0xFFEA4335)
00034 val WarningOrange = Color(0xFFFFB300)
```

4.53 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/ui/theme/Theme.kt

Konfiguracja motywu Material3 z obsługą jasnego, ciemnego i systemowego trybu.

Pakiety

- package [DarkColorScheme](#)

Zmienne

- Object [DarkColorScheme.LightColorScheme](#)
- Object [DarkColorScheme.colorScheme](#)
- Object [DarkColorScheme.view](#)
- Object [DarkColorScheme.window](#)

4.53.1 Opis szczegółowy

Konfiguracja motywu Material3 z obsługą jasnego, ciemnego i systemowego trybu.

Definicja w pliku [Theme.kt](#).

4.54 Theme.kt

[Idź do dokumentacji tego pliku.](#)

```
00001 /**
00002  * @file Theme.kt
00003  * @brief Konfiguracja motywu Material3 z obsługą jasnego, ciemnego i systemowego trybu.
00004  */
00005 package com.example.otomotuzplus.ui.theme
00006
00007 import android.app.Activity
00008 import androidx.compose.foundation.isSystemInDarkTheme
00009 import androidx.compose.material3.MaterialTheme
00010 import androidx.compose.material3.darkColorScheme
00011 import androidx.compose.material3.lightColorScheme
00012 import androidx.compose.runtime.Composable
00013 import androidx.compose.ui.graphics.Color
00014 import androidx.compose.ui.platform.LocalView
00015 import androidx.core.view.WindowCompat
00016
00017 private val DarkColorScheme = darkColorScheme(
00018     primary = BrandGold,
00019     secondary = DarkSlate900,
00020     tertiary = ActionBlue,
00021     background = DarkSlate950,
00022     surface = DarkSlate900,
00023     onPrimary = Color.Black,
00024     onSecondary = Color.White,
00025     onTertiary = Color.White,
00026     onBackground = Slate300,
00027     onSurface = Slate300,
00028     surfaceVariant = DarkSlate800,
00029     onSurfaceVariant = Slate400,
00030     outline = DarkSlate700
00031 )
00032
00033 private val LightColorScheme = lightColorScheme(
00034     primary = BrandGold,
00035     secondary = Slate100,
00036     tertiary = ActionBlue,
00037     background = LightBackground,
00038     surface = Color.White,
00039     onPrimary = Color.Black,
00040     onSecondary = Color.White,
```

```

00041     onTertiary = Color.White,
00042     onBackground = DarkText,
00043     onSurface = DarkText,
00044     surfaceVariant = Slate100,
00045     onSurfaceVariant = Slate400,
00046     outline = Slate200
00047 )
00048
00049 @Composable
00050 fun OtomotUZplusTheme(
00051     darkTheme: Boolean = isSystemInDarkTheme(),
00052     content: @Composable () -> Unit
00053 ) {
00054     val colorScheme = if (darkTheme) DarkColorScheme else LightColorScheme
00055     val view = LocalView.current
00056     if (!view.isInEditMode) {
00057         val window = (view.context as Activity).window
00058         WindowCompat.getInsetsController(window, view).isAppearanceLightStatusBars = !darkTheme
00059     }
00060
00061     MaterialTheme(
00062         colorScheme = colorScheme,
00063         typography = Typography,
00064         content = content
00065     )
00066 }

```

4.55 Dokumentacja pliku

app/src/main/java/com/example/otomotuzplus/ui/theme/Type.kt

Typografia Material3 aplikacji OtomotUZplus.

Pakiety

- package [Typography](#)

4.55.1 Opis szczegółowy

Typografia Material3 aplikacji OtomotUZplus.

Definicja w pliku [Type.kt](#).

4.56 Type.kt

[Idź do dokumentacji tego pliku.](#)

```

00001 /**
00002  * @file Type.kt
00003  * @brief Typografia Material3 aplikacji OtomotUZplus.
00004  */
00005 package com.example.otomotuzplus.ui.theme
00006
00007 import androidx.compose.material3.Typography
00008 import androidx.compose.ui.text.TextStyle
00009 import androidx.compose.ui.text.font.FontFamily
00010 import androidx.compose.ui.text.font.FontWeight
00011 import androidx.compose.ui.unit.sp
00012
00013 // Set of Material typography styles to start with
00014 val Typography = Typography(
00015     bodyLarge = TextStyle(
00016         fontFamily = FontFamily.Default,
00017         fontWeight = FontWeight.Normal,
00018         fontSize = 16.sp,
00019         lineHeight = 24.sp,
00020         letterSpacing = 0.5.sp
00021     )
00022     /* Other default text styles to override

```

```

00023         titleLarge = TextStyle(
00024             fontFamily = FontFamily.Default,
00025             fontWeight = FontWeight.Normal,
00026             fontSize = 22.sp,
00027             lineHeight = 28.sp,
00028             letterSpacing = 0.sp
00029         ),
00030         labelSmall = TextStyle(
00031             fontFamily = FontFamily.Default,
00032             fontWeight = FontWeight.Medium,
00033             fontSize = 11.sp,
00034             lineHeight = 16.sp,
00035             letterSpacing = 0.5.sp
00036         )
00037     */
00038 )

```

4.57 Dokumentacja pliku app/src/main/java/com/example/otomotuzplus/Utils/NotificationHelper.kt

Singleton pomocniczy do tworzenia kanału i wysyłania lokalnych powiadomień.

Pakiety

- package [NotificationHelper](#)

Funkcje

- Object [NotificationHelper.createNotificationChannel](#) (Object context, Object strings)
Identyfikator kanału powiadomień współdzielony z [MyFirebaseMessagingService].
- Object [NotificationHelper.sendNotification](#) (Object context, Object title, Object message)
Wysła lokalne powiadomienie z podanym tytułem i treścią.

4.57.1 Opis szczegółowy

Singleton pomocniczy do tworzenia kanału i wysyłania lokalnych powiadomień.

Definicja w pliku [NotificationHelper.kt](#).

4.58 NotificationHelper.kt

[Idź do dokumentacji tego pliku.](#)

```

00001 /**
00002  * @file NotificationHelper.kt
00003  * @brief Singleton pomocniczy do tworzenia kanału i wysyłania lokalnych powiadomień.
00004  */
00005 package com.example.otomotuzplus.Utils
00006
00007 import android.app.NotificationChannel
00008 import android.app.NotificationManager
00009 import android.app.PendingIntent
00010 import android.content.Context
00011 import android.content.Intent
00012 import android.os.Build
00013 import androidx.core.app.NotificationCompat
00014 import com.example.otomotuzplus.MainActivity
00015 import com.example.otomotuzplus.R
00016 import com.example.otomotuzplus.ui.models.AppStrings
00017 /**
00018  * Singleton pomocniczy do tworzenia kanału powiadomień i wysyłania lokalnych
00019  * powiadomień push.

```

```

00020 *
00021 * Kanał powiadomień ('offers_channel') jest tworzony raz przy uruchomieniu aplikacji
00022 * wewnątrz `MainActivity.onCreate`. Wszystkie powiadomienia wysyłane przez tego pomocnika
00023 * i przez [MyFirebaseMessagingService] współdzielią ten kanał.
00024 */
00025 object NotificationHelper {
00026     /** Identyfikator kanału powiadomień współdzielony z [MyFirebaseMessagingService]. */
00027     const val CHANNEL_ID = "offers_channel"
00028
00029     /**
00030      * Tworzy kanał powiadomień `offers_channel` (tylko Android 8+).
00031      *
00032      * Bezpieczne do wielokrotnego wywoływania -- Android deduplikuje tworzenie kanałów po ID.
00033      *
00034      * @param context Kontekst aplikacji lub Activity.
00035      * @param strings Aktywne [AppStrings] dostarczające zlokalizowaną nazwę kanału.
00036      */
00037     fun createNotificationChannel(context: Context, strings: AppStrings) {
00038         val name = strings.newOffers
00039         val descriptionText = "Powiadomienia z aplikacji OtomotUZplus"
00040         val importance = NotificationManager.IMPORTANCE_DEFAULT
00041         val channel = NotificationChannel(CHANNEL_ID, name, importance).apply {
00042             description = descriptionText
00043         }
00044         val notificationManager = context.getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager
00045         notificationManager.createNotificationChannel(channel)
00046     }
00047
00048     /**
00049      * Wysyła lokalne powiadomienie z podanym tytułem i treścią.
00050      *
00051      * Używa identyfikatora powiadomienia opartego na znaczniku czasu, aby kolejne wywołania
00052      * nie nadpisywały się nawzajem. Dotknięcie powiadomienia uruchamia [MainActivity]
00053      * i czyści stos back.
00054      *
00055      * @param context Kontekst aplikacji lub Activity.
00056      * @param title Ciąg tytułu powiadomienia.
00057      * @param message Tekst treści powiadomienia.
00058      */
00059     fun sendNotification(context: Context, title: String, message: String) {
00060         val intent = Intent(context, MainActivity::class.java).apply {
00061             flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
00062         }
00063
00064         val pendingIntent = PendingIntent.getActivity(
00065             context, 0, intent,
00066             PendingIntent.FLAG_IMMUTABLE
00067         )
00068
00069         val notification = NotificationCompat.Builder(context, CHANNEL_ID)
00070             .setSmallIcon(R.drawable.ic_launcher_foreground)
00071             .setContentTitle(title) // Używamy przekazanego tytułu
00072             .setContentText(message) // Używamy przekazanej treści
00073             .setPriority(NotificationCompat.PRIORITY_DEFAULT)
00074             .setAutoCancel(true)
00075             .setContentIntent(pendingIntent)
00076             .build()
00077
00078         val notificationManager = context.getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager
00079         notificationManager.notify(System.currentTimeMillis().toInt(), notification)
00080     }
00081 }

```


Indeks

app/src/main/AndroidManifest.xml, 4
app/src/main/java/com/example/otomotuzplus/data/FirebaseRepository.kt, 5, 6
app/src/main/java/com/example/otomotuzplus/data/NominalDestination.kt, 9
app/src/main/java/com/example/otomotuzplus/data/PreferenceManager.kt, 10, 11
app/src/main/java/com/example/otomotuzplus/fcm/MyFirebaseMessagingService.kt, 12
app/src/main/java/com/example/otomotuzplus/LoginActivity.kt, 13
app/src/main/java/com/example/otomotuzplus/MainActivity.kt, 14, 15
app/src/main/java/com/example/otomotuzplus/models/CarAd.kt, 20
app/src/main/java/com/example/otomotuzplus/RegisterActivity.kt, 21
app/src/main/java/com/example/otomotuzplus/ui/components/ScreenOptions.kt, 22, 23
app/src/main/java/com/example/otomotuzplus/ui/components/CommonComponents.kt, 23, 24
app/src/main/java/com/example/otomotuzplus/ui/components/ListingCard.kt, 25, 26
app/src/main/java/com/example/otomotuzplus/ui/models/AppStrings.kt, 28
app/src/main/java/com/example/otomotuzplus/ui/models/ListingCatalog.kt, 33
app/src/main/java/com/example/otomotuzplus/ui/models/ServerValueTranslations.kt, 35
app/src/main/java/com/example/otomotuzplus/ui/navigation/NavDestinations.kt, 36, 37
app/src/main/java/com/example/otomotuzplus/ui/screens/add/AddScreen.kt, 37
app/src/main/java/com/example/otomotuzplus/ui/screens/details/AdDetailScreen.kt, 44
app/src/main/java/com/example/otomotuzplus/ui/screens/favorites/FavoritesScreen.kt, 48, 49
app/src/main/java/com/example/otomotuzplus/ui/screens/home/HomeScreen.kt, 50, 51
app/src/main/java/com/example/otomotuzplus/ui/screens/profile/ProfileScreen.kt, 56, 57
app/src/main/java/com/example/otomotuzplus/ui/screens/search/CarClusterOverlay.kt, 60, 61
app/src/main/java/com/example/otomotuzplus/ui/screens/search/CarMapView.kt, 63
app/src/main/java/com/example/otomotuzplus/ui/screens/search/SearchScreen.kt, 66
app/src/main/java/com/example/otomotuzplus/ui/screens/settings/SettingsScreen.kt, 75
app/src/main/java/com/example/otomotuzplus/ui/theme/Color.kt, 77, 78
app/src/main/java/com/example/otomotuzplus/ui/theme/Theme.kt, 78, 79
app/src/main/java/com/example/otomotuzplus/ui/theme/Type.kt, 79, 80
app/src/main/java/com/example/otomotuzplus/utls/NotificationHelper.kt, 89, 91
AppCompatActivity, 4
AppCompatActivity.kt, 4
AppStrings, 4
Dispatchers, 4
Dispatchers.IO, 4
FirebaseMessagingService, 4
FirebaseRepository, 4
Modifier, 4
NotificationHelper, 4
Overlay, 4
ScreenOptions.kt, 22, 23
String, 4
ThemeMode, 4
Unit, 4